A Chair-Readable And Implementation-Ready Companion to Tensor-Train Sequential Filtering and Fixed-Branch Gradients

Technical Note

2026-06-02

Abstract

This note gives a self-contained mathematical companion to tensor-train sequential filtering in the style of Zhao and Cui [4]. It follows the state-space, tensor-train, squared-density, conditional Knothe–Rosenblatt, and preconditioning constructions in their paper, while expanding the intermediate equations and implementation meanings that are often compressed in the original presentation. The second half develops a fixed branch variant: all ranks, points, domains, shifts, and fitting choices are declared before differentiation, so the resulting analytical derivative is the derivative of a named approximate filtering scalar rather than of the adaptive algorithm itself.

Contents

1 Notation And Reading Convention

The purpose of the note is to make the mathematical mechanism readable without asking the reader to reconstruct missing steps from the original paper. The starting point is the tensor-train sequential learning method of Zhao and Cui [4]; tensor trains themselves follow Oseledets [2], and the transport maps used for conditional sampling are Knothe–Rosenblatt maps in the sense of Rosenblatt's triangular transformation [3]. Squared inverse Rosenblatt transport ideas also connect to the work of Cui and Dolgov [1].

Zhao and Cui write the unknown static parameter as θ [4]. The reconstruction below follows that convention when the parameter is part of a posterior density. The fixed-branch derivative section later writes the differentiated external parameter as β, so that the same symbol is not used both for an integration coordinate and for the argument of an approximate likelihood.

The state dimension is m, observation dimension is n, and parameter dimension is d. Thus

$$x_t = (x_{t,1}, \dots, x_{t,m}) \in \mathcal{X} \subseteq \mathbb{R}^m, \qquad y_t = (y_{t,1}, \dots, y_{t,n}) \in \mathcal{Y} \subseteq \mathbb{R}^n,$$

and

$$\theta = (\theta_1, \dots, \theta_d) \in \Theta \subseteq \mathbb{R}^d.$$

For coordinate groups, the paper uses

$$x_{t,<j} = (x_{t,1}, \dots, x_{t,j-1}), \qquad x_{t,>j} = (x_{t,j+1}, \dots, x_{t,m}), \tag{1.1}$$

and similarly

$$x_{t,\le j} = (x_{t,1}, \dots, x_{t,j}), \qquad x_{t,\ge j} = (x_{t,j}, \dots, x_{t,m}). \tag{1.2}$$

The point of this notation is conditional density bookkeeping. A lower triangular map conditions coordinate j on $x_{<j}$; an upper triangular map conditions it on $x_{>j}$.

All densities in the reconstruction are assumed absolutely continuous with respect to the relevant Lebesgue measure. Zhao and Cui [4] use p for normalized posterior densities and π for unnormalized densities. Their approximations are $\widehat{p}$ and $\widehat{\pi}$. When p and $\widehat{p}$ are normalized densities, the Hellinger distance is

$$D_H(p, \widehat{p}) = \left[\frac{1}{2} \int \left(\sqrt{p(r)} - \sqrt{\widehat{p}(r)} \right)^2 \mathrm{d}r \right]^{1/2}. \tag{1.3}$$

This distance appears because the squared-TT method approximates square roots of densities.

If $S : \mathcal{X} \to \mathcal{U}$ is a diffeomorphism and $X \sim p$, then the density of $U = S(X)$ is the pushforward

$$S_{\#}p(u) = p(S^{-1}(u)) \left| \det \nabla S^{-1}(u) \right|. \tag{1.4}$$

If $U \sim \eta$, then the density of $X = S^{-1}(U)$ is the pullback

$$S^{\#}\eta(x) = \eta(S(x)) \left| \det \nabla S(x) \right|. \tag{1.5}$$

These two identities are the entire mathematical engine behind the Knothe–Rosenblatt maps and the preconditioning maps in Section 5. The implementation stores an evaluator for S, an evaluator for S^{-1}, and the relevant log-Jacobian determinant when the map is used as a density transform.

2 Threaded Nonlinear Example Used Throughout The Note

This example is intentionally small enough to hold in one line, but nonlinear enough to show why Gaussian projection alone may be too rigid. Let

$$x_0 \sim N(\mu_0, \sigma_0^2), \qquad x_t = \beta x_{t-1} + \sigma\epsilon_t, \qquad \epsilon_t \sim N(0,1), \tag{2.1}$$

and

$$y_t = \sin(x_t) + \eta_t, \qquad \eta_t \sim N(0, r^2). \tag{2.2}$$

The transition and observation densities are therefore

$$f_\beta(x_t \mid x_{t-1}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_t - \beta x_{t-1})^2}{2\sigma^2}\right], \tag{2.3}$$

and

$$g_\beta(y_t \mid x_t) = \frac{1}{\sqrt{2\pi r^2}} \exp\left[-\frac{(y_t - \sin x_t)^2}{2r^2}\right]. \tag{2.4}$$

At time 1, the exact unnormalized adjacent-state target is

$$q_1(x_1, x_0; \beta) = p_\beta(x_0) f_\beta(x_1 \mid x_0) g_\beta(y_1 \mid x_1). \tag{2.5}$$

The squared-TT approximation fits

$$\phi_1(z_1, z_0; \beta) \approx e^{c_1/2} \sqrt{q_1(\Psi_1(z_1, z_0); \beta) J_1(z_1, z_0)}, \tag{2.6}$$

and defines

$$\widehat{q}_1(z_1, z_0; \beta) = e^{-c_1} \phi_1(z_1, z_0; \beta)^2 + \tau_1 \lambda_1(z_1, z_0). \tag{2.7}$$

The first evidence increment and retained filter are

$$\widehat{Z}_1(\beta) = \int \widehat{q}_1(z_1, z_0; \beta)\, \mathrm{d}z_1\, \mathrm{d}z_0, \qquad \widehat{p}_1(z_1; \beta) = \frac{\int \widehat{q}_1(z_1, z_0; \beta)\, \mathrm{d}z_0}{\widehat{Z}_1(\beta)}. \tag{2.8}$$

At time 2, the same object reappears inside the next target:

$$q_2(x_2, x_1; \beta) = \widehat{p}_1(x_1; \beta) f_\beta(x_2 \mid x_1) g_\beta(y_2 \mid x_2). \tag{2.9}$$

If a bridge $\rho_2(x_2, x_1)$ is used, the preconditioned residual in reference coordinates is

$$q_2^\sharp(u; \beta) = \frac{q_2(T_2^{-1}(u); \beta)}{\rho_2(T_2^{-1}(u))} \eta(u). \tag{2.10}$$

The fixed-branch scalar for the two-step example is

$$\widehat{\ell}_2(\beta; B) = \log \widehat{Z}_1(\beta; B_1) + \log \widehat{Z}_2(\beta; B_2), \tag{2.11}$$

with derivative

$$\partial_\beta \widehat{\ell}_2(\beta; B) = \frac{\partial_\beta \widehat{Z}_1(\beta; B_1)}{\widehat{Z}_1(\beta; B_1)} + \frac{\partial_\beta \widehat{Z}_2(\beta; B_2)}{\widehat{Z}_2(\beta; B_2)}. \tag{2.12}$$

The recurrence table for this example is

location	anchor	role
filtering target	(2.5), (2.9)	Bayes numerator before fitting
squared-TT density	(2.6)–(2.8)	nonnegative fitted approximation
preconditioning	(2.10)	bridge removes dominant geometry
gradient	(2.11)–(2.12)	same saved-branch scalar and derivative.

(2.13)

3 Reader Orientation: Five Mathematical Objects

The Zhao and Cui [4] method is easier to follow if the notation is organized around five objects. They are not five separate algorithms; they are five views of one sequential density approximation:

$$q_t, \qquad \phi_t, \qquad C_{t,k}, \qquad \widehat{Z}_t, \qquad \partial_\beta \log \widehat{Z}_t. \tag{3.1}$$

The first three describe the approximated density. The fourth is the scalar contributed to the likelihood. The fifth is the sensitivity needed by the fixed-branch likelihood specialization.

Object 1: the unnormalized density

Bayes' rule begins with an unnormalized numerator:

$$\text{posterior density} = \frac{\text{likelihood} \times \text{prior}}{\int \text{likelihood} \times \text{prior}}. \tag{3.2}$$

In filtering, the prior at time t is the previous filter propagated through the transition model. A typical adjacent-state target has the form

$$q_t(x_t, x_{t-1}; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t). \tag{3.3}$$

The implementation question is more concrete than the notation suggests: which coordinates are used, and at which fitting points is q_t evaluated? A fixed branch stores

$$Z_{\text{fit}} = \begin{bmatrix} z_1^{(1)} & z_2^{(1)} \\ \vdots & \vdots \\ z_1^{(N)} & z_2^{(N)} \end{bmatrix}, \qquad \text{shape}(Z_{\text{fit}}) = (N, 2), \tag{3.4}$$

and a target vector

$$\widetilde{q}_t^{\text{fit}} = \left(\widetilde{q}_t(z^{(1)}; \beta), \ldots, \widetilde{q}_t(z^{(N)}; \beta) \right)^\top, \qquad \text{shape}(\widetilde{q}_t^{\text{fit}}) = (N,). \tag{3.5}$$

Object 2: the square-root approximation

A direct TT fit of q_t can be negative. Zhao and Cui [4] therefore fit a square root and square it back:

$$\phi_t(z; \beta) \approx e^{c_t/2} \sqrt{\widetilde{q}_t(z; \beta)}, \tag{3.6}$$

$$\widehat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z). \tag{3.7}$$

The fitted values are

$$y_t^{\text{fit}}[j] = e^{c_t/2} \sqrt{\widetilde{q}_t(z^{(j)}; \beta)}, \qquad \text{shape}(y_t^{\text{fit}}) = (N,), \tag{3.8}$$

and the fixed-branch derivative values are

$$\dot{y}_t^{\text{fit}}[j] = e^{c_t/2} \frac{\dot{\widetilde{q}}_t(z^{(j)}; \beta)}{2\sqrt{\widetilde{q}_t(z^{(j)}; \beta)}}. \tag{3.9}$$

Object 3: the tensor-train cores

For two coordinates, the fitted square root can be written

$$\phi_t(z_1, z_2; \beta) = \sum_{r=1}^{R} \left[\sum_{\ell=0}^{p-1} C_1[0, \ell, r] b_1(z_1)[\ell] \right] \left[\sum_{m=0}^{p-1} C_2[r, m, 0] b_2(z_2)[m] \right]. \tag{3.10}$$

The stored arrays are

$$\operatorname{shape}(C_1) = \operatorname{shape}(\dot{C}_1) = (1, p, R), \qquad \operatorname{shape}(C_2) = \operatorname{shape}(\dot{C}_2) = (R, p, 1). \tag{3.11}$$

The branch freezes the fitting rule, not the fitted numerical core values:

$$C_k(\beta; B) = \text{core returned by the fixed fitting equations at } \beta. \tag{3.12}$$

Object 4: the normalizer

The evidence increment is the mass of the declared approximate density:

$$\widehat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t, \qquad R_t(\beta) = \int \phi_t(z; \beta)^2 \, \mathrm{d}z. \tag{3.13}$$

In TT form, this mass is computed by contractions rather than by a full tensor grid. For the two-coordinate case,

$$S_2[r, s] = \sum_{\ell, m} C_2[r, \ell, 0] B_2[\ell, m] C_2[s, m, 0], \tag{3.14}$$

$$R_t = \sum_{r, s, \ell, m} C_1[0, \ell, r] B_1[\ell, m] C_1[0, m, s] S_2[r, s]. \tag{3.15}$$

Object 5: the derivative of the log normalizer

The derivative used by the fixed-branch likelihood is

$$\partial_\beta \log \widehat{Z}_t = \frac{\dot{\widehat{Z}}_t}{\widehat{Z}_t}. \tag{3.16}$$

Since c_t, τ_t, λ_t are frozen in the fixed branch,

$$\dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t. \tag{3.17}$$

The derivative of the carried filter is equally important because it becomes part of the next target:

$$\dot{\widehat{p}}_t(z_t; \beta) = \frac{\dot{a}_t(z_t; \beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(z_t; \beta) \dot{\widehat{Z}}_t(\beta)}{\widehat{Z}_t(\beta)^2}. \tag{3.18}$$

This front orientation is not a replacement for the derivation below. It is the small map used to read the longer source-order reconstruction.

4 What Problem Is Being Solved?

Mathematical question. Zhao and Cui [4] begin with a hidden Markov state, observations, and possibly unknown static parameters. The question answered here is: what density must a sequential algorithm update when new data arrive?

The data arrive in time order. At time t there is a hidden state

$$x_t \in \mathbb{R}^{m_x}$$

and an observation

$$y_t \in \mathbb{R}^{m_y}.$$

There may also be a static unknown parameter

$$\alpha \in \mathbb{R}^{m_\alpha}.$$

The filtering problem is to update the conditional density of the current state and parameter when y_t arrives:

$$p(x_t, \alpha \mid y_{1:t}).$$

The path problem is to update the density of the whole trajectory and parameter:

$$p(x_{0:t}, \alpha \mid y_{1:t}).$$

The smoothing problem asks for an earlier state after later data have been seen:

$$p(x_k \mid y_{1:t}), \qquad k < t.$$

The bottleneck is always an integral. At time $t-1$ suppose we know, or have approximated,

$$p(x_{t-1}, \alpha \mid y_{1:t-1}).$$

After seeing y_t, Bayes' rule introduces the unnormalized three-block density

$$q_t(x_t, \alpha, x_{t-1}) = p(x_{t-1}, \alpha \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

The next filter is obtained by integrating out x_{t-1}:

$$p(x_t, \alpha \mid y_{1:t}) = \frac{\int q_t(x_t, \alpha, x_{t-1}) \, \mathrm{d}x_{t-1}}{\int q_t(x_t, \alpha, x_{t-1}) \, \mathrm{d}x_t \, \mathrm{d}\alpha \, \mathrm{d}x_{t-1}}.$$

In low dimension one can attack this integral by quadrature, particles, or Gaussian projection. Zhao and Cui [4]'s idea is different: approximate the whole function q_t in a separable tensor-train form so that marginal integrals are cheap contractions instead of high-dimensional numerical integrations.

Implementation object. A minimal implementation must expose evaluators for $\widehat{p}_{t-1}$, f_α, and g_α, then build an evaluator for q_t. The first failure check is that q_t must be finite and nonnegative at all fitting points; otherwise neither a square root nor an importance weight is meaningful.

5 State-Space Model And Notation Used In This Note

Mathematical question. The first two source equations answer: what conditional independence structure defines the state-space model? The variables $X_t \in \mathcal{X} \subseteq \mathbb{R}^m$ are latent states, $Y_t \in \mathcal{Y} \subseteq \mathbb{R}^n$ are observations, and $\alpha \in \Theta \subseteq \mathbb{R}^d$ is the static parameter.

Zhao and Cui [4] use θ for the unknown parameter. In this note the parameter is written α, because θ is often used later as a generic argument in derivative formulas. The paper's model equations (1)–(2) become

$$X_t \mid (X_{0:t-1} = x_{0:t-1}, \alpha) \equiv X_t \mid (X_{t-1} = x_{t-1}, \alpha) \sim f_\alpha(x_t \mid x_{t-1}), \tag{5.1}$$

and

$$Y_t \mid (X_{0:t} = x_{0:t}, Y_{1:t-1} = y_{1:t-1}, \alpha) \equiv Y_t \mid (X_t = x_t, \alpha) \sim g_\alpha(y_t \mid x_t). \tag{5.2}$$

The first statement says the latent state process is Markov once α is fixed. The second says the new observation only sees the current state, again once α is fixed.

Implementation meaning. Inputs are (x_t, x_{t-1}, α) for the transition evaluator and (y_t, x_t, α) for the observation evaluator. Outputs are nonnegative densities or log densities. Quantities to retain are the model specification and the observed y_t. Diagnostics are finite log-density checks and shape checks for m, n, d.

Mathematical question. The next question is: how do the local transition and observation densities combine into the full joint density?

The joint density of parameter, states, and observations through time t, corresponding to Zhao and Cui [4], Eq. (3), is

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha)p_\alpha(x_0)\prod_{j=1}^{t} f_\alpha(x_j \mid x_{j-1})g_\alpha(y_j \mid x_j). \tag{5.3}$$

This is obtained by the chain rule:

$$\begin{aligned} p(\alpha, x_{0:t}, y_{1:t}) &= p(\alpha)p_\alpha(x_0)\prod_{j=1}^{t} p(x_j \mid x_{0:j-1}, y_{1:j-1}, \alpha)p(y_j \mid x_{0:j}, y_{1:j-1}, \alpha) \\ &= p(\alpha)p_\alpha(x_0)\prod_{j=1}^{t} f_\alpha(x_j \mid x_{j-1})g_\alpha(y_j \mid x_j), \end{aligned}$$

where the last equality uses the Markov and observation assumptions.

The derivation is the probabilistic analogue of multiplying conditional reaction probabilities along a time-ordered chain. Every new time index adds one transition factor and one observation factor. An implementation stores this as a log-density sum, not as a product, to avoid underflow:

$$\log p(\alpha, x_{0:t}, y_{1:t}) = \log p(\alpha) + \log p_\alpha(x_0) + \sum_{j=1}^{t}\{\log f_\alpha(x_j \mid x_{j-1}) + \log g_\alpha(y_j \mid x_j)\}. \tag{5.4}$$

The diagnostic is simple: any impossible model transition should produce $-\infty$ log density intentionally, not a NaN.

Mathematical question. The fourth source equation answers: how does the joint density become a posterior density after data are observed?

Conditioning on data gives Zhao and Cui [4], Eq. (4) in the notation of this note:

$$p(\alpha, x_{0:t} \mid y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}, \qquad p(y_{1:t}) = \int p(\alpha, x_{0:t}, y_{1:t}) \, \mathrm{d}\alpha \, \mathrm{d}x_{0:t}. \tag{5.5}$$

The denominator is the evidence. It is a scalar, but it is usually not available analytically because the integral spans the parameter and all states.

Implementation meaning. The numerator is evaluable pointwise. The evidence is a scalar integral. A filtering algorithm that claims a likelihood must specify how it computes or approximates this scalar. The failure check is that the approximated evidence must be positive and finite.

6 The Four Marginal Learning Problems

Mathematical question. The next four source equations answer: which posterior marginals correspond to filtering, parameter learning, path estimation, and smoothing?

Zhao and Cui [4], Eqs. (5)–(8) are all marginalizations of the same posterior $p(\alpha, x_{0:t} \mid y_{1:t})$. Filtering keeps only the current state:

$$p(x_t \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t-1} \, \mathrm{d}\alpha. \tag{6.1}$$

Parameter learning keeps only α:

$$p(\alpha \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t}. \tag{6.2}$$

Path learning keeps the full state path but removes α:

$$p(x_{0:t} \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}\alpha. \tag{6.3}$$

Fixed-lag or retrospective smoothing keeps one old state:

$$p(x_k \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:k-1} \, \mathrm{d}x_{k+1:t} \, \mathrm{d}\alpha, \qquad k < t. \tag{6.4}$$

The important point is not the names. It is that the same high-dimensional posterior density produces all four outputs by integration. A method that stores only a cloud of samples can estimate these integrals by sample averages. A method that stores a Gaussian can compute only Gaussian marginals. A method that stores a tensor-train density aims to keep a non-Gaussian function representation while making many marginal integrals cheap.

Implementation meaning. The stored object must support integrating out named coordinate blocks. The outputs are marginal density evaluators. The failure mode is inconsistent normalization: if integrating a marginal over its retained coordinates does not give one, the contraction or normalizer is wrong.

7 The Exact Recursive Bottleneck

Mathematical question. These source equations answer: how can the posterior update be performed sequentially without storing the whole old path forever?

We now derive Zhao and Cui [4], Eqs. (9)–(11). Start from Bayes' rule:

$$p(\alpha, x_{0:t} \mid y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}.$$

Using the factorization (BF-3),

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha, x_{0:t-1}, y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Also,

$$p(\alpha, x_{0:t-1}, y_{1:t-1}) = p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}).$$

Therefore

$$\begin{aligned} p(\alpha, x_{0:t} \mid y_{1:t}) &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_{1:t})} \\ &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \tag{7.1}$$

The last step uses $p(y_{1:t}) = p(y_t \mid y_{1:t-1}) p(y_{1:t-1})$.

Now integrate (BF-9) over $x_{0:t-2}$. The only factor depending on the old path $x_{0:t-2}$ is $p(\alpha, x_{0:t-1} \mid y_{1:t-1})$, so

$$\begin{aligned} p(\alpha, x_t, x_{t-1} \mid y_{1:t}) &= \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t-2} \\ &= \frac{p(\alpha, x_{t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \tag{7.2}$$

Finally, integrate over x_{t-1} to obtain the next parameter-state filter:

$$p(\alpha, x_t \mid y_{1:t}) = \int p(\alpha, x_t, x_{t-1} \mid y_{1:t}) \, \mathrm{d}x_{t-1}. \tag{7.3}$$

The numerator in (BF-10) is pointwise evaluable if the previous filter is evaluable. The denominator and the marginalization in (BF-11) are the hard parts. Zhao and Cui [4] turn this into a function-approximation problem by replacing the unnormalized numerator with a tensor train.

Implementation meaning. 1-Eq9–Eq11. Inputs are the retained evaluator for $p(\alpha, x_{t-1} \mid y_{1:t-1})$, the new observation y_t, and model evaluators f_α, g_α. The output of the numerator step is a pointwise evaluator for the three-block target. The quantities to retain after marginalization is an evaluator for $p(\alpha, x_t \mid y_{1:t})$ and an evidence increment. The failure check is that the marginal integral over (α, x_t) must equal one after normalization.

8 Tensor Trains From First Principles

Mathematical question. The question answered here is: what separable function class will replace a full high-dimensional grid?

This section starts from the object a numerical reader can picture: a table of function values. If $D = 2$, a function $h(r_1, r_2)$ sampled on a grid is a matrix. Low-rank matrix approximation writes

$$h(r_1, r_2) \approx \sum_{\rho=1}^{R} u_\rho(r_1) v_\rho(r_2).$$

Each term is separable: one factor depends only on r_1, one factor only on r_2. A tensor train is the many-variable analogue. It does not require a full D-dimensional table. It stores a chain of low-rank links, so that information can pass from coordinate 1 to coordinate D through the rank indices.

Let $r = (r_1, \dots, r_D) \in \Omega_1 \times \cdots \times \Omega_D$. A functional tensor train approximates a scalar function $h(r)$ by multiplying matrix-valued one-dimensional functions:

$$h(r) \approx \widehat{h}(r) = H_1(r_1) H_2(r_2) \cdots H_D(r_D), \tag{8.1}$$

where

$$H_k(r_k) \in \mathbb{R}^{R_{k-1} \times R_k}, \qquad R_0 = R_D = 1.$$

Zhao and Cui [4]'s displayed TT formula includes the endpoint rank indices:

$$\widehat{h}(r) = \sum_{\rho_0=1}^{R_0} \sum_{\rho_1=1}^{R_1} \cdots \sum_{\rho_D=1}^{R_D} H_1^{(\rho_0,\rho_1)}(r_1) \cdots H_D^{(\rho_{D-1},\rho_D)}(r_D), \qquad R_0 = R_D = 1. \tag{8.2}$$

The endpoint sums have only one term. They are written so every core has the same two rank labels. A programmer usually evaluates the matrix product (TT-1); a proof often expands the rank contractions as in (TT-1a).

Because the first object has one row and the last has one column, the product is a 1×1 scalar. Written with indices,

$$\widehat{h}(r) = \sum_{\rho_1=1}^{R_1} \cdots \sum_{\rho_{D-1}=1}^{R_{D-1}} H_1^{1,\rho_1}(r_1) H_2^{\rho_1,\rho_2}(r_2) \cdots H_D^{\rho_{D-1},1}(r_D). \tag{8.3}$$

The numbers R_k are TT ranks. A high rank lets the representation express strong interactions across a split $(r_1, \dots, r_k)$ and $(r_{k+1}, \dots, r_D)$. A low rank is useful only when the function has low-dimensional structure across that split.

To see the role of the ranks, split the variables into a left block and a right block:

$$r_{\le k} = (r_1, \dots, r_k), \qquad r_{>k} = (r_{k+1}, \dots, r_D).$$

Equation (TT-2) can be regrouped as

$$\widehat{h}(r_{\le k}, r_{>k}) = \sum_{\rho=1}^{R_k} L_\rho(r_{\le k}) R_\rho(r_{>k}), \tag{8.4}$$

where

$$L_\rho(r_{\le k}) = \sum_{\rho_1,\dots,\rho_{k-1}} H_1^{1,\rho_1}(r_1) \cdots H_k^{\rho_{k-1},\rho}(r_k),$$

and

$$R_\rho(r_{>k}) = \sum_{\rho_{k+1},\dots,\rho_{D-1}} H_{k+1}^{\rho,\rho_{k+1}}(r_{k+1}) \cdots H_D^{\rho_{D-1},1}(r_D).$$

Thus R_k is the number of separated terms allowed across that split. If a posterior density has only local or weak long-range dependence in the chosen ordering, moderate ranks can be enough. If a posterior has strong global coupling, the ranks grow and the method becomes expensive.

The low-dimensional analogy is the singular value decomposition of a matrix. For a two-coordinate function, rank R means R separable columns times rows. For many coordinates, R_k plays the same role at the cut after coordinate k. The failure diagnostic is rank growth: if any fitted R_k approaches the imposed maximum, the chosen coordinate ordering, basis, or preconditioner is not expressing the density compactly.

Rank-Count Plausibility Calculation

If every coordinate uses p basis functions and the function has D coordinates, a full tensor stores

$$N_{\text{full}} = p^D \tag{8.5}$$

coefficients. This is the algebraic curse of dimensionality. For $p = 8$ and $D = 20$,

$$8^{20} = 2^{60} \approx 1.15 \times 10^{18}. \tag{8.6}$$

A tensor train with uniform rank R stores approximately

$$N_{\text{TT}} = \sum_{k=1}^{D} R_{k-1} p R_k \approx DpR^2, \qquad R_0 = R_D = 1. \tag{8.7}$$

For $p = 8, D = 20, R = 10$,

$$DpR^2 = 20 \cdot 8 \cdot 10^2 = 16000. \tag{8.8}$$

Mass contractions usually cost one extra rank factor:

$$C_{\text{mass}} \approx O(DpR^3). \tag{8.9}$$

This comparison is the reason the method is plausible, not a proof that it will work. The entire empirical hinge is

$$R \text{ stays moderate under the chosen ordering, basis, domain, and preconditioner.} \tag{8.10}$$

If ranks grow toward the full tensor scale, the TT representation has not escaped the curse of dimensionality for that target.

9 From Bayesian Filtering To A Tensor-Train Approximation Family

The reason to introduce tensor trains is not storage arithmetic alone. The starting mathematical object is a posterior density. At time t, before normalization, the exact adjacent-state filtering target has the form

$$q_t(x_t, \theta, x_{t-1}) = p(x_{t-1}, \theta \mid y_{1:t-1}) f_\theta(x_t \mid x_{t-1}) g_\theta(y_t \mid x_t). \tag{9.1}$$

If the state has m components and the parameter has d components, then

$$(x_t, \theta, x_{t-1}) \in \mathbb{R}^{2m+d}. \tag{9.2}$$

Thus the exact object is a function of many variables, not a vector of probabilities. Filtering is the operation

$$p(x_t, \theta \mid y_{1:t}) = \frac{\int q_t(x_t, \theta, x_{t-1})\, \mathrm{d}x_{t-1}}{\int q_t(x_t, \theta, x_{t-1})\, \mathrm{d}x_{t-1}\, \mathrm{d}x_t\, \mathrm{d}\theta}. \tag{9.3}$$

The denominator is the evidence increment and the numerator is the retained filter. A high-dimensional filtering method must therefore answer two questions at the same time:

$$\text{How do we represent } q_t? \qquad \text{How do we integrate the represented object?} \tag{9.4}$$

A Gaussian filter answers the first question by replacing the posterior with a mean and covariance:

$$p(x_t, \theta \mid y_{1:t}) \approx N\left(\begin{bmatrix} x_t \\ \theta \end{bmatrix}; \mu_t, \Sigma_t\right). \tag{9.5}$$

This is efficient, but it assumes the relevant posterior shape can be captured by ellipsoids. In the threaded example,

$$y_t = \sin(x_t) + \eta_t, \tag{9.6}$$

the likelihood is

$$g(y_t \mid x_t) \propto \exp\left[-\frac{(y_t - \sin x_t)^2}{2r^2}\right]. \tag{9.7}$$

For y_t near zero, this function can have high density near several separated values of x_t. The posterior may be skewed, curved, or multimodal. A single Gaussian can still be useful as a preconditioner, but it is not a full representation of the nonlinear density.

Tensor trains answer the first question by approximating a function with separated interaction channels. After mapping physical coordinates to reference coordinates $z = (z_1, \ldots, z_D)$, write

$$\widetilde{q}_t(z) = q_t(\Psi_t(z))\, |\det \nabla \Psi_t(z)|\,. \tag{9.8}$$

The TT approximation seeks a product of low-rank matrix-valued univariate functions:

$$\widetilde{q}_t(z) \approx H_1(z_1) H_2(z_2) \cdots H_D(z_D). \tag{9.9}$$

This does not assume independence. Independence would be the rank-one factorization

$$\widetilde{q}_t(z_1, \ldots, z_D) = \prod_{k=1}^{D} h_k(z_k). \tag{9.10}$$

A rank-R tensor train allows R interaction channels to pass information across coordinate splits. At a split $z_{\le k} \mid z_{>k}$, the local meaning is

$$\widetilde{q}_t(z_{\le k}, z_{>k}) \approx \sum_{a=1}^{R_k} U_{k,a}(z_{\le k}) V_{k,a}(z_{>k}). \tag{9.11}$$

Thus TT rank is a measure of how much dependence crosses the split. It is a controlled relaxation of independence, not a return to independent coordinates.

The second question, integration, is answered by using basis functions and mass matrices. If

$$H_k(z_k) = \sum_{\ell=0}^{p_k - 1} C_k[:, \ell, :]\, b_{k,\ell}(z_k), \tag{9.12}$$

then one-dimensional integrals such as

$$B_k[\ell, m] = \int_{-1}^{1} b_{k,\ell}(z) b_{k,m}(z)\, \mathrm{d}z \tag{9.13}$$

turn high-dimensional squared-TT integrals into repeated small matrix contractions. The approximation family is therefore matched to the filtering task: it represents a nonlinear density and gives a way to marginalize it.

The method is plausible when the posterior geometry has limited interaction width after ordering and preconditioning. A local transition model has the schematic form

$$f(x_t \mid x_{t-1}) \approx \prod_{j=1}^{m} f_j(x_{t,j} \mid x_{t-1,j}, x_{t-1,\mathcal{N}(j)}), \tag{9.14}$$

and a local observation model has

$$g(y_t \mid x_t) \approx \prod_{\ell=1}^{n} g_\ell(y_{t,\ell} \mid x_{t,\mathcal{I}_\ell}). \tag{9.15}$$

If the coordinate order keeps statistically neighboring variables close, then many splits separate only a moderate number of direct interactions. The rank diagnostic

$$\max_k R_k \le R_{\max} \tag{9.16}$$

is a check of this structural hypothesis. This locality argument is a modeling and numerical heuristic for why low or moderate TT ranks may approximate $\widetilde{q}_t$ well in structured cases; it is not a theorem that such ranks suffice uniformly across models, observations, coordinate orderings, or preconditioners.

The method should not be expected to work automatically. A sharp global constraint,

$$y_t \approx \sum_{j=1}^{m} a_j x_{t,j}, \qquad r^2 \ll 1, \tag{9.17}$$

can create dependence crossing many splits. A many-mode posterior can do the same. In those cases the practical signs are

$$R_k \text{ grows}, \qquad \frac{\|Ag - y\|_W}{\|y\|_W} \text{ stays large}, \qquad \mathrm{cond}(A^\top W A + \rho I) \text{ grows}. \tag{9.18}$$

These are failure diagnostics, not embarrassments. They tell the panel that the approximation family is being tested against the posterior geometry rather than assumed correct.

Preconditioning explains why a Gaussian object can still be useful without being the final approximation. Let ρ_t be a bridge density capturing location, scale, and dominant correlations. The residual target

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \eta(u) \tag{9.19}$$

asks the TT to represent a correction factor. If ρ_t captures the main shape, then

$$\frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \tag{9.20}$$

is flatter than the original density, and lower TT rank becomes more plausible. This is the conceptual reason the Zhao–Cui method combines tensor trains, squared densities, conditional maps, and preconditioning rather than relying on one device alone.

10 Why Moderate Tensor-Train Rank Is A Plausible Hypothesis

Storage counts explain why tensor trains are attractive, but they do not by themselves explain why the rank should remain moderate for a nonlinear posterior. The mathematical reason to try the method is that many filtering densities have structure across coordinate splits. Write a transformed filtering target as

$$h(r_1, \dots, r_D) = q_t(r_1, \dots, r_D) J_t(r_1, \dots, r_D), \tag{10.1}$$

where r denotes physical or transformed coordinates and J_t is the Jacobian factor from the chosen domain map. At the split after coordinate k, define

$$r_{\le k} = (r_1, \dots, r_k), \qquad r_{>k} = (r_{k+1}, \dots, r_D). \tag{10.2}$$

A rank-R_k separated approximation across this split has the form

$$h(r_{\le k}, r_{>k}) \approx \sum_{a=1}^{R_k} U_a(r_{\le k}) V_a(r_{>k}). \tag{10.3}$$

The TT rank R_k is the number of left–right interaction channels needed at that split. It is not a property of one coordinate in isolation; it is a property of the dependence between the first k coordinates and the remaining coordinates.

For a two-block discretization, collect $h(r_{\le k}, r_{>k})$ on a tensor product grid and reshape it into a matrix

$$H_k[i, j] = h(r_{\le k}^{(i)}, r_{>k}^{(j)}). \tag{10.4}$$

The best matrix rank-R_k approximation is controlled by the singular values

$$H_k = \sum_{a \ge 1} \sigma_{k,a} u_{k,a} v_{k,a}^\top, \qquad \|H_k - H_{k,R}\|_F^2 = \sum_{a > R} \sigma_{k,a}^2. \tag{10.5}$$

The continuous TT split rank is the same idea applied successively to all left–right splits. Moderate rank is plausible when the split singular values decay quickly after a good coordinate ordering and preconditioning map.

Filtering models often create this kind of structure. If the transition density has local coupling,

$$f(x_t \mid x_{t-1}) \approx \prod_{j=1}^{m} f_j(x_{t,j} \mid x_{t-1,j}, x_{t-1,j-1}, x_{t-1,j+1}), \tag{10.6}$$

and the observation density is conditionally local,

$$g(y_t \mid x_t) \approx \prod_{\ell=1}^{n} g_\ell(y_{t,\ell} \mid x_{t,\mathcal{I}_\ell}), \tag{10.7}$$

then a coordinate order that keeps neighboring variables near one another can leave only a limited amount of dependence crossing each split. The TT ranks then measure the width of the statistical interaction crossing the ordering, not the ambient dimension alone.

Here is the same rank idea in a physical high-dimensional example. Imagine measurements from a one-dimensional sensor array, a polymer chain, or a reaction coordinate discretized into neighboring sites. Let x_j be the unknown local state at site j. A common local energy model has the form

$$\log q(x_1, \dots, x_m) = \sum_{j=1}^{m} a_j(x_j) + \sum_{j=1}^{m-1} b_j(x_j, x_{j+1}), \tag{10.8}$$

where a_j is the single-site evidence and b_j is the nearest-neighbor interaction. Across the split after coordinate k, all terms in (10.8) are entirely left or entirely right except the one bridge term $b_k(x_k, x_{k+1})$:

$$q(x_1, \dots, x_m) = \exp\left[\sum_{j \le k} a_j(x_j) + \sum_{j<k} b_j(x_j, x_{j+1})\right] \times \exp[b_k(x_k, x_{k+1})] \exp\left[\sum_{j>k} a_j(x_j) + \sum_{j>k} b_j(x_j, x_{j+1})\right]. \tag{10.9}$$

Only the middle factor crosses the split. If that bridge factor has a separated expansion

$$\exp[b_k(x_k, x_{k+1})] \approx \sum_{a=1}^{R_{\text{edge}}} u_{k,a}(x_k) v_{k,a}(x_{k+1}), \tag{10.10}$$

then the whole density has a split-separated representation with rank at most R_{edge} at that cut:

$$q(x_{\le k}, x_{>k}) \approx \sum_{a=1}^{R_{\text{edge}}} U_{k,a}(x_{\le k}) V_{k,a}(x_{>k}). \tag{10.11}$$

For a two-dimensional grid ordered by rows, a vertical cut crosses several nearest-neighbor edges rather than one. If w_k edges cross the split and each edge needs at most R_{edge} channels, the crude product bound is

$$R_k \lesssim R_{\text{edge}}^{w_k}. \tag{10.12}$$

This bound is not a sharp theorem, but it makes the hypothesis vivid: moderate TT rank is plausible when the coordinate order keeps the number and complexity of cross-split physical interactions moderate. It is much less plausible when one observation creates a sharp constraint involving nearly all coordinates at once.

The same equations show when the method should fail. If a likelihood creates a global constraint such as

$$y_t \approx \sum_{j=1}^{m} a_j x_{t,j} \tag{10.13}$$

with very small noise, then many coordinates are coupled through one sharp surface. The split singular values in (10.5) may decay slowly, and rank growth becomes a diagnostic rather than a surprise. A multimodal posterior with modes separated along many coordinate directions can have the same effect.

Preconditioning is included precisely because the raw target may have poor rank behavior. Suppose a map T_t sends a simpler reference density η toward the dominant shape of q_t. In u-coordinates the residual target is

$$h_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \eta(u), \tag{10.14}$$

where ρ_t is the density represented by the map. If ρ_t captures the main location, scale, and correlation structure, the ratio

$$\omega_t(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))} \tag{10.15}$$

is flatter than q_t, and the residual TT is asked to learn a correction rather than the whole posterior geometry. That is the mathematical reason preconditioning can reduce rank pressure.

The practical hypothesis is therefore

$$\max_k R_k \le R_{\max} \quad \text{and} \quad \frac{\|Ag - y\|_W}{\|y\|_W} \le \varepsilon_{\text{fit}} \quad \text{with stable condition numbers.} \tag{10.16}$$

The rank assumption is not a theorem for arbitrary nonlinear posteriors. It is a structural and empirical claim checked by ranks, residuals, conditioning, defensive mass, bridge ratios, and finite-difference consistency.

11 Coordinate Systems And Density Transformations

The method uses several coordinate systems because different operations are easiest in different spaces. Let

$$r = (x_t, \theta, x_{t-1}) \tag{11.1}$$

denote physical filtering coordinates. A domain map sends a reference cube coordinate $z \in [-1, 1]^D$ to physical coordinates:

$$r = \Psi_t(z), \qquad J_{\Psi,t}(z) = \left|\det \nabla \Psi_t(z)\right|. \tag{11.2}$$

The target seen by the TT in reference coordinates is

$$\widetilde{q}_t(z) = q_t(\Psi_t(z)) J_{\Psi,t}(z). \tag{11.3}$$

Thus z is the coordinate in which basis functions, fitting points, and mass matrices are usually declared.

Preconditioning introduces another coordinate u. Let T_t be an invertible map from physical coordinates to a preconditioned coordinate:

$$u = T_t(r), \qquad r = T_t^{-1}(u). \tag{11.4}$$

If $\eta(u)$ is the reference density used in u-space, then the pushforward/pullback identities give

$$q_t(r)\,\mathrm{d}r = q_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right| \,\mathrm{d}u. \tag{11.5}$$

The preconditioned residual in (10.14) is a density in u-coordinates. A second domain map may still send a cube coordinate z to u:

$$u = \Psi_t^\sharp(z), \qquad \widetilde{q}_t^\sharp(z) = h_t^\sharp(\Psi_t^\sharp(z)) \left|\det \nabla \Psi_t^\sharp(z)\right|. \tag{11.6}$$

The retained coordinate z_t is not another global transform. It is the part of the reference coordinate retained after marginalizing the previous state:

$$z = (z_t, z_{t-1}), \qquad a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1})\,\mathrm{d}z_{t-1}, \tag{11.7}$$

and

$$\widehat{p}_t(z_t) = \frac{a_t(z_t)}{\widehat{Z}_t}. \tag{11.8}$$

If the next time step evaluates the previous filter at a physical point x_t, the implementation first maps that point to the retained reference coordinate:

$$z_t = \Psi_{t,\text{ret}}^{-1}(x_t), \qquad \widehat{p}_t(x_t) = \widehat{p}_t(z_t) \left|\det \nabla \Psi_{t,\text{ret}}^{-1}(x_t)\right|. \tag{11.9}$$

Symbol	Space	Main density identity	What is stored
r	physical state/parameter space	$q_t(r)\,\mathrm{d}r$	model evaluator and physical domains
z	reference cube	$\widetilde{q}_t(z) = q_t(\Psi_t(z))J_{\Psi,t}(z)$	basis values, points, TT cores, mass matrices
u	preconditioned space	$q_t(T_t^{-1}(u))\lvert\det\nabla T_t^{-1}(u)\rvert$	preconditioner, bridge density, log Jacobian
z_t	retained reference coordinates	$a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1})\,\mathrm{d}z_{t-1}$	retained numerator/filter and derivative

Most implementation mistakes in this method are coordinate mistakes: using a physical density in reference coordinates without the Jacobian, applying a KR map in u-space to a point still in r-space, or carrying a retained filter in z_t while evaluating it as if it were already a density in x_t.

12 One Observation Step Through All Coordinate Systems

The symbols r, z, u, z_t become easier to track when one observation update is written as a single chain. Begin in physical coordinates

$$r = (x_t, \theta, x_{t-1}), \qquad q_t(r) = \widehat{p}_{t-1}(x_{t-1}, \theta) f_\theta(x_t \mid x_{t-1}) g_\theta(y_t \mid x_t). \tag{12.1}$$

This formula answers the physical inference question: after observing y_t, which triples (x_t, θ, x_{t-1}) are plausible?

The TT does not fit q_t directly on an infinite physical domain. A domain map

$$r = \Psi_t(z), \qquad z \in [-1, 1]^D, \qquad J_{\Psi,t}(z) = \lvert\det\nabla\Psi_t(z)\rvert \tag{12.2}$$

turns the physical density into a reference-coordinate density

$$\widetilde{q}_t(z) = q_t(\Psi_t(z))J_{\Psi,t}(z). \tag{12.3}$$

The equality of masses is

$$\int_{\Psi_t(A)} q_t(r)\,\mathrm{d}r = \int_A \widetilde{q}_t(z)\,\mathrm{d}z. \tag{12.4}$$

Thus the Jacobian is not a numerical decoration; it is what keeps probability mass unchanged after changing coordinates.

The squared-TT step fits the square root in reference coordinates:

$$\sqrt{\widetilde{q}_t(z)} \approx e^{-c_t/2}\phi_t(z), \tag{12.5}$$

and then defines the nonnegative approximation

$$\widehat{q}_t(z) = e^{-c_t}\phi_t(z)^2 + \tau_t\lambda_t(z). \tag{12.6}$$

The carried filter is not the full joint object. If

$$z = (z_t, z_{t-1}), \tag{12.7}$$

then the retained numerator and normalizer are

$$a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1})\,\mathrm{d}z_{t-1}, \qquad \widehat{Z}_t = \int a_t(z_t)\,\mathrm{d}z_t, \tag{12.8}$$

and the retained reference filter is

$$\widehat{p}_t^{\mathrm{ref}}(z_t) = \frac{a_t(z_t)}{\widehat{Z}_t}. \tag{12.9}$$

If a later formula needs the filter as a physical density in x_t, θ, write

$$(x_t, \theta) = \Psi_{t,\mathrm{ret}}(z_t), \qquad \widehat{p}_t^{\mathrm{phys}}(x_t, \theta) = \widehat{p}_t^{\mathrm{ref}}(\Psi_{t,\mathrm{ret}}^{-1}(x_t, \theta))\left|\det \nabla \Psi_{t,\mathrm{ret}}^{-1}(x_t, \theta)\right|. \tag{12.10}$$

A useful bookkeeping rule is

$$\begin{aligned} &\widetilde{q}_t(z),\ a_t(z_t),\ \widehat{Z}_t,\ \widehat{p}_t^{\mathrm{ref}}(z_t) \quad \text{are reference-coordinate objects integrated with } \mathrm{d}z, \\ &\widehat{p}_t^{\mathrm{phys}}(r_t) = \widehat{p}_t^{\mathrm{ref}}(\Psi_{t,\mathrm{ret}}^{-1}(r_t))\left|\det \nabla \Psi_{t,\mathrm{ret}}^{-1}(r_t)\right| \quad \text{is the corresponding physical density.} \end{aligned} \tag{12.11}$$

Preconditioning inserts one additional coordinate u. A bridge density $\rho_t(r)$ and map $T_t : r \mapsto u$ are chosen so that the dominant shape of q_t is easier in u-space. The residual target is

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\widehat{\rho}_t(T_t^{-1}(u))}\eta(u). \tag{12.12}$$

This is still the same observation step. The difference is that the TT now fits the residual density $q_t^\sharp$ rather than the raw density q_t. The pullback relation after fitting is

$$\widehat{q}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))}\widehat{\rho}_t(r). \tag{12.13}$$

The next observation uses the retained filter by querying it at the previous state coordinate of the next fitting point. If a next-step fitting point is

$$z_{t+1}^{(j)} = (z_{t+1}^{(j)}, z_t^{(j)}), \tag{12.14}$$

then the target value at that point uses

$$\widehat{p}_t^{\mathrm{ref}}(z_t^{(j)}) \tag{12.15}$$

inside

$$\widetilde{q}_{t+1}(z_{t+1}^{(j)}) = \widehat{p}_t^{\mathrm{ref}}(z_t^{(j)}) f_\theta(x_{t+1}^{(j)} \mid x_t^{(j)}) g_\theta(y_{t+1} \mid x_{t+1}^{(j)}) J_{t+1}(z_{t+1}^{(j)}). \tag{12.16}$$

This is the full coordinate story in one line of work:

$$q_t(r) \to \widetilde{q}_t(z) \to \widehat{q}_t(z) \to \widehat{p}_t^{\mathrm{ref}}(z_t) \to \widetilde{q}_{t+1}(z_{t+1}, z_t). \tag{12.17}$$

The preconditioned route adds

$$q_t(r) \to q_t^\sharp(u) \to \widehat{\nu}_t^\sharp(u) \to \widehat{q}_t(r), \tag{12.18}$$

but it does not change the filtering task. It changes the coordinate system in which the hard part of the density is approximated.

13 Why The Fixed-Branch Gradient Is Useful And Narrow

The adaptive tensor-train algorithm chooses many discrete objects from the current target: rank pattern, pivot points, fitting points, stopping time, domain, shift, preconditioner, and sometimes a triangular transport map. Let

$$B(\beta) = \{\mathcal{R}_t(\beta), Z_{\mathrm{fit,t}}(\beta), \Psi_t(\beta), c_t(\beta), \tau_t(\beta), T_t(\beta), \text{sweep decisions}\}_{t=1}^{T} \tag{13.1}$$

denote the adaptive branch selected at parameter value β. The fully adaptive scalar is

$$\widehat{\ell}_T^{\,\mathrm{adapt}}(\beta) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta; B(\beta)). \tag{13.2}$$

This map need not be globally differentiable: if a pivot changes or a rank increases, the computational graph changes.

The fixed-branch derivative instead freezes a branch B_0 chosen at a reference value β_0:

$$B_0 = B(\beta_0), \qquad \widehat{\ell}_T(\beta; B_0) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta; B_0). \tag{13.3}$$

The derivative computed later is

$$\nabla_\beta \widehat{\ell}_T(\beta; B_0), \tag{13.4}$$

not a derivative of $\widehat{\ell}_T^{\,\mathrm{adapt}}$. This narrower object is useful because it answers a concrete numerical question: once the approximation choices have been declared, does the analytical derivative match the scalar actually evaluated by that declared approximation? It supports finite difference diagnostics,

$$\frac{\widehat{\ell}_T(\beta + he_j; B_0) - \widehat{\ell}_T(\beta - he_j; B_0)}{2h} \approx e_j^\top \nabla_\beta \widehat{\ell}_T(\beta; B_0), \tag{13.5}$$

and it can be used by local gradient-based exploration of the declared approximation. It deliberately does not claim to differentiate the adaptive policy that produced the branch.

Each univariate matrix entry is represented in a basis:

$$H_k^{a,b}(r_k) = \sum_{\ell=0}^{p_k-1} C_k[a, \ell, b]\psi_{k,\ell}(r_k). \tag{13.6}$$

This is the paper's basis expansion with renamed coefficient tensor. Zhao and Cui [4] write the coefficient as $A_k[\alpha_{k-1}, j, \alpha_k]$; here it is $C_k[a, \ell, b]$. The three axes are left rank, basis index, and right rank. The basis index describes one-coordinate shape; the rank indices transmit dependence to neighboring coordinates.

The tensor $C_k \in \mathbb{R}^{R_{k-1} \times p_k \times R_k}$ is the k-th TT core. Thus an implementation stores the basis family, the domain map for each coordinate, and the core arrays $C_1, \ldots, C_D$. To evaluate the function at one point r, it evaluates all basis vectors

$$\psi_k(r_k) = (\psi_{k,0}(r_k), \ldots, \psi_{k,p_k-1}(r_k))^\top,$$

forms the matrices $H_k(r_k)$, and multiplies them.

Implementation meaning. 2. Inputs are a coordinate r_k, basis definition, and core tensor C_k. The operation is basis evaluation followed by a contraction over the basis index:

$$H_k(r_k) = \sum_{\ell=0}^{p_k-1} \psi_{k,\ell}(r_k)\, C_k[:, \ell, :]. \tag{13.7}$$

The quantities to retain is the list of cores, basis objects, and coordinate domains. The failure checks are basis overflow, invalid coordinate outside the domain, and incompatible adjacent ranks.

13.1 A Concrete Basis Choice

Mathematical question. Zhao and Cui [4] allow several basis families. This note fixes one basis to make the paper's abstract core formula executable.

Zhao and Cui [4]'s software supports several basis families. For a self-contained minimal implementation, choose one. On a finite interval $[a_k, b_k]$, map a physical coordinate r_k to

$$z_k = \frac{2r_k - (a_k + b_k)}{b_k - a_k} \in [-1, 1], \qquad r_k = \frac{a_k + b_k}{2} + \frac{b_k - a_k}{2} z_k. \tag{13.8}$$

Use normalized Legendre polynomials

$$\psi_{k,\ell}(r_k) = \sqrt{\frac{2\ell + 1}{b_k - a_k}}\, P_\ell(z_k), \qquad \ell = 0, \ldots, p_k - 1. \tag{13.9}$$

The Legendre polynomials are evaluated by the recurrence

$$P_0(z) = 1, \qquad P_1(z) = z,$$
$$(\ell + 1)P_{\ell+1}(z) = (2\ell + 1)zP_\ell(z) - \ell P_{\ell-1}(z), \qquad \ell \geq 1. \tag{13.10}$$

This basis is useful pedagogically because its mass matrix is the identity:

$$\int_{a_k}^{b_k} \psi_{k,\ell}(r_k)\psi_{k,\ell'}(r_k)\, \mathrm{d}r_k = \delta_{\ell\ell'}. \tag{13.11}$$

If another basis is used, every formula below remains the same after replacing the identity mass matrix by the corresponding one-dimensional mass matrix.

Zhao and Cui [4] state that functional alternating schemes with cross interpolation typically use

$$O(DpR^2) \quad \text{target evaluations} \tag{13.12}$$

and

$$O(DpR^3) \quad \text{floating point operations}, \tag{13.13}$$

where $p = \max_k p_k$ and $R = \max_k R_k$. This is a cost statement after the ranks are moderate; it is not a promise that R stays small. The implementation should therefore record fitted ranks, residuals, and ill-conditioning diagnostics.

14 Why TT Marginalization Is Cheap Once The Representation Exists

Mathematical question. The question answered here is: once a function is in TT form, how does the algorithm integrate out one coordinate without returning to a full grid?

Suppose h is represented by (TT-1). If we need to integrate out coordinate r_k, the product structure gives

$$\int \widehat{h}(r_1, \ldots, r_D)\, \mathrm{d}r_k = H_1(r_1) \cdots H_{k-1}(r_{k-1}) \left(\int H_k(r_k)\, \mathrm{d}r_k \right) H_{k+1}(r_{k+1}) \cdots H_D(r_D). \tag{14.1}$$

Only the k-th core changes. Its integrated matrix is

$$\overline{H}_k = \int H_k(r_k)\, \mathrm{d}r_k, \qquad \overline{H}_k[a, b] = \sum_{\ell=0}^{p_k - 1} C_k[a, \ell, b] \int \psi_{k,\ell}(r_k)\, \mathrm{d}r_k. \tag{14.2}$$

If the one-dimensional basis integrals are precomputed, marginalizing one coordinate is a small matrix contraction. Repeatedly applying (TT-4) integrates whole blocks of variables.

This explains why Zhao and Cui [4] target a TT representation of (x_t, α, x_{t-1}). Once that three-block function is separable enough, the new filter $p(x_t, \alpha \mid y_{1:t})$ is obtained by integrating the old-state block x_{t-1}, one coordinate at a time.

Implementation meaning. 2-core-integration. Inputs are the TT cores and precomputed one-dimensional basis integrals. The operation replaces one core by $\overline{H}_k$ and contracts adjacent matrices. The retained object is a lower-dimensional TT or evaluator. The failure check is dimension/order consistency: integrating the wrong block silently returns the wrong filter.

15 Algorithm 1, Fully Annotated

Mathematical question. Algorithm 1(a) answers: what target can be evaluated at time t using only the retained approximation from time $t-1$?

The exact recursion (BF-10) contains the previous exact filter. A numerical algorithm has the previous approximation. Let $\widehat{\pi}_{t-1}(x_{t-1}, \alpha)$ denote an unnormalized approximation of $p(x_{t-1}, \alpha \mid y_{1:t-1})$. Zhao and Cui [4], Eq. (12) becomes

$$q_t(x_t, \alpha, x_{t-1}) = \widehat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \tag{15.1}$$

This is not separable in general. Even if $\widehat{\pi}_{t-1}$ is a TT over (α, x_{t-1}), the transition and likelihood couple the new and old state coordinates.

Zhao and Cui [4] use approximate proportionality before normalization. In explicit terms, $h_1 \propto_{\text{approx}} h_2$ means

$$\frac{h_1(r)}{\int h_1(s)\,\mathrm{d}s} \approx \frac{h_2(r)}{\int h_2(s)\,\mathrm{d}s}. \tag{15.2}$$

This is why the algorithm can store an unnormalized TT. The normalization is not forgotten; it is postponed until the complete TT contraction $\widehat{Z}_t$ is computed.

Algorithm 1 has three mathematical steps.

Step 1: build a pointwise target. Given $r_t = (x_t, \alpha, x_{t-1})$, evaluate $q_t(r_t)$ by multiplying the previous approximation, transition density, and likelihood:

$$r_t \mapsto \widehat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Storage needed: an evaluator for the previous TT approximation and evaluators for f_α and g_α.

Implementation meaning. Inputs: $\widehat{\pi}_{t-1}$, f_α, g_α, y_t, and a point (x_t, α, x_{t-1}). Output: q_t or $\log q_t$. Quantities to retain: none beyond the evaluator. Operation: multiply three factors or add three log factors. Diagnostic: reject NaN, negative density values, or inconsistent coordinate block sizes.

Step 2: re-approximate by a TT.

Mathematical question. This paragraph answers: how does the nonseparable target become a separable object that can be marginalized?

Choose a variable order, in the paper usually

$$(x_t, \alpha, x_{t-1}).$$

Fit a TT

$$\widehat{\pi}_t(x_t, \alpha, x_{t-1}) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) H_{t,1}(x_{t-1,1}) \cdots H_{t,m_x}(x_{t-1,m_x}) \tag{15.3}$$

so that $\widehat{\pi}_t \approx q_t$. Zhao and Cui [4] use functional TT tools with alternating approximation and rank adaptation. For implementation one must choose a domain map, basis, fitting points, rank rule, and stopping rule. The paper's research code supplies one such family; the fixed-branch section below supplies a differentiable fixed-branch specialization.

The most direct implementation view is least squares. Choose fitting points

$$r^{(j)} = (x_t^{(j)}, \alpha^{(j)}, x_{t-1}^{(j)}), \qquad j = 1, \ldots, N_{\text{fit}},$$

and define target values

$$y_j = q_t(r^{(j)}).$$

For fixed TT ranks and fixed all cores except core k, the fitted function is linear in the entries of core C_k. There is a design row $A_{j,k}$ such that

$$\widehat{\pi}_t(r^{(j)}) = A_{j,k} g_k, \qquad g_k = \text{vec}(C_k). \tag{15.4}$$

Let $L_{j,k}$ be the left environment obtained by multiplying all cores before k at point $r^{(j)}$, and let $R_{j,k}$ be the right environment from all cores after k. Let $\psi_k(r_k^{(j)})$ be the local basis vector. Then

$$A_{j,k} = R_{j,k}^\top \otimes \psi_k(r_k^{(j)})^\top \otimes L_{j,k}. \tag{15.5}$$

This formula is worth lingering over. The left environment tells the current core what the earlier coordinates have already contributed. The right environment tells it what later coordinates will contribute. The basis vector gives the local coordinate dependence. The Kronecker product stitches these three pieces into a row that multiplies the vectorized core.

With weights $w_j > 0$, the core update solves

$$\min_{g_k} \sum_{j=1}^{N_{\text{fit}}} w_j \big(A_{j,k} g_k - y_j\big)^2 + \rho \|g_k\|_2^2, \tag{15.6}$$

or equivalently

$$(A_k^\top W A_k + \rho I) g_k = A_k^\top W y. \tag{15.7}$$

An alternating scheme sweeps through the cores, updating one core at a time. Zhao and Cui [4]'s adaptive implementation is more sophisticated, but this least- squares view is the easiest way to see how a TT approximation becomes an actual numerical object.

Implementation meaning. Inputs: target evaluator q_t, domains, basis objects, fitting points, weights, ranks, and sweep settings. Output: cores F, A, H for the fitted TT. Quantities to retain: cores, rank vector, basis definition, fit residual, and any scaling shift. Operation: alternating core least-squares or cross approximation. Diagnostic: weighted residual, condition number of each normal matrix, rank growth, and negative values if this basic non-squared algorithm is interpreted as a density.

Deterministic Fixed-Rank Sweep Protocol

For the fixed-branch derivative, the sweep rule must be a finite deterministic map. The branch declares

$$\mathcal{S}_{\mathrm{fit}} = \{Z_{\mathrm{fit}}, W, R_{0:D}, p_{1:D}, \rho, S_{\max}, \varepsilon_{\mathrm{rel}}, \varepsilon_{\mathrm{abs}}, s_{\mathrm{rescale}}\}. \tag{15.8}$$

Here Z_{fit} are fitting points, W are weights, $R_{0:D}$ are fixed ranks, $p_{1:D}$ are basis sizes, ρ is the ridge parameter, $S_{\max}$ is the maximum number of bidirectional sweeps, $\varepsilon_{\mathrm{rel}}, \varepsilon_{\mathrm{abs}}$ are stopping tolerances, and s_{rescale} is the declared rescaling convention.

For each fitting point $z^{(j)} = (z_1^{(j)}, \ldots, z_D^{(j)})$, define the target value used by the sweep as

$$y_j = e^{c_t/2}\sqrt{\widetilde{q}_t(z^{(j)}; \beta)}, \qquad y = (y_1, \ldots, y_{N_{\mathrm{fit}}})^\top. \tag{15.9}$$

The current core matrices at that point are

$$H_k^{(j)} = \sum_{\ell=0}^{p_k-1} \psi_{k,\ell}(z_k^{(j)})\, C_k[:, \ell, :]. \tag{15.10}$$

Before updating core k, build the left and right environments from the current values of the other cores:

$$L_{j,1} = 1, \qquad L_{j,k+1} = L_{j,k} H_k^{(j)}, \tag{15.11}$$

and

$$R_{j,D+1} = 1, \qquad R_{j,k} = H_k^{(j)} R_{j,k+1}. \tag{15.12}$$

Thus $L_{j,k} \in \mathbb{R}^{1 \times R_{k-1}}$ summarizes coordinates left of the core being updated, while $R_{j,k+1} \in \mathbb{R}^{R_k \times 1}$ summarizes coordinates right of it. With the local basis row

$$b_{j,k}^\top = \big(\psi_{k,0}(z_k^{(j)}), \ldots, \psi_{k,p_k-1}(z_k^{(j)})\big), \tag{15.13}$$

the least-squares row for the vectorized core $g_k = \mathrm{vec}(C_k)$ is

$$A_k[j, :] = R_{j,k+1}^\top \otimes b_{j,k}^\top \otimes L_{j,k}. \tag{15.14}$$

The weighted normal-equation pieces are therefore

$$N_k = A_k^\top W A_k + \rho I, \qquad d_k = A_k^\top W y. \tag{15.15}$$

After solving for g_k, set $C_k = \mathrm{vec}^{-1}(g_k)$, recompute $H_k^{(j)}$, and update the affected environment before moving to the next core:

$$\text{forward pass: } L_{j,k+1} \leftarrow L_{j,k} H_k^{(j)}, \qquad \text{backward pass: } R_{j,k} \leftarrow H_k^{(j)} R_{j,k+1}. \tag{15.16}$$

This is the minimal deterministic sweep object. Adaptive TT-cross can choose new interpolation points, pivots, or ranks; the fixed-branch derivative here does not.

One forward sweep applies the core updates

$$k = 1, 2, \ldots, D, \qquad N_k g_k = d_k, \qquad N_k = A_k^\top W A_k + \rho I. \tag{15.17}$$

One backward sweep applies the same update in the reverse order

$$k = D, D-1, \ldots, 1. \tag{15.18}$$

After a complete bidirectional sweep s, compute the weighted residual

$$r_s^2 = \frac{\sum_{j=1}^{N_{\mathrm{fit}}} w_j \left[\phi_s(z^{(j)}) - y_j\right]^2}{\sum_{j=1}^{N_{\mathrm{fit}}} w_j y_j^2 + \varepsilon_{\mathrm{abs}}}. \tag{15.19}$$

The deterministic stopping rule is

$$s = s_{\max} \quad \text{or} \quad r_s \le \varepsilon_{\mathrm{rel}} \quad \text{or} \quad |r_{s-1} - r_s| \le \varepsilon_{\mathrm{abs}}. \tag{15.20}$$

For a same-scalar derivative, the realized number of sweeps is frozen:

$$S = S(\beta_0), \qquad S(\beta_0 + h) = S(\beta_0 - h) = S. \tag{15.21}$$

Thus finite differences compare the same finite composition of solve maps. If the residual criterion would stop at a different sweep count for a perturbed parameter, the fixed-branch diagnostic must continue to the saved S rather than changing the branch.

Optional rescaling is allowed only if it is declared. A simple convention is to normalize each core after its update and carry the product of normalization constants into the global shift:

$$C_k \leftarrow C_k/\alpha_k, \qquad c_t \leftarrow c_t - 2\sum_k \log \alpha_k. \tag{15.22}$$

If rescaling is not used, record $s_{\mathrm{rescale}} = \text{none}$. The failure recovery rule for the fixed-rank protocol is not to increase rank silently. Instead record

$$r_S > \varepsilon_{\mathrm{rel}} \quad \Rightarrow \quad \text{fixed-rank fit failure for this branch.} \tag{15.23}$$

Rank adaptation may be studied later, but it is a different branch.

Step 3: integrate the old state.

Mathematical question. This paragraph answers: after the three-block target is fitted, what is retained for the next sequential update?

The next approximate filter is

$$\widehat{\pi}_t(x_t, \alpha) = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_{t-1}. \tag{15.24}$$

Because x_{t-1} occupies the last block, this integral is a sequence of core integrals:

$$\widehat{\pi}_t(x_t, \alpha) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) \overline{H}_{t,1} \cdots \overline{H}_{t,m_x}. \tag{15.25}$$

The approximate normalizer is

$$\widehat{Z}_t = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_t\ \mathrm{d}\alpha\ \mathrm{d}x_{t-1}, \tag{15.26}$$

again a complete TT contraction.

The same fitted TT gives the parameter marginal and filtering marginal:

$$\widehat{p}_t(\alpha) = \frac{\int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_t\ \mathrm{d}x_{t-1}}{\widehat{Z}_t}, \qquad \widehat{p}_t(x_t) = \frac{\int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}\alpha\ \mathrm{d}x_{t-1}}{\widehat{Z}_t}. \tag{15.27}$$

The retained object for the next time step is not merely a scalar normalizer. It is the evaluable two-block density

$$\widehat{\pi}_t(x_t, \alpha) = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_{t-1}, \tag{15.28}$$

because time $t+1$ will plug this function into the next target q_{t+1}.

Implementation meaning. Inputs: fitted three-block TT and block ordering (x_t, α, x_{t-1}). Outputs: retained evaluator $\widehat{\pi}_t(x_t, \alpha)$, normalizer $\widehat{Z}_t$, optional marginals $\widehat{p}_t(x_t)$ and $\widehat{p}_t(\alpha)$. Quantities to retain: retained TT/evaluator and contraction data. Operation: endpoint TT core integration. Diagnostic: $\widehat{Z}_t$ must be finite and positive, and normalizing retained marginals must integrate to one.

The main failure mode of Algorithm 1 is sign. A fitted TT is a real-valued function approximation. Truncation or interpolation can make $\widehat{\pi}_t(r_t) < 0$, which is impossible for a density. Negative values also break transport maps and importance-sampling ratios. This motivates the squared-TT construction.

16 Why Nonnegativity Fails And Why Square-Root TT Repairs It

Mathematical question. This paragraph answers: why is Algorithm 1 not enough for transport maps and importance correction?

Approximating a nonnegative function by a truncated expansion does not preserve nonnegativity. The one-dimensional analogy is simple:

$$h(x) \geq 0 \quad \text{but} \quad \widehat{h}(x) = \sum_{\ell=0}^{p-1} c_\ell \psi_\ell(x) \text{ may be negative.}$$

The tensor-train case has the same issue. A low-rank approximation can reduce error in a least-squares sense while still creating small negative regions.

Zhao and Cui [4]'s repair is to approximate the square root of an unnormalized density. If

$$\sqrt{\pi(r)} \approx \phi(r),$$

then

$$\phi(r)^2 \geq 0$$

for all r. A small defensive density is then added so that the proposal density is positive wherever the target might be positive.

17 Squared-TT Density, Defensive Reference, And Normalizer

Mathematical question. The question answered here is: how can a TT approximation be forced to define a nonnegative density whose support covers the target?

Zhao and Cui [4], Eq. (13) becomes the following. Let $\pi(r) \geq 0$ be an unnormalized target on a domain $\Omega \subseteq \mathbb{R}^D$, with normalizer

$$Z = \int_\Omega \pi(r)\, \mathrm{d}r.$$

Let $\lambda(r)$ be a normalized reference density on Ω,

$$\lambda(r) \geq 0, \qquad \int_\Omega \lambda(r)\, \mathrm{d}r = 1,$$

and choose $\tau > 0$. Fit a TT $\phi(r)$ to $\sqrt{\pi(r)}$. Define

$$\widehat{\pi}(r) = \phi(r)^2 + \tau\lambda(r), \qquad \widehat{Z} = \int_\Omega \widehat{\pi}(r)\,\mathrm{d}r, \qquad \widehat{p}(r) = \frac{\widehat{\pi}(r)}{\widehat{Z}}. \tag{17.1}$$

Since λ is normalized,

$$\widehat{Z} = \int_\Omega \phi(r)^2\,\mathrm{d}r + \tau. \tag{17.2}$$

The defensive term has two jobs. First, it keeps the approximation positive even if ϕ is zero somewhere. Second, if π/λ is bounded, then

$$\frac{p(r)}{\widehat{p}(r)} = \frac{\pi(r)/Z}{\widehat{\pi}(r)/\widehat{Z}} = \frac{\widehat{Z}}{Z}\frac{\pi(r)}{\phi(r)^2 + \tau\lambda(r)} \le \frac{\widehat{Z}}{Z\tau}\frac{\pi(r)}{\lambda(r)}. \tag{17.3}$$

This is the importance-sampling reason for the defensive density: the true target is absolutely continuous with respect to the approximation when the reference covers the target support.

Zhao and Cui [4]'s Lemma 1 ties the square-root fit to density error. In the notation above, suppose

$$\|\phi - \sqrt{\pi}\|_{L^2} \le \varepsilon, \qquad \sqrt{\tau} \le \|\phi - \sqrt{\pi}\|_{L^2}. \tag{17.4}$$

Then the square-root error of the defensive approximation satisfies

$$\|\sqrt{\widehat{\pi}} - \sqrt{\pi}\|_{L^2} \le 2\varepsilon, \tag{17.5}$$

and the normalized Hellinger distance obeys

$$D_H(\widehat{p}, p) \le \frac{\sqrt{2}}{\sqrt{Z}}\|\sqrt{\widehat{\pi}} - \sqrt{\pi}\|_{L^2} \le \frac{2\sqrt{2}\varepsilon}{\sqrt{Z}}. \tag{17.6}$$

For this note, the important lesson is local: fitting the square root in L^2 controls the normalized density in Hellinger distance after normalization. The lemma does not say ranks will be small, nor does it make an adaptive implementation globally differentiable.

Local assumptions for Lemma 1. This note uses only the following local content: if the square-root fit is small in L^2, then the normalized defensive density is close in Hellinger distance under the stated support and finite-normalizer conditions. P18 does not re-prove the external Cui and Dolgov [1] theorem, does not claim a rank bound, and does not claim adaptive differentiability.

Implementation meaning. 1-Eq13. Inputs are a square-root target evaluator, a reference density λ, a defensive mass τ, and a fitted TT ϕ. Outputs are $\widehat{\pi}$, $\widehat{Z}$, and $\widehat{p}$. Quantities to retain is ϕ's cores, λ, τ, and $\widehat{Z}$. Diagnostics are positivity, finite $\widehat{Z}$, support coverage by λ, and the defensive ratio $\tau/\widehat{Z}$.

18 Squared-TT Marginalization And Mass Matrices

Mathematical question. This proposition answers: how do we marginalize a squared TT without expanding the square into a full tensor?

The square changes the marginalization algebra. Let

$$\phi(r) = H_1(r_1)\cdots H_D(r_D), \qquad H_k(r_k) \in \mathbb{R}^{R_{k-1}\times R_k}.$$

For a split after coordinate k, define

$$H_{\le k}(r_{\le k}) = H_1(r_1)\cdots H_k(r_k) \in \mathbb{R}^{1\times R_k},$$

and

$$H_{>k}(r_{>k}) = H_{k+1}(r_{k+1})\cdots H_D(r_D) \in \mathbb{R}^{R_k\times 1}.$$

Then

$$\phi(r) = H_{\le k}(r_{\le k})H_{>k}(r_{>k}).$$

The marginal square term is

$$\begin{aligned}\int \phi(r)^2\,\mathrm{d}r_{>k} &= \int \left[H_{\le k}(r_{\le k})H_{>k}(r_{>k})\right]^2\,\mathrm{d}r_{>k}\\ &= \sum_{a=1}^{R_k}\sum_{b=1}^{R_k} H_{\le k,a}(r_{\le k})\left[\int H_{>k,a}(r_{>k})H_{>k,b}(r_{>k})\,\mathrm{d}r_{>k}\right] H_{\le k,b}(r_{\le k}).\end{aligned} \tag{18.1}$$

Define the right mass matrix

$$M_{>k}[a,b] = \int H_{>k,a}(r_{>k})H_{>k,b}(r_{>k})\,\mathrm{d}r_{>k}. \tag{18.2}$$

This matrix is positive semidefinite. If $M_{>k} = L_{>k}L_{>k}^\top$ is a Cholesky or square-root factorization, then

$$\int \phi(r)^2\,\mathrm{d}r_{>k} = \sum_{\gamma=1}^{R_k}\left[\sum_{a=1}^{R_k} H_{\le k,a}(r_{\le k})L_{>k}[a,\gamma]\right]^2. \tag{18.3}$$

Including the defensive reference, if $\lambda(r) = \prod_{j=1}^D \lambda_j(r_j)$, gives

$$\widehat{p}(r_{\le k}) = \frac{\sum_{\gamma=1}^{R_k}\left[\sum_{a=1}^{R_k} H_{\le k,a}(r_{\le k})L_{>k}[a,\gamma]\right]^2 + \tau\prod_{j=1}^k \lambda_j(r_j)}{\widehat{Z}}. \tag{18.4}$$

This is Zhao and Cui [4], Eq. (14) in the notation of this note.

The practical recursion starts from the right end. Set $M_{>D} = 1$. Moving from $j = D$ down to 1, compute

$$M_{>j-1} = \int H_j(r_j)M_{>j}H_j(r_j)^\top\,\mathrm{d}r_j. \tag{18.5}$$

In basis coefficients this integral is finite-dimensional. If

$$H_j^{a,b}(r_j) = \sum_\ell C_j[a,\ell,b]\psi_{j,\ell}(r_j),$$

and

$$B_j[\ell,\ell'] = \int \psi_{j,\ell}(r_j)\psi_{j,\ell'}(r_j)\,\mathrm{d}r_j,$$

then

$$M_{>j-1}[a,a'] = \sum_{b,b'=1}^{R_j}\sum_{\ell,\ell'} C_j[a,\ell,b]\,M_{>j}[b,b']\,C_j[a',\ell',b']\,B_j[\ell,\ell']. \tag{18.6}$$

This formula is the bridge from the paper to code: marginalization stores mass matrices or their square roots and never enumerates a full tensor grid.

Implementation meaning. 1-Proposition2. Inputs are TT cores, basis mass matrices B_j, defensive reference factors λ_j, and τ. Outputs are marginal density evaluators and mass contractions $M_{>k}$ or their Cholesky factors. Quantities to retain is the list of mass matrices needed by later conditional CDFs. Diagnostics are symmetry error $\|M - M^\top\|$, Cholesky failure, negative eigenvalues beyond tolerance, and normalizer mismatch between independent contraction paths.

Local assumptions for Proposition 2. This note uses the finite-dimensional contraction identity for a squared TT with integrable basis products and positive semidefinite mass matrices. The note derives the contraction algebra explicitly in (18.1)–(18.9). It does not claim that every numerical Cholesky factorization is stable; that is a diagnostic obligation.

18.1 A Small Three-Coordinate Marginalization

Take $D = 3$, and suppose

$$\phi(r_1, r_2, r_3) = H_1(r_1)H_2(r_2)H_3(r_3).$$

If $H_1 \in \mathbb{R}^{1\times R_1}$, $H_2 \in \mathbb{R}^{R_1\times R_2}$, and $H_3 \in \mathbb{R}^{R_2\times 1}$, then marginalizing out r_3 gives

$$\int \phi(r_1, r_2, r_3)^2 \, \mathrm{d}r_3 = H_1(r_1)H_2(r_2) \left[\int H_3(r_3)H_3(r_3)^\top \, \mathrm{d}r_3\right] H_2(r_2)^\top H_1(r_1)^\top. \tag{18.7}$$

The bracketed quantity is an $R_2 \times R_2$ matrix. It is not a function of r_1 or r_2; it is a stored contraction. If we also marginalize out r_2, then

$$M_{>1} = \int H_2(r_2) \left[\int H_3(r_3)H_3(r_3)^\top \, \mathrm{d}r_3\right] H_2(r_2)^\top \, \mathrm{d}r_2, \tag{18.8}$$

and the one-coordinate marginal is

$$\int \phi(r_1, r_2, r_3)^2 \, \mathrm{d}r_2 \, \mathrm{d}r_3 = H_1(r_1)M_{>1}H_1(r_1)^\top. \tag{18.9}$$

This is exactly what the recursive mass formula (M5) does. The notation in the paper is compact because it assumes the reader already sees this matrix chain. The implementation sees it as repeated multiplication and integration of small matrices.

19 Conditional Densities And KR Maps From Marginal Ratios

Mathematical question. The question answered here is: how do marginal densities become a transport map that can sample from the approximate density?

For a normalized density $\widehat{p}(r_1, \ldots, r_D)$, the chain rule gives

$$\widehat{p}(r) = \widehat{p}(r_1) \prod_{k=2}^{D} \widehat{p}(r_k \mid r_{<k}), \qquad \widehat{p}(r_k \mid r_{<k}) = \frac{\widehat{p}(r_{\le k})}{\widehat{p}(r_{<k})}. \tag{19.1}$$

The squared-TT marginal formula (18.4) supplies both numerator and denominator. Thus each conditional density is computable from TT contractions.

Define conditional distribution functions

$$F_1(r_1) = \int_{-\infty}^{r_1} \widehat{p}(s_1) \, \mathrm{d}s_1, \tag{19.2}$$

and, for $k \geq 2$,

$$F_k(r_k \mid r_{<k}) = \int_{-\infty}^{r_k} \widehat{p}(s_k \mid r_{<k}) \, \mathrm{d}s_k. \tag{19.3}$$

The lower triangular Knothe–Rosenblatt map is

$$F(r) = \big(F_1(r_1), F_2(r_2 \mid r_1), \ldots, F_D(r_D \mid r_{<D})\big). \tag{19.4}$$

If $R \sim \widehat{p}$, then $F(R)$ is uniform on $[0,1]^D$. Conversely, if $\Xi \sim \mathrm{Unif}([0,1]^D)$, then $F^{-1}(\Xi) \sim \widehat{p}$.

The proof is the triangular Jacobian calculation. The Jacobian of F is lower triangular, and

$$\frac{\partial F_k(r_k \mid r_{<k})}{\partial r_k} = \widehat{p}(r_k \mid r_{<k}).$$

Therefore

$$|\det \nabla F(r)| = \prod_{k=1}^{D} \widehat{p}(r_k \mid r_{<k}) = \widehat{p}(r). \tag{19.5}$$

The pullback of the uniform density by F is exactly $|\det \nabla F(r)| = \widehat{p}(r)$.

Why This Transport Is The Right Device

The KR map is useful because it turns the question "how do we sample from a high-dimensional density?" into a sequence of one-dimensional questions. Start with the ordinary one-dimensional probability integral transform:

$$U = F_X(X) = \int_{-\infty}^{X} p_X(s) \, \mathrm{d}s, \qquad X \sim p_X, \tag{19.6}$$

which implies $U \sim \mathrm{Unif}(0,1)$. Conversely,

$$X = F_X^{-1}(U), \qquad U \sim \mathrm{Unif}(0,1), \tag{19.7}$$

has density p_X. The multivariate construction repeats this idea after conditioning on the coordinates that have already been chosen:

$$\begin{aligned} u_1 &= F_1(r_1), \\ u_2 &= F_2(r_2 \mid r_1), \\ u_3 &= F_3(r_3 \mid r_1, r_2), \\ &\vdots \\ u_D &= F_D(r_D \mid r_1, \ldots, r_{D-1}). \end{aligned} \tag{19.8}$$

The inverse construction is equally sequential:

$$\begin{aligned} r_1 &= F_1^{-1}(u_1), \\ r_2 &= F_2^{-1}(u_2 \mid r_1), \\ r_3 &= F_3^{-1}(u_3 \mid r_1, r_2), \\ &\vdots \\ r_D &= F_D^{-1}(u_D \mid r_1, \ldots, r_{D-1}). \end{aligned} \tag{19.9}$$

Thus a product draw $u \sim \mathrm{Unif}([0,1]^D)$ becomes a dependent draw $r \sim \widehat{p}$. This is the reason the map is a transport map: it transports the simple product measure to the fitted dependent density.

The triangular form is the essential feature. Its Jacobian matrix has zeros above the diagonal,

$$\nabla F(r) = \begin{bmatrix} \partial_{r_1} F_1 & 0 & \cdots & 0 \\ \partial_{r_1} F_2 & \partial_{r_2} F_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \partial_{r_1} F_D & \partial_{r_2} F_D & \cdots & \partial_{r_D} F_D \end{bmatrix}, \tag{19.10}$$

so the determinant is just the product of one-dimensional conditional densities:

$$\det \nabla F(r) = \prod_{k=1}^{D} \partial_{r_k} F_k(r_k \mid r_{<k}) = \prod_{k=1}^{D} \widehat{p}(r_k \mid r_{<k}) = \widehat{p}(r). \tag{19.11}$$

For the fitted squared TT, the hard part is not this determinant identity; it is obtaining the conditional densities in the middle product. The squared-TT mass contractions provide exactly those marginal ratios:

$$\widehat{p}(r_k \mid r_{<k}) = \frac{\widehat{p}(r_{\le k})}{\widehat{p}(r_{<k})} = \frac{\text{TT contraction retaining } r_{\le k}}{\text{TT contraction retaining } r_{<k}}. \tag{19.12}$$

This is why conditional/KR maps are not an ornament added after fitting. They are the natural way to convert the fitted nonnegative density into a sampler, a particle proposal, or a backward smoother.

Worked Two-Dimensional KR Example

For $D = 2$, write the normalized approximate density as

$$\widehat{p}(r_1, r_2) \ge 0, \qquad \iint \widehat{p}(r_1, r_2)\, \mathrm{d}r_1\, \mathrm{d}r_2 = 1. \tag{19.13}$$

The first marginal is

$$\widehat{p}_1(r_1) = \int \widehat{p}(r_1, s_2)\, \mathrm{d}s_2, \tag{19.14}$$

and the second conditional density is

$$\widehat{p}_{2|1}(r_2 \mid r_1) = \frac{\widehat{p}(r_1, r_2)}{\widehat{p}_1(r_1)} \quad \text{when } \widehat{p}_1(r_1) > 0. \tag{19.15}$$

The two conditional distribution functions are

$$F_1(r_1) = \int_{a_1}^{r_1} \widehat{p}_1(s_1)\, \mathrm{d}s_1, \tag{19.16}$$

and

$$F_2(r_2 \mid r_1) = \int_{a_2}^{r_2} \widehat{p}_{2|1}(s_2 \mid r_1)\, \mathrm{d}s_2. \tag{19.17}$$

The lower triangular map is

$$F(r_1, r_2) = \big(F_1(r_1), F_2(r_2 \mid r_1)\big). \tag{19.18}$$

Its Jacobian matrix is

$$\nabla F(r_1, r_2) = \begin{bmatrix} \partial_{r_1} F_1(r_1) & 0 \\ \partial_{r_1} F_2(r_2 \mid r_1) & \partial_{r_2} F_2(r_2 \mid r_1) \end{bmatrix}. \tag{19.19}$$

Because the matrix is triangular,

$$\det \nabla F = \widehat{p}_1(r_1)\widehat{p}_{2|1}(r_2 \mid r_1) = \widehat{p}(r_1, r_2). \tag{19.20}$$

Now connect this analytic conditional density to the object stored by a squared-TT implementation. In two dimensions, write

$$\widehat{p}(r_1, r_2) = \frac{\phi(r_1, r_2)^2 + \tau\lambda(r_1, r_2)}{\widehat{Z}}. \tag{19.21}$$

Suppose the square-root TT has two cores,

$$\phi(r_1, r_2) = H_1(r_1)H_2(r_2), \qquad H_1(r_1) \in \mathbb{R}^{1\times R_1}, \quad H_2(r_2) \in \mathbb{R}^{R_1\times 1}. \tag{19.22}$$

The square-mass object for the second coordinate is

$$G_2 = \int H_2(s)H_2(s)^\top \, \mathrm{d}s \in \mathbb{R}^{R_1\times R_1}. \tag{19.23}$$

Therefore the unnormalized first marginal is

$$n_1(r_1) = H_1(r_1)G_2H_1(r_1)^\top + \tau\lambda_1(r_1), \qquad \lambda_1(r_1) = \int \lambda(r_1, s)\, \mathrm{d}s. \tag{19.24}$$

Dividing by $\widehat{Z}$ gives $\widehat{p}_1(r_1) = n_1(r_1)/\widehat{Z}$. The conditional density evaluator used by the CDF machinery is

$$d_2(s \mid r_1) = \widehat{p}_{2|1}(s \mid r_1) = \frac{\phi(r_1, s)^2 + \tau\lambda(r_1, s)}{n_1(r_1)}. \tag{19.25}$$

The global normalizer cancels. This is the exact two-dimensional bridge from TT square-mass contraction to the operational conditional density in (19.35)–(19.37).

To sample from $\widehat{p}$, draw $u_1, u_2 \sim \mathrm{Unif}(0, 1)$ and solve

$$F_1(r_1) = u_1, \qquad F_2(r_2 \mid r_1) = u_2. \tag{19.26}$$

Endpoint and monotonicity diagnostics are

$$F_1(a_1) = 0, \quad F_1(b_1) = 1, \quad F_2(a_2 \mid r_1) = 0, \quad F_2(b_2 \mid r_1) = 1, \tag{19.27}$$

and

$$\partial_{r_1} F_1(r_1) \geq 0, \qquad \partial_{r_2} F_2(r_2 \mid r_1) \geq 0. \tag{19.28}$$

If these fail, the map is not a valid sampler until the conditional CDF representation is repaired.

Mathematical question. Remark 3 answers: what changes if the conditional map is built from the right end rather than from the left end?

The paper also states the reverse-order construction. If instead we compute right-to-left marginals

$$\widehat{p}(r_{\geq k}) = \int \widehat{p}(r)\, \mathrm{d}r_{<k}, \tag{19.29}$$

then the conditional densities are

$$\widehat{p}(r_k \mid r_{>k}) = \frac{\widehat{p}(r_{\geq k})}{\widehat{p}(r_{>k})}, \tag{19.30}$$

and the upper triangular KR map is

$$F^u(r) = \big(F_1^u(r_1 \mid r_{>1}), \ldots, F_{D-1}^u(r_{D-1} \mid r_D), F_D^u(r_D)\big). \tag{19.31}$$

This is not a different density approximation. It is the same density integrated in the opposite direction. Zhao and Cui [4] use the lower map for backward smoothing and the upper map for forward particle proposals.

Implementation meaning. Inputs are the same squared-TT density and the choice of reverse integration direction. Outputs are right-to-left mass contractions and upper-triangular conditional CDFs. Quantities to retain must record the direction, because using lower-map contractions inside an upper-map proposal silently changes the proposal density. The failure check is endpoint normalization of every reverse conditional CDF.

The computational costs in the paper separate preprocessing from per-sample map evaluation. Building all conditional densities from the squared-TT mass recursions costs

$$O(DpR^3). \tag{19.32}$$

For N samples, evaluating conditional densities along a triangular map costs approximately

$$O(NDpR^2), \tag{19.33}$$

and constructing/evaluating one-dimensional distribution functions adds basis and root-finding work. If c_{root} is the fixed number of root-finding iterations, inverse-map evaluation scales as

$$O(DpR^3 + NDpR^2 + NDp(\log p + R) + NDp\, c_{\text{root}}). \tag{19.34}$$

The formula is less important than the decomposition: build the conditional objects once, then pay a per-sample triangular evaluation cost.

Implementation meaning. 1-KR-ratios. Inputs are marginal density evaluators from Proposition 2. Outputs are conditional CDF evaluators and inverse-CDF samplers. Quantities to retain includes integration direction, mass matrices, one-dimensional CDF coefficients, and root-finding tolerances. Diagnostics are monotonicity of each CDF, endpoint values near 0 and 1, and inverse residuals.

Operational KR Construction Details

For each conditional coordinate k, store a conditional density evaluator

$$d_k(s \mid r_{<k}) = \widehat{p}(r_k = s \mid r_{<k}). \tag{19.35}$$

Represent the conditional CDF by quadrature on the coordinate interval $[a_k, b_k]$:

$$F_k(s \mid r_{<k}) = \int_{a_k}^{s} d_k(u \mid r_{<k})\, \mathrm{d}u. \tag{19.36}$$

A deterministic baseline is Gauss–Legendre subinterval quadrature. For nodes ξ_ν and weights ω_ν on $[a_k, s]$,

$$F_k(s \mid r_{<k}) \approx \sum_{\nu=1}^{N_q} \omega_\nu d_k(\xi_\nu \mid r_{<k}). \tag{19.37}$$

After evaluation, clip only the reported probability, not the density:

$$F_k^{\text{clip}}(s \mid r_{<k}) = \min\{1, \max\{0, F_k(s \mid r_{<k})\}\}. \tag{19.38}$$

The branch records whether clipping occurred. Clipping is a diagnostic, not a mathematical cure.

The inverse problem is

$$F_k(s \mid r_{<k}) - u = 0. \tag{19.39}$$

The baseline inverse rule is bisection:

$$[a^{(0)}, b^{(0)}] = [a_k, b_k], \qquad m^{(n)} = \frac{a^{(n)} + b^{(n)}}{2}. \tag{19.40}$$

If $F_k(m^{(n)} \mid r_{<k}) < u$, set

$$a^{(n+1)} = m^{(n)}, \qquad b^{(n+1)} = b^{(n)}; \tag{19.41}$$

otherwise set

$$a^{(n+1)} = a^{(n)}, \qquad b^{(n+1)} = m^{(n)}. \tag{19.42}$$

Stop when

$$b^{(n)} - a^{(n)} \le \varepsilon_{\text{root}} \quad \text{or} \quad |F_k(m^{(n)} \mid r_{<k}) - u| \le \varepsilon_{\text{cdf}}. \tag{19.43}$$

The operational diagnostic vector is

$$\mathcal{D}_{\text{KR}} = \{F_k(a_k), F_k(b_k), \min \Delta F_k, \max |F_k(F_k^{-1}(u)) - u|, \text{clip count}\}. \tag{19.44}$$

If $F_k(b_k)$ differs from 1, if $\min \Delta F_k < -\varepsilon_{\text{mono}}$, or if inverse residuals exceed tolerance, the conditional map is rejected for that branch.

Local assumptions for KR exactness. This note uses the triangular CDF identity under positive conditional densities, well-defined marginal ratios, and invertible one-dimensional conditional CDFs. If a conditional CDF is flat, discontinuous, numerically non-monotone, or unsupported by the reference domain, the KR sampler is not valid until that failure is fixed.

20 Algorithm 2, Fully Annotated

Mathematical question. Algorithm 2 answers: how does the sequential recursion change when the fitted object is the square root and the stored density is squared?

Algorithm 2 is Algorithm 1 with a square-root TT and a defensive density. At time $t-1$, assume the previous filter approximation is evaluable as

$$\widehat{p}_{t-1}(x_{t-1}, \alpha).$$

The pointwise unnormalized target, Zhao and Cui [4], Eq. (15), is

$$q_t(x_t, \alpha, x_{t-1}) = \widehat{p}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \tag{20.1}$$

Fit a TT to the square root:

$$\sqrt{q_t(x_t, \alpha, x_{t-1})} \approx \phi_t(x_t, \alpha, x_{t-1}). \tag{20.2}$$

Then define Zhao and Cui [4], Eq. (16):

$$\widehat{q}_t(x_t, \alpha, x_{t-1}) = \phi_t(x_t, \alpha, x_{t-1})^2 + \tau_t \lambda_t(x_t) \lambda_\alpha(\alpha) \lambda_{t-1}(x_{t-1}). \tag{20.3}$$

The approximate normalizer is

$$\widehat{Z}_t = \int \widehat{q}_t(x_t, \alpha, x_{t-1}) \, \mathrm{d}x_t \, \mathrm{d}\alpha \, \mathrm{d}x_{t-1}. \tag{20.4}$$

The next normalized approximate filter is

$$\widehat{p}_t(x_t, \alpha) = \frac{\int \widehat{q}_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_{t-1}}{\widehat{Z}_t}. \tag{20.5}$$

Everything is now nonnegative. The approximation risk has moved: the method must fit the square root accurately enough, choose a defensive term that is not too small for stability or too large for accuracy, and keep TT ranks manageable.

Implementation meaning. Inputs: previous retained density evaluator, model evaluators, reference factors $\lambda_t, \lambda_\alpha, \lambda_{t-1}, \tau_t$, basis/rank/fitting controls, and observation y_t. Outputs: square-root TT ϕ_t, nonnegative density evaluator $\widehat{q}_t$, normalizer $\widehat{Z}_t$, retained filter $\widehat{p}_t(x_t, \alpha)$, and mass contractions for conditional maps. Quantities to retain: all of those objects. Diagnostics: square-root fit residual, defensive mass ratio, $\widehat{Z}_t > 0$, rank growth, and Cholesky/mass-matrix health.

21 Forward Conditional Map And Particle-Filter Correction

Mathematical question. These equations answer: how does the squared-TT approximation become a particle proposal, and how is approximation bias corrected?

The same $\widehat{q}_t(x_t, \alpha, x_{t-1})$ defines a conditional density for the new state given the old state and parameter:

$$\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t}) = \frac{\widehat{q}_t(x_t, \alpha, x_{t-1})}{\int \widehat{q}_t(s, \alpha, x_{t-1})\, \mathrm{d}s}. \tag{21.1}$$

Constructing the corresponding upper triangular conditional KR map gives a proposal sampler:

$$\widehat{X}_t^{(i)} = (F_{t,t}^u)^{-1}\left(\Xi_t^{(i)} \mid \widehat{\alpha}^{(i)}, \widehat{x}_{t-1}^{(i)}\right), \qquad \Xi_t^{(i)} \sim \mathrm{Unif}([0,1]^{m_x}). \tag{21.2}$$

This is Zhao and Cui [4], Eq. (21) in the fixed-branch notation used here.

The proposal density for the full path after sampling is

$$\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t}) p(\alpha, x_{0:t-1} \mid y_{1:t-1}), \tag{21.3}$$

which corresponds to Zhao and Cui [4], Eq. (22). The exact posterior recursion is

$$p(\alpha, x_{0:t} \mid y_{1:t}) \propto p(\alpha, x_{0:t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Dividing exact target by proposal gives the unnormalized correction factor

$$\omega_f(\alpha, x_{t-1:t}) = \frac{f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t})}. \tag{21.4}$$

This is Zhao and Cui [4], Eq. (23). The correction is conceptually important: the TT proposal can be biased, but if its conditional density is evaluable and covers the target, importance weights can correct expectations.

With incoming normalized weights $W_{t-1}^{(i)}$, the particle-filter weight update is

$$\widetilde{W}_t^{(i)} = W_{t-1}^{(i)} \omega_f(\widehat{\alpha}^{(i)}, \widehat{x}_{t-1:t}^{(i)}), \tag{21.5}$$

followed by

$$W_t^{(i)} = \frac{\widetilde{W}_t^{(i)}}{\sum_{j=1}^N \widetilde{W}_t^{(j)}}. \tag{21.6}$$

Algorithm 3 is therefore an ordinary importance sampler whose proposal is defined by the upper conditional KR map of the TT approximation. The implementation stores the conditional-map object, the sampled $x_t^{(i)}$, the proposal density evaluator, and the normalized weights. A standard failure diagnostic is effective sample size,

$$\mathrm{ESS}_t = \frac{1}{\sum_{i=1}^{N} (W_t^{(i)})^2}. \tag{21.7}$$

Low ESS means that the TT proposal is not close enough to the exact recursive posterior for stable correction.

Implementation meaning. Inputs: weighted particles at time $t-1$, the upper conditional map, model density evaluators, and proposal density evaluator. Outputs: new particles and normalized weights. Quantities to retain: particles, weights, ESS, and optional resampling history. Operation: inverse triangular sampling plus importance-weight update. Diagnostics: ESS, infinite/zero proposal density, root-finding failure, and weight underflow.

22 Backward Conditional Map, Path Estimation, And Smoothing

Mathematical question. These equations answer: how can the same sequence of local TT approximations produce full paths and smoothed old states after all data are observed?

The lower conditional map goes the other direction:

$$\widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}) = \frac{\widehat{q}_t(x_t, \alpha, x_{t-1})}{\int \widehat{q}_t(x_t, \alpha, s)\, \mathrm{d}s}. \tag{22.1}$$

The corresponding inverse conditional KR map samples

$$\widehat{X}_{t-1}^{(i)} = (F_{t,t-1}^{l})^{-1}\left(\Xi_{t-1}^{(i)} \mid \widehat{X}_t^{(i)}, \widehat{\alpha}^{(i)}\right). \tag{22.2}$$

This is Zhao and Cui [4], Eq. (24).

Starting at final time T, draw

$$(\widehat{X}_T, \widehat{\alpha}, \widehat{X}_{T-1}) \sim \widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}),$$

then move backward using (22.2). The resulting approximate path density factors as Zhao and Cui [4], Eq. (25):

$$\widehat{p}(\alpha, x_{0:T} \mid y_{1:T}) = \widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} \widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}). \tag{22.3}$$

The exact smoothing posterior has the same Markov factorization form with exact conditionals:

$$p(\alpha, x_{0:T} \mid y_{1:T}) = p(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} p(x_{t-1} \mid x_t, \alpha, y_{1:t}). \tag{22.4}$$

The approximation can therefore be corrected by an importance ratio. The unnormalized backward weight, Zhao and Cui [4], Eq. (26), is

$$\omega_b(\alpha, x_{0:T}) = \frac{p(\alpha) p_\alpha(x_0) \prod_{t=1}^{T} f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{\widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} \widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t})}. \tag{22.5}$$

The numerator is the exact unnormalized path posterior. The denominator is the proposal density generated by the backward TT maps.

Algorithm 4 normalizes the backward weights in the same way:

$$W^{(i)} = \frac{\omega_b(\widehat{\alpha}^{(i)}, \widehat{x}_{0:T}^{(i)})}{\sum_{j=1}^{N} \omega_b(\widehat{\alpha}^{(j)}, \widehat{x}_{0:T}^{(j)})}. \tag{22.6}$$

The paper emphasizes a Markov identity used by the backward recursion:

$$p(x_{t-1} \mid \alpha, x_t, y_{1:t}) = p(x_{t-1} \mid \alpha, x_t, y_{1:T}), \qquad t < T. \tag{22.7}$$

The reason is conditional independence. Once (α, x_t) are known, the future observations $y_{t+1:T}$ depend on the past state x_{t-1} only through x_t. This is why a conditional object built at time t can be used inside a final-time smoothing path.

Zhao and Cui [4] also give the computational reason for the variable order (x_t, α, x_{t-1}). Marginalizing a squared TT from an end costs

$$O(pR^3) \quad \text{per coordinate}, \tag{22.8}$$

because it updates one boundary mass matrix. Marginalizing a coordinate in the middle can cost

$$O(pR^6) \quad \text{per coordinate}, \tag{22.9}$$

because left and right rank couplings must be paired. The order (x_t, α, x_{t-1}) lets Algorithm 2 integrate x_{t-1} from the right end and lets the smoothing map reuse the same direction. The upper map for particle filtering goes in the opposite direction and therefore incurs an additional $O(mpR^3)$-type preprocessing cost.

Implementation meaning. Inputs: final-time approximation, lower conditional maps for all times, model density evaluators, and reference uniform samples. Outputs: path samples and normalized smoothing weights. Quantities to retain: map sequence, samples, weights, and ESS. Operation: final joint sample, then backward inverse-map sampling. Diagnostics: backward proposal support, weight variance, root residuals, and mismatch between source-time conditionals and final-time path use.

23 Error Propagation: What It Proves And What It Does Not Prove

Section 4 of Zhao and Cui [4] explains how local approximation errors propagate. The key identity is the triangle inequality. Let π_t be the exact unnormalized joint density of (x_t, α, x_{t-1}) at time t, q_t the propagated density using the previous approximation, and $\widehat{\pi}_t$ the new TT approximation. Then

$$D(\widehat{\pi}_t, \pi_t) \le D(\widehat{\pi}_t, q_t) + D(q_t, \pi_t). \tag{23.1}$$

The first term is the new fit error. The second is inherited from the previous time step. Under bounded likelihood or bounded prediction-likelihood conditions, the inherited error is multiplied by a finite constant.

For squared TT, Zhao and Cui [4] use an L^2 error on square roots because it controls Hellinger distance after normalization. The practical message is precise but modest: if every local square-root approximation is accurate enough and the model does not amplify errors too severely, the sequence of approximate densities can remain controlled. This is not a theorem that ranks will stay small, that the companion implementation is globally differentiable, or that a particular high-dimensional model will be accurate without diagnostics.

24 Preconditioning

Mathematical question. These equations answer: how can a hard target density be transformed into a less concentrated residual before fitting a TT?

The direct target $q_t(x_t, \alpha, x_{t-1})$ can be sharply concentrated. A TT may then need high rank. Zhao and Cui [4], Section 5 introduces a change of variables that makes the function closer to a product reference density before fitting.

Let

$$r = (x_t, \alpha, x_{t-1}), \qquad u = T_t(r),$$

and let $\eta(u) = \eta_1(u_1) \cdots \eta_D(u_D)$ be a product reference density. Choose a bridging density $\rho_t(r)$ that is easier to approximate than $q_t(r)$, and construct T_t so that the pushforward of ρ_t is close to η:

$$(T_t)_{\#}\rho_t(u) = \rho_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right| \propto \eta(u). \tag{24.1}$$

This is Zhao and Cui [4], Eq. (30).

The pushforward of q_t is

$$(T_t)_{\#}q_t(u) = q_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right|.$$

Using (24.1), the Jacobian factor is proportional to $\eta(u)/\rho_t(T_t^{-1}(u))$, so the function to fit is

$$q_t^{\sharp}(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))}\eta(u). \tag{24.2}$$

This is Zhao and Cui [4], Eq. (31). If ρ_t captures the main shape of q_t, then q_t/ρ_t is flatter than q_t, and TT ranks may be smaller.

In density language, the preconditioner should move the hard geometry into the map T_t. The residual $q_t^{\sharp}$ then asks the TT to learn only what the bridge missed. The implementation failure mode is division by a bridge that is too small: $\rho_t(T_t^{-1}(u))$ or $\widehat{\rho}_t(T_t^{-1}(u))$ must not vanish on target-relevant points.

Before/After Flattening Calculation

The preconditioning claim can be written as a before/after ratio. Before preconditioning, the TT sees the target

$$q(r). \tag{24.3}$$

Choose a positive bridge $\rho(r)$ and write

$$q(r) = \rho(r)\, m(r), \qquad m(r) = \frac{q(r)}{\rho(r)}. \tag{24.4}$$

If ρ captures the main concentration, then the multiplier m varies more slowly than q. In logarithmic form,

$$\log q(r) = \log \rho(r) + \log m(r). \tag{24.5}$$

The bridge map T moves ρ to a product reference η:

$$\rho(T^{-1}(u)) \left|\det \nabla T^{-1}(u)\right| = Z_\rho\, \eta(u). \tag{24.6}$$

Therefore the transformed target is

$$q^{\sharp}(u) = q(T^{-1}(u)) \left|\det \nabla T^{-1}(u)\right| = Z_\rho\, m(T^{-1}(u))\eta(u). \tag{24.7}$$

The TT now fits the residual multiplier times a product reference, not the full concentrated target:

$$\sqrt{q^\sharp(u)} = \sqrt{Z_\rho}\,\sqrt{m(T^{-1}(u))}\sqrt{\eta(u)}. \tag{24.8}$$

The rank-pressure statement is qualitative but precise:

$$\text{if } m(T^{-1}(u)) \text{ has weaker cross-coordinate coupling than } q(r), \text{ then smaller TT ranks may suffice.} \tag{24.9}$$

The failure case is equally explicit. If

$$\inf_{r\in\Omega_{\text{fit}}} \rho(r) \approx 0 \quad \text{while} \quad q(r) > 0, \tag{24.10}$$

then $m = q/\rho$ can explode and the residual is harder, not easier. The bridge-support diagnostic is therefore

$$\kappa_{\text{bridge}} = \max_{r\in Z_{\text{fit}}} \frac{q(r)}{\rho(r) + \varepsilon_{\text{bridge}}}, \qquad \kappa_{\text{bridge}} < \kappa_{\max}. \tag{24.11}$$

For the running example (2.1)–(2.10), the unpreconditioned target is

$$q_t(r) = \widehat{p}_{t-1}(x_{t-1})\varphi(x_t; \beta x_{t-1}, \sigma^2)\varphi(y_t; \sin x_t, \tau_y^2), \qquad r = (x_t, x_{t-1}). \tag{24.12}$$

A Gaussian bridge may keep a local mean and covariance for this two-dimensional target while missing the sinusoidal likelihood curvature. The residual after the bridge is therefore

$$m(T^{-1}(u)) = \frac{\widehat{p}_{t-1}(x_{t-1}(u))\varphi(x_t(u); \beta x_{t-1}(u), \sigma^2)\varphi(y_t; \sin x_t(u), \tau_y^2)}{\rho(x_t(u), x_{t-1}(u))}. \tag{24.13}$$

This is (24.7) with the concrete factors from the running example substituted. The bridge helps only if the remaining multiplier in (24.13) is flatter and less coupled than the original target.

Fit a square-root TT in u-space:

$$\sqrt{q_t^\sharp(u)} \approx \phi_t^\sharp(u),$$

and define

$$\widehat{\nu}_t^\sharp(u) = \phi_t^\sharp(u)^2 + \tau_t^\sharp \eta(u). \tag{24.14}$$

This is Zhao and Cui [4], Eq. (32). Pulling the approximation back to physical coordinates gives

$$\widehat{\pi}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))}\rho_t(r). \tag{24.15}$$

This is Zhao and Cui [4], Eq. (33). The normalizer is unchanged by the change of variables:

$$\int \widehat{\pi}_t(r)\,\mathrm{d}r = \int \widehat{\nu}_t^\sharp(u)\,\mathrm{d}u. \tag{24.16}$$

Why Preconditioning And KR Maps Belong Together

The preconditioner and the KR map solve complementary problems. The KR map can sample from a fitted density once the density has been represented well. The preconditioner tries to make the density easier to represent before the TT fit is performed. In equations, let the exact target be decomposed as

$$q(r) = \rho(r)m(r), \qquad m(r) = q(r)/\rho(r), \tag{24.17}$$

and choose T so that ρ is mapped to a product reference η:

$$\rho(r)\,\mathrm{d}r = \eta(u)\,\mathrm{d}u, \qquad u = T(r). \tag{24.18}$$

Substituting $r = T^{-1}(u)$ gives

$$q(r)\,\mathrm{d}r = m(T^{-1}(u))\eta(u)\,\mathrm{d}u. \tag{24.19}$$

The TT therefore sees the residual multiplier $m(T^{-1}(u))$, not the entire original density geometry. If ρ captures the main posterior ellipsoid, ridge, or concentration region, then

$$\operatorname{osc}\{\log m(T^{-1}(u)) : u \in \Omega_u\} < \operatorname{osc}\{\log q(r) : r \in \Omega_r\} \tag{24.20}$$

on the fitting domain, where $\operatorname{osc}(f) = \sup f - \inf f$. This is the concrete mathematical sense in which the residual can be flatter.

After the residual has been fitted, a KR map can be built in the easier u-coordinates:

$$F^\sharp(u) = \big(F_1^\sharp(u_1), F_2^\sharp(u_2 \mid u_1), \ldots, F_D^\sharp(u_D \mid u_{<D})\big). \tag{24.21}$$

If $\xi \sim \mathrm{Unif}([0,1]^D)$, then the two-step sampler is

$$u = (F^\sharp)^{-1}(\xi), \qquad r = T^{-1}(u). \tag{24.22}$$

The density calculation is the product of the two Jacobian identities:

$$\Big|\det \nabla F^\sharp(u)\Big|\,|\det \nabla T(r)| = \widehat{\nu}^\sharp(u)\,|\det \nabla T(r)| \approx q(r). \tag{24.23}$$

This is the teach-back version of the method: the preconditioner removes the dominant geometry, the squared TT fits the remaining density, and the KR map turns the fitted density into sequential conditional inversions.

Equivalently, the bridge answers a coarse geometric question and the KR map answers a sampling question:

$$\rho \approx q \quad\Longrightarrow\quad q^\sharp(u) \approx \eta(u) \times \text{moderate residual}, \qquad F^\sharp :\ q^\sharp \longrightarrow \mathrm{Unif}([0,1]^D). \tag{24.24}$$

The inverse proposal then has the readable form

$$\xi \sim \mathrm{Unif}([0,1]^D) \quad\mapsto\quad u = (F^\sharp)^{-1}(\xi) \quad\mapsto\quad r = T^{-1}(u). \tag{24.25}$$

So the bridge captures the coarse posterior geometry, the squared TT captures what the bridge missed, and the KR inverse converts that fitted residual density into a sequential one-dimensional proposal construction.

Implementation meaning. 1-Eq30–Eq33. Inputs are q_t, bridge ρ_t or $\widehat{\rho}_t$, map T_t, reference density η, and residual TT $\widehat{\nu}_t^\sharp$. Outputs are a physical-coordinate approximate density evaluator $\widehat{\pi}_t$, a normalizer, and later marginal/conditional map objects. Quantities to retain: map evaluators, inverse map evaluators, log-Jacobians or density ratios, bridge TT, residual TT, and normalizer. Diagnostics: bridge support ratio, map invertibility, normalizer equality between physical and reference coordinates, and residual rank.

Mathematical question. This paragraph answers: what is the simplest concrete preconditioner?

A simple bridge is Gaussian. Estimate a mean and covariance for r, set $\rho_t = \mathcal{N}(\mu_t, \Sigma_t)$, and use an affine Gaussian whitening map. If

$$\Sigma_t = L_t L_t^\top \tag{24.26}$$

is a Cholesky factorization with L_t lower triangular, then the lower linear KR whitening map is

$$T_t^\ell(r) = L_t^{-1}(r - \mu_t), \tag{24.27}$$

and $T_t^\ell(R) \sim N(0, I)$ when $R \sim N(\mu_t, \Sigma_t)$. The reference density η is then the standard Gaussian product density. The implementation stores μ_t, L_t, triangular solves for L_t^{-1}, and the log determinant

$$\log|\det \nabla T_t^\ell| = -\sum_j \log L_t[j, j]. \tag{24.28}$$

An upper triangular linear map is obtained by permuting or reversing the coordinate order before applying the same whitening idea.

Implementation meaning. 2. Inputs: particles or quadrature samples used to estimate μ_t, Σ_t. Outputs: triangular factor L_t, whitening map T_t^ℓ, inverse map $r = \mu_t + L_t u$, and log determinant. Quantities to retain: μ_t, L_t and any covariance regularization. Diagnostics: positive-definiteness of Σ_t, conditioning of L_t, and whether the Gaussian bridge assigns negligible density to target-relevant samples.

Mathematical question. This paragraph answers: how can the bridge retain nonlinear structure from the previous filter, transition, and likelihood?

A nonlinear bridge uses powers of the three factors in q_t:

$$\rho_t(r) = \widehat{p}_{t-1}(x_{t-1}, \alpha)^{\beta_\pi} f_\alpha(x_t \mid x_{t-1})^{\beta_f} g_\alpha(y_t \mid x_t)^{\beta_g}, \qquad \beta_\pi, \beta_f, \beta_g \in [0, 1]. \tag{24.29}$$

This is Zhao and Cui [4], Eq. (34). A squared-TT approximation of this bridge is

$$\widehat{\rho}_t(r) = \phi_{\rho,t}(r)^2 + \tau_{\rho,t}\lambda(r). \tag{24.30}$$

This is Zhao and Cui [4], Eq. (35). The same algebra as (24.15) applies with ρ_t replaced by $\widehat{\rho}_t$.

Implementation meaning. 3. Inputs: exponents $\beta_\pi, \beta_f, \beta_g$, previous filter evaluator, transition evaluator, likelihood evaluator, and bridge TT controls. Outputs: $\widehat{\rho}_t$ and its KR map. Quantities to retain: exponents, bridge TT cores, defensive mass, and bridge mass contractions. Diagnostics: bridge fit residual, bridge rank, support coverage, and whether the powered factors are finite at fitting points.

24.1 The Five Densities In The Preconditioned Step

Mathematical question. This subsection expands the compact Section 5 notation into the five densities an implementer must distinguish.

The preconditioned step is easy to lose because the paper moves between physical coordinates $r = (x_t, \theta, x_{t-1})$, reference coordinates $u = (u_t, u_\theta, u_{t-1})$, an exact bridge, an approximate bridge, and a residual density. The complete chain is as follows.

First, there is the exact target we ultimately want to approximate:

$$q_t(r) = \widehat{\pi}_{t-1}(x_{t-1}, \theta) f_\theta(x_t \mid x_{t-1}) g_\theta(y_t \mid x_t). \tag{24.31}$$

Second, there is an exact bridge $\rho_t(r)$. It is not the final answer. It is a simpler density chosen to resemble the main shape of q_t. In the tempered construction it is the powered product in (24.29). In the Gaussian construction it is $N(\mu_t, \Sigma_t)$.

Third, because the exact bridge may still be nonseparable, Algorithm 5(b.1) approximates the bridge itself by a squared TT:

$$\sqrt{\rho_t(r)} \approx \phi_{\rho,t}(r), \qquad \widehat{\rho}_t(r) = \phi_{\rho,t}(r)^2 + \tau_{\rho,t}\lambda(r). \tag{24.32}$$

This is the first line of Algorithm 5 in mathematical form. The implementation stores the bridge TT cores, the defensive mass $\tau_{\rho,t}$, the reference density λ, the bridge normalizer, and the mass contractions needed to construct conditional maps from $\widehat{\rho}_t$.

Fourth, build the preconditioning map from the bridge approximation. Let

$$T_t : \ r \mapsto u \tag{24.33}$$

be a KR-based map satisfying

$$(T_t)_{\#}\widehat{\rho}_t(u) \propto \eta(u). \tag{24.34}$$

Using $\widehat{\rho}_t$, not the unavailable exact bridge, the residual target to approximate in reference coordinates is

$$q_t^{\sharp}(u) = \frac{q_t(T_t^{-1}(u))}{\widehat{\rho}_t(T_t^{-1}(u))}\eta(u). \tag{24.35}$$

This formula answers the question "what remains after the bridge has removed the main shape?" If the bridge is good, the ratio $q_t/\widehat{\rho}_t$ is flatter than q_t, so the residual may need smaller TT ranks.

Fifth, approximate the square root of the residual:

$$\sqrt{q_t^{\sharp}(u)} \approx \phi_t^{\sharp}(u), \qquad \widehat{\nu}_t^{\sharp}(u) = \phi_t^{\sharp}(u)^2 + \tau_t^{\sharp}\eta(u). \tag{24.36}$$

Finally pull this residual approximation back to physical coordinates:

$$\widehat{\pi}_t(r) = \frac{\widehat{\nu}_t^{\sharp}(T_t(r))}{\eta(T_t(r))}\widehat{\rho}_t(r). \tag{24.37}$$

To check the algebra, substitute $u = T_t(r)$. Since $(T_t)_{\#}\widehat{\rho}_t \propto \eta$, the factor $\widehat{\rho}_t(r)/\eta(T_t(r))$ restores the physical-coordinate density scale. The normalizer is computed in reference space:

$$\int \widehat{\pi}_t(r)\,\mathrm{d}r = \int \widehat{\nu}_t^{\sharp}(u)\,\mathrm{d}u. \tag{24.38}$$

Algorithm 5 stores exactly the objects in this chain:

$$\widehat{\rho}_t, \quad T_t, \quad q_t^{\sharp} \text{ evaluator}, \quad \widehat{\nu}_t^{\sharp}, \quad \widehat{\pi}_t \text{ evaluator}, \quad \text{marginal and conditional maps.} \tag{24.39}$$

This is the preconditioning story in one line of thought: approximate the bridge, use the bridge to flatten the target, approximate the flattened residual, then pull the residual back.

Failure checks for the five-density chain. Verify all of the following:

$$q_t(r) \geq 0, \qquad \widehat{\rho}_t(r) > 0 \text{ on fitted target points}, \qquad q_t^{\sharp}(u) \geq 0, \tag{24.40}$$

$$0 < \int \widehat{\nu}_t^{\sharp}(u)\,\mathrm{d}u < \infty, \qquad \left| \int \widehat{\pi}_t(r)\,\mathrm{d}r - \int \widehat{\nu}_t^{\sharp}(u)\,\mathrm{d}u \right| \leq \varepsilon_{\mathrm{norm}}. \tag{24.41}$$

These are not cosmetic checks. They assess whether the density chain still defines the same scalar after moving between coordinate systems.

Mathematical question. Algorithm 5(b.1) answers: what approximation is built before the residual target is fitted?

The bridge approximation is $\widehat{\rho}_t$, already defined in (24.32). Its mathematical input-output relation is

$$(r \mapsto \rho_t(r), \lambda, \tau_{\rho,t}, \text{basis}, \text{ranks}) \mapsto (\widehat{\rho}_t, \text{bridge mass contractions}). \tag{24.42}$$

Failure checks: bridge fit residual, positivity of $\widehat{\rho}_t$, and nonzero bridge values on target fitting points.

Mathematical question. Algorithm 5(b.2) answers: how does the bridge approximation become a lower preconditioning map and a bridge marginal?

Build a KR map R_t from $\widehat{\rho}_t$ to a uniform variable:

$$(R_t)_{\#}\widehat{\rho}_t(\xi_t, \xi_\theta, \xi_{t-1}) \propto \mathrm{Unif}(\xi_t, \xi_\theta, \xi_{t-1}). \tag{24.43}$$

If D is a diagonal coordinate map satisfying

$$D_{\#}\eta(\xi_t, \xi_\theta, \xi_{t-1}) = \mathrm{Unif}(\xi_t, \xi_\theta, \xi_{t-1}), \tag{24.44}$$

then

$$T_t = D^{-1} \circ R_t \tag{24.45}$$

pushes the bridge approximation to the product reference η:

$$(T_t)_{\#}\widehat{\rho}_t(u_t, u_\theta, u_{t-1}) \propto \eta(u_t, u_\theta, u_{t-1}). \tag{24.46}$$

For the lower-triangular version, write

$$T_t^\ell(x_t, \theta, x_{t-1}) = \big(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t), T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta)\big). \tag{24.47}$$

The same bridge mass contractions also provide the bridge marginal

$$\widehat{\rho}_t(x_t, \theta) = \int \widehat{\rho}_t(x_t, \theta, x_{t-1})\, \mathrm{d}x_{t-1}. \tag{24.48}$$

Mathematical input and output:

$$\widehat{\rho}_t \mapsto (T_t^\ell, (T_t^\ell)^{-1}, \widehat{\rho}_t(x_t, \theta), T_{t,t-1}^\ell). \tag{24.49}$$

Failure checks: monotone conditional CDFs, inverse-map residuals, and agreement between the bridge marginal and direct bridge contraction.

Mathematical question. Algorithm 5(b.3) answers: after the bridge map is available, what residual density is fitted in reference coordinates?

With this nonlinear preconditioner, the residual target becomes

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\widehat{\rho}_t(T_t^{-1}(u))}\eta(u), \tag{24.50}$$

and the pulled-back unnormalized posterior approximation is

$$\widehat{\pi}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))}\widehat{\rho}_t(r). \tag{24.51}$$

Mathematical input and output:

$$(q_t, T_t^{-1}, \widehat{\rho}_t, \eta, \text{basis}, \text{ranks}) \mapsto (\widehat{\nu}_t^\sharp, S_t^\ell, \text{residual mass contractions}). \tag{24.52}$$

Failure checks: finite ratio $q_t/\widehat{\rho}_t$, residual rank growth, and normalizer $\int \widehat{\nu}_t^\sharp(u)\, \mathrm{d}u \in (0, \infty)$.

Mathematical question. Algorithm 5(c.1) answers: what residual marginal is needed before returning to physical coordinates?

Use the lower triangular decompositions

$$S_t^\ell(u_t, u_\theta, u_{t-1}) = \begin{bmatrix} S_{t,t}^\ell(u_t) \\ S_{t,\theta}^\ell(u_\theta \mid u_t) \\ S_{t,t-1}^\ell(u_{t-1} \mid u_t, u_\theta) \end{bmatrix}, \qquad T_t^\ell(x_t, \theta, x_{t-1}) = \begin{bmatrix} T_{t,t}^\ell(x_t) \\ T_{t,\theta}^\ell(\theta \mid x_t) \\ T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta) \end{bmatrix}. \tag{24.53}$$

Step (c.1) integrates the last block (u_{t-1}) of $\widehat{\nu}_t^\sharp$ to obtain

$$\widehat{\nu}_t^\sharp(u_t, u_\theta) = \int \widehat{\nu}_t^\sharp(u_t, u_\theta, u_{t-1})\, \mathrm{d}u_{t-1}. \tag{24.54}$$

This is an endpoint squared-TT marginalization in u-space, so it uses the same mass-matrix algebra as Proposition 2. It also creates the residual lower conditional map $S_{t,t-1}^\ell(u_{t-1} \mid u_t, u_\theta)$.

Mathematical question. Algorithm 5(c.2) answers: how does the residual marginal become the retained physical-coordinate filter?

Using

$$(u_t, u_\theta) = \big(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t)\big), \tag{24.55}$$

start from the pulled-back three-block density (24.51):

$$\widehat{\pi}_t(x_t, \theta, x_{t-1}) = \frac{\widehat{\nu}_t^\sharp(T_t^\ell(x_t, \theta, x_{t-1}))}{\eta(T_t^\ell(x_t, \theta, x_{t-1}))} \widehat{\rho}_t(x_t, \theta, x_{t-1}). \tag{24.56}$$

Now change variables only in the old-state conditional block. For fixed (x_t, θ), the lower bridge map sends

$$x_{t-1} \mapsto u_{t-1} = T_{t,t-1}^\ell(x_{t-1} \mid x_t, \theta). \tag{24.57}$$

Because T_t^ℓ was built from the bridge approximation, the bridge density decomposes locally. Write

$$u_t = T_{t,t}^\ell(x_t), \qquad u_\theta = T_{t,\theta}^\ell(\theta \mid x_t). \tag{24.58}$$

The conditional part of the triangular pushforward identity says that, for fixed (x_t, θ),

$$\widehat{\rho}_t(x_{t-1} \mid x_t, \theta)\, \mathrm{d}x_{t-1} = \eta(u_{t-1} \mid u_t, u_\theta)\, \mathrm{d}u_{t-1}. \tag{24.59}$$

Since the reference density factorizes in the transformed coordinates,

$$\eta(u_t, u_\theta, u_{t-1}) = \eta(u_{t-1} \mid u_t, u_\theta)\eta(u_t, u_\theta), \tag{24.60}$$

the conditional density in (24.59) can be written as

$$\eta(u_{t-1} \mid u_t, u_\theta) = \frac{\eta(T_t^\ell(x_t, \theta, x_{t-1}))}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))}. \tag{24.61}$$

Multiplying (24.59) by the bridge marginal $\widehat{\rho}_t(x_t, \theta)$ gives the local bridge decomposition

$$\widehat{\rho}_t(x_t, \theta, x_{t-1})\, \mathrm{d}x_{t-1} = \frac{\widehat{\rho}_t(x_t, \theta)}{\eta(T_{t,t}^\ell(x_t), T_{t,\theta}^\ell(\theta \mid x_t))} \eta(T_t^\ell(x_t, \theta, x_{t-1}))\, \mathrm{d}u_{t-1}, \tag{24.62}$$

where the Jacobian of the conditional map is included in the differential $\mathrm{d}u_{t-1}$. This is the missing middle step in plain words: the three-block bridge density equals its two-block marginal times the conditional bridge density, and the conditional bridge density becomes the conditional reference density after the triangular map. Substituting (24.62) into (24.56) and integrating over x_{t-1} gives

$$\begin{aligned}\widehat{\pi}_t(x_t, \theta \mid y_{1:t}) &= \int \widehat{\pi}_t(x_t, \theta, x_{t-1})\, \mathrm{d}x_{t-1} \\ &= \frac{\widehat{\rho}_t(x_t, \theta)}{\eta(T^\ell_{t,t}(x_t), T^\ell_{t,\theta}(\theta \mid x_t))} \int \widehat{\nu}^\sharp_t(u_t, u_\theta, u_{t-1})\, \mathrm{d}u_{t-1}.\end{aligned} \tag{24.63}$$

Using (24.54) with $(u_t, u_\theta) = (T^\ell_{t,t}(x_t), T^\ell_{t,\theta}(\theta \mid x_t))$, the retained marginal density for the next filtering step is

$$\widehat{\pi}_t(x_t, \theta \mid y_{1:t}) = \frac{\widehat{\nu}^\sharp_t\Big(T^\ell_{t,t}(x_t), T^\ell_{t,\theta}(\theta \mid x_t)\Big)}{\eta\Big(T^\ell_{t,t}(x_t), T^\ell_{t,\theta}(\theta \mid x_t)\Big)} \widehat{\rho}_t(x_t, \theta). \tag{24.64}$$

This is the source formula in Algorithm 5(c.2) written without relying on paper shorthand. The implementation stores the two maps, their marginal evaluators, and a retained evaluator for $\widehat{\pi}_t(x_t, \theta)$.

As a byproduct, the lower conditional map for path estimation is the last block of the composite map:

$$F^\ell_{t,t-1}(x_{t-1} \mid x_t, \theta) = S^\ell_{t,t-1}\Big(T^\ell_{t,t-1}(x_{t-1} \mid x_t, \theta) \,\Big|\, T^\ell_{t,t}(x_t), T^\ell_{t,\theta}(\theta \mid x_t)\Big). \tag{24.65}$$

This identity matters because preconditioning does not eliminate smoothing; it changes which conditional maps must be composed and stored.

Boxed Algorithm 5 dataflow.

Step	Evaluator signature	Quantities retained	Main diagnostic
b.1	$\rho_t(r) \mapsto \widehat{\rho}_t(r)$	bridge TT, $\tau_{\rho,t}, \lambda$	bridge residual/rank
b.2	$\widehat{\rho}_t \mapsto T^\ell_t, (T^\ell_t)^{-1}$	bridge maps, bridge marginal	CDF monotonicity
b.3	$(q_t, T_t^{-1}, \widehat{\rho}_t, \eta) \mapsto \widehat{\nu}^\sharp_t$	residual TT, S^ℓ_t	ratio finite
c.1	$\widehat{\nu}^\sharp_t(u_t, u_\theta, u_{t-1}) \mapsto \widehat{\nu}^\sharp_t(u_t, u_\theta)$	residual marginal, $S^\ell_{t,t-1}$	mass contraction
c.2	$(\widehat{\nu}^\sharp_t, T^\ell_t, \widehat{\rho}_t) \mapsto \widehat{\pi}_t(x_t, \theta)$	retained filter evaluator	normalizer invariance

25 Fixed-Branch Likelihood Specialization

The equations above explain Zhao and Cui [4]'s adaptive sequential algorithm: tensor-train approximation, squared-TT nonnegativity, conditional KR maps, particle correction, smoothing, and preconditioning. The fixed-branch likelihood specialization freezes approximation choices so that an analytical derivative can differentiate the same scalar computed by the forward pass. Zhao and Cui [4]'s adaptive rank, pivot, fitting, preconditioning, and branch choices should not be read as globally differentiable.

Reconciliation: Paper Objects Versus Fixed-Branch Objects

The paper's main sequential-learning density carries a learned static parameter as a random coordinate:

$$r^{\mathrm{ZC}}_t = (x_t, \theta, x_{t-1}). \tag{25.1}$$

The fixed-branch likelihood specialization conditions on an external parameter value β and differentiates with respect to that external argument:

$$r_t^{\mathrm{FB}} = (x_t, x_{t-1}; \beta). \tag{25.2}$$

The semicolon is deliberate: β is not integrated out by the TT density. It is the input at which the approximate likelihood is evaluated.

The exact mapping of objects is:

$$\begin{gathered}
\widehat{\pi}_{t-1}(x_{t-1}, \theta) \mapsto \widehat{p}_{t-1}(x_{t-1}; \beta), \\
f(x_t \mid x_{t-1}, \theta) \mapsto f_\beta(x_t \mid x_{t-1}), \qquad g(y_t \mid x_t, \theta) \mapsto g_\beta(y_t \mid x_t), \\
\phi_t(x_t, \theta, x_{t-1}) \mapsto \phi_t(x_t, x_{t-1}; \beta), \\
\tau_t \lambda(x_t)\lambda(\theta)\lambda(x_{t-1}) \mapsto \tau_t \lambda_t(x_t, x_{t-1}), \\
\widehat{Z}_t = \int \widehat{q}_t(x_t, \theta, x_{t-1}) \mapsto \widehat{Z}_t(\beta) = \int \widehat{q}_t(x_t, x_{t-1}; \beta), \\
\text{adaptive ranks/pivots/maps} \mapsto \text{fixed ranks/points/maps/sweeps.}
\end{gathered} \tag{25.3}$$

The first line keeps the retained-filter role but drops the integrated θ coordinate. The second line keeps the model evaluators, evaluated at the external β. The third and fourth lines keep the squared-TT and defensive-density forms in a lower-dimensional state space. The fifth line keeps the evidence-increment role. The last line is the essential change: adaptive choices are replaced by frozen branch choices before differentiation. Thus the fixed-branch section does not differentiate the adaptive Zhao and Cui [4] algorithm. It differentiates a lower-dimensional scalar constructed by applying the same squared-TT filtering algebra after freezing all discrete approximation choices. If one wants a fixed-branch derivative while also learning a static parameter as a random coordinate, one can append that coordinate to the state; the external derivative variable must still be distinguished from the integrated coordinate.

26 Fixed-Branch Objects And Array Shapes

A reader implementing a minimal method must store more than an equation. At time t, a squared-TT filter object contains:

$$\mathcal{B}_t = \{\Omega_{t,k}, \Psi_{t,k}, \psi_{t,k,\ell}, C_{t,k}, R_{t,k}, B_{t,k}, \lambda_{t,k}, \tau_t, c_t, \text{mass contractions}, \text{map data}\}.$$

Here $\Omega_{t,k}$ is the coordinate domain, $\Psi_{t,k}$ maps reference coordinates to physical coordinates, $\psi_{t,k,\ell}$ are basis functions, $C_{t,k}$ are TT cores, $R_{t,k}$ are ranks, $B_{t,k}$ are mass matrices, $\lambda_{t,k}$ are reference densities, τ_t is the defensive mass, and c_t is any log-scaling shift used during fitting.

For the adaptive Zhao and Cui [4] algorithm these objects may change with the data, with samples, and with approximation decisions. For a differentiable same-scalar target, the realized branch must be frozen before differentiation.

26.1 What A Core Object Looks Like

For coordinate k, store

$$C_k \in \mathbb{R}^{R_{k-1} \times p_k \times R_k}.$$

Given a scalar coordinate value r_k, first compute the basis vector

$$b_k(r_k) = \big(\psi_{k,0}(r_k), \ldots, \psi_{k,p_k-1}(r_k)\big)^\top.$$

Then form the matrix

$$H_k(r_k)[a,b] = \sum_{\ell=0}^{p_k-1} C_k[a,\ell,b]\, b_k(r_k)[\ell]. \tag{26.1}$$

Evaluation of the TT is the loop

$$v_0 = 1, \qquad v_k = v_{k-1} H_k(r_k), \quad k = 1,\dots,D, \qquad \widehat{h}(r) = v_D. \tag{26.2}$$

The square-density object additionally stores mass contractions

$$M_{>k} = \int H_{k+1}(r_{k+1}) \cdots H_D(r_D) H_D(r_D)^\top \cdots H_{k+1}(r_{k+1})^\top \,\mathrm{d}r_{k+1:D}. \tag{26.3}$$

These matrices are enough to evaluate marginals from the left. Analogous left-side mass contractions evaluate reverse-order marginals.

26.2 What Must Be Saved For The Next Time Step

After time t, the next step needs to evaluate

$$\widehat{p}_t(x_t;\beta) \quad \text{and} \quad \partial_{\beta_i}\widehat{p}_t(x_t;\beta)$$

at future fitting points. It is not enough to store a cloud of samples or only $\widehat{Z}_t$. A fixed-branch derivative implementation stores:

$$\left\{ C_{t,k}, \dot{C}^{(i)}_{t,k}, M_{t,>k}, \dot{M}^{(i)}_{t,>k}, \widehat{Z}_t, \dot{\widehat{Z}}^{(i)}_t, \Psi_t, \lambda_t, \tau_t \right\}. \tag{26.4}$$

The derivative of the normalized marginal has the quotient form

$$\partial_{\beta_i}\widehat{p}_t(x_t;\beta) = \frac{\partial_{\beta_i} a_t(x_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(x_t;\beta)\partial_{\beta_i}\widehat{Z}_t(\beta)}{\widehat{Z}_t(\beta)^2}, \tag{26.5}$$

where

$$a_t(x_t;\beta) = \int \widehat{q}_t(x_t, x_{t-1};\beta)\,\mathrm{d}x_{t-1} \tag{26.6}$$

is the unnormalized retained numerator. Equation (26.5) is the carried-filter derivative needed by the next-step product-rule derivative.

Lemma 1 (Carried filter object feeds the next target)**.** *Fix the branch at time t. Suppose the stored object contains*

$$\mathcal{F}_t = \{a_t, \dot{a}^{(i)}_t, \widehat{Z}_t, \dot{\widehat{Z}}^{(i)}_t, \Psi_t, \lambda_t, \tau_t\}.$$

Define $\widehat{p}_t$ and $\dot{\widehat{p}}^{(i)}_t$ by (26.5). Then the next transformed target and its derivative at any next-step fitting point are computable from $\mathcal{F}_t$, the model density evaluators, and their derivatives.

Proof. At time $t+1$, the fixed-branch target has the form

$$\widetilde{q}_{t+1}(z;\beta) = \widehat{p}_t(x_t(z);\beta) f_\beta(x_{t+1}(z) \mid x_t(z)) g_\beta(y_{t+1} \mid x_{t+1}(z)) J_{t+1}(z).$$

The stored object evaluates $\widehat{p}_t$. The model supplies f_β and g_β, and the branch supplies J_{t+1}. Differentiating gives the corresponding product-rule expression at time $t+1$. The first term uses $\dot{\widehat{p}}^{(i)}_t$, which is the quotient derivative in (26.5). No old samples or external source objects are needed. □

27 Consolidated Fixed Least-Squares Formulation

The implementable formulation used for the fixed-branch derivative is a fixed least-squares formulation. It is not adaptive TT-cross. Adaptive TT-cross chooses pivots, interpolation sets, and ranks from the target. The construction here declares the branch first, then uses deterministic ridge least squares to fit the TT cores. In symbols,

$$\text{fixed least-squares derivative formulation} = \text{fixed points+fixed ranks+fixed sweep order+ridge least squares.} \tag{27.1}$$

This narrower construction is sufficient to define a scalar and a derivative of that same scalar.

27.1 Inputs, Outputs, And Stored Fields

For one time step, the inputs are

$$\mathcal{I}_t = \{y_t, \beta, \widehat{p}_{t-1}, \dot{\widehat{p}}_{t-1}, f_\beta, \dot{f}_\beta, g_\beta, \dot{g}_\beta, \Psi_t, Z_{\text{fit}}^{(t)}, W^{(t)}, b_k, R_k, S, \rho, c_t, \tau_t\}. \tag{27.2}$$

Here S is the declared number of sweeps and ρ is the ridge scale. The forward outputs are

$$\mathcal{O}_t = \{C_{t,k}, M_{t,>k}, M_{t,<k}, \widehat{Z}_t, a_t, \widehat{p}_t\}_{k=1}^{D}, \tag{27.3}$$

and the derivative outputs are

$$\dot{\mathcal{O}}_t = \{\dot{C}_{t,k}, \dot{M}_{t,>k}, \dot{M}_{t,<k}, \dot{\widehat{Z}}_t, \dot{a}_t, \dot{\widehat{p}}_t\}_{k=1}^{D}. \tag{27.4}$$

The frozen branch fields are

$$\mathcal{B}_{t,\text{frozen}} = \{\Psi_t, Z_{\text{fit}}^{(t)}, W^{(t)}, b_k, R_k, S, \rho, c_t, \tau_t, \lambda_t, \text{sweep direction sequence}\}. \tag{27.5}$$

The differentiable fields are recomputed when β changes:

$$\mathcal{B}_{t,\text{diff}}(\beta) = \{y_j, A_k, N_k, d_k, g_k, C_{t,k}, M_{t,>k}, \widehat{Z}_t, a_t, \widehat{p}_t\}. \tag{27.6}$$

The following shape table fixes the array contract used by the equations below:

object	shape
C_k	$R_{k-1} \times p_k \times R_k$
$L_{j,k}$	$1 \times R_{k-1}$
$R_{j,k}$	$R_k \times 1$
A_k	$n_{\text{fit}} \times (R_{k-1} p_k R_k)$
$g_k = \text{vec}(C_k),\ d_k$	$R_{k-1} p_k R_k$
N_k	$(R_{k-1} p_k R_k) \times (R_{k-1} p_k R_k)$

(27.7)

The vectorization convention is

$$\iota_k(a, \ell, b) = (a-1)p_k R_k + \ell R_k + b, \qquad g_k[\iota_k(a, \ell, b)] = C_k[a, \ell, b], \tag{27.8}$$

where $a = 1, \ldots, R_{k-1}$, $\ell = 0, \ldots, p_k - 1$, and $b = 1, \ldots, R_k$. With the one-dimensional basis mass matrix

$$B_k[\ell, m] = \int b_{k,\ell}(z) b_{k,m}(z)\, \mathrm{d}z, \tag{27.9}$$

the left-to-right squared-mass contraction is

$$G_k[b, b'] = \sum_{a,a',\ell,m} G_{k-1}[a, a']\, C_k[a, \ell, b]\, B_k[\ell, m]\, C_k[a', m, b'], \qquad G_0[1, 1] = 1. \tag{27.10}$$

For a pure squared TT, the final square mass is $G_D[1, 1]$; the defensive component adds the declared τ_t-mass separately.

27.2 Initialization And Sweep Order

Initialize every core with the declared ranks:

$$C_k^{(0)} \in \mathbb{R}^{R_{k-1}\times p_k\times R_k}, \qquad R_0 = R_D = 1. \tag{27.11}$$

The initialization rule must be deterministic, for example

$$C_k^{(0)}[a,\ell,b] = \sigma_{\text{init}} \sin((a+1)(\ell+1)(b+1)), \tag{27.12}$$

followed by endpoint normalization. The exact deterministic rule is less important than saving it in the branch, because changing initialization changes the fixed fitting map.

One sweep consists of a left-to-right pass followed by a right-to-left pass:

$$1, 2, \dots, D, D-1, \dots, 1. \tag{27.13}$$

After S sweeps the fitted cores are

$$(C_1, \dots, C_D) = \mathcal{S}_{B_t}(\beta), \tag{27.14}$$

where $\mathcal{S}_{B_t}$ denotes the finite composition of fixed core-update maps.

27.3 Forward Core Update

At fitting point $z^{(j)}$, define environments for core k:

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \qquad R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}). \tag{27.15}$$

For endpoint ranks this means $L_{j,k} \in \mathbb{R}^{1\times R_{k-1}}$ and $R_{j,k} \in \mathbb{R}^{R_k\times 1}$. The design row for coefficient (a,ℓ,b) is

$$A_k[j,(a,\ell,b)] = L_{j,k}[1,a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b,1]. \tag{27.16}$$

The local ridge system is

$$N_k g_k = d_k, \qquad N_k = A_k^\top W A_k + \rho I, \qquad d_k = A_k^\top W y. \tag{27.17}$$

After solving (27.17), reshape g_k into C_k:

$$C_k[a,\ell,b] = g_k[(a,\ell,b)]. \tag{27.18}$$

27.4 Derivative Core Update

The derivative environments are obtained by differentiating the matrix products:

$$\dot L_{j,k} = \sum_{r=1}^{k-1} H_1 \cdots H_{r-1} \dot H_r H_{r+1} \cdots H_{k-1}, \tag{27.19}$$

$$\dot R_{j,k} = \sum_{r=k+1}^{D} H_{k+1} \cdots H_{r-1} \dot H_r H_{r+1} \cdots H_D. \tag{27.20}$$

The derivative design row is

$$\begin{aligned} \dot A_k[j,(a,\ell,b)] = {} & \dot L_{j,k}[1,a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b,1] \\ & + L_{j,k}[1,a]\, b_k(z_k^{(j)})[\ell]\, \dot R_{j,k}[b,1], \end{aligned} \tag{27.21}$$

because the basis and fitting points are frozen. If a later branch allows parameter-dependent domain maps or basis functions, additional terms are required; they are not part of this fixed branch.

The derivative normal equations are

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \qquad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \tag{27.22}$$

Differentiating $N_k g_k = d_k$ gives

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \tag{27.23}$$

After solving (27.23), reshape

$$\dot{C}_k[a, \ell, b] = \dot{g}_k[(a, \ell, b)]. \tag{27.24}$$

27.5 Alternating-Sweep Recomputations

The symbols $L_{j,k}$, $R_{j,k}$, A_k, N_k, and d_k are not static arrays during an alternating sweep. They are recomputed from the current cores at the moment a core is updated. Let

$$H_{j,k} = H_k(z_k^{(j)}; C_k), \qquad \dot{H}_{j,k} = H_k(z_k^{(j)}; \dot{C}_k), \tag{27.25}$$

where the basis values and fitting point are fixed and only the coefficients change. For a current core collection C^{cur}, define prefix and suffix products by

$$L_{j,1}^{\mathrm{cur}} = 1, \qquad L_{j,k}^{\mathrm{cur}} = L_{j,k-1}^{\mathrm{cur}} H_{j,k-1}^{\mathrm{cur}}, \qquad k = 2, \dots, D, \tag{27.26}$$

and

$$R_{j,D}^{\mathrm{cur}} = 1, \qquad R_{j,k}^{\mathrm{cur}} = H_{j,k+1}^{\mathrm{cur}} R_{j,k+1}^{\mathrm{cur}}, \qquad k = D-1, \dots, 1. \tag{27.27}$$

The dotted prefix and suffix products are computed by the same recursions with one product-rule term added:

$$\dot{L}_{j,1}^{\mathrm{cur}} = 0, \qquad \dot{L}_{j,k}^{\mathrm{cur}} = \dot{L}_{j,k-1}^{\mathrm{cur}} H_{j,k-1}^{\mathrm{cur}} + L_{j,k-1}^{\mathrm{cur}} \dot{H}_{j,k-1}^{\mathrm{cur}}, \tag{27.28}$$

and

$$\dot{R}_{j,D}^{\mathrm{cur}} = 0, \qquad \dot{R}_{j,k}^{\mathrm{cur}} = \dot{H}_{j,k+1}^{\mathrm{cur}} R_{j,k+1}^{\mathrm{cur}} + H_{j,k+1}^{\mathrm{cur}} \dot{R}_{j,k+1}^{\mathrm{cur}}. \tag{27.29}$$

At a core update k, the design matrix and normal equations are built from the current environments:

$$A_k^{\mathrm{cur}} = A_k(L_{:,k}^{\mathrm{cur}}, R_{:,k}^{\mathrm{cur}}), \qquad N_k^{\mathrm{cur}} = (A_k^{\mathrm{cur}})^\top W A_k^{\mathrm{cur}} + \rho I, \qquad d_k^{\mathrm{cur}} = (A_k^{\mathrm{cur}})^\top W y. \tag{27.30}$$

Solve $N_k^{\mathrm{cur}} g_k^{\mathrm{new}} = d_k^{\mathrm{cur}}$ and replace only the current core:

$$C_k^{\mathrm{cur}} \leftarrow C_k^{\mathrm{new}}, \qquad C_\ell^{\mathrm{cur}} \text{ unchanged for } \ell \neq k. \tag{27.31}$$

The derivative objects for the same update are then

$$\begin{aligned} \dot{A}_k^{\mathrm{cur}} &= \dot{A}_k(\dot{L}_{:,k}^{\mathrm{cur}}, L_{:,k}^{\mathrm{cur}}, \dot{R}_{:,k}^{\mathrm{cur}}, R_{:,k}^{\mathrm{cur}}), \\ \dot{N}_k^{\mathrm{cur}} &= (\dot{A}_k^{\mathrm{cur}})^\top W A_k^{\mathrm{cur}} + (A_k^{\mathrm{cur}})^\top W \dot{A}_k^{\mathrm{cur}}, \\ \dot{d}_k^{\mathrm{cur}} &= (\dot{A}_k^{\mathrm{cur}})^\top W y + (A_k^{\mathrm{cur}})^\top W \dot{y}. \end{aligned} \tag{27.32}$$

The derivative core is updated by

$$N_k^{\mathrm{cur}} \dot{g}_k^{\mathrm{new}} = \dot{d}_k^{\mathrm{cur}} - \dot{N}_k^{\mathrm{cur}} g_k^{\mathrm{new}}, \qquad \dot{C}_k^{\mathrm{cur}} \leftarrow \dot{C}_k^{\mathrm{new}}. \tag{27.33}$$

The next environment update depends on sweep direction. In a left-to-right pass, after replacing C_k, the next core is $k+1$, so

$$L^{\mathrm{cur}}_{j,k+1} = L^{\mathrm{cur}}_{j,k} H^{\mathrm{new}}_{j,k}, \qquad \dot{L}^{\mathrm{cur}}_{j,k+1} = \dot{L}^{\mathrm{cur}}_{j,k} H^{\mathrm{new}}_{j,k} + L^{\mathrm{cur}}_{j,k} \dot{H}^{\mathrm{new}}_{j,k}. \tag{27.34}$$

The suffix $R^{\mathrm{cur}}_{j,k+1}$ is already valid because it involves only cores $k+2, \ldots, D$. In a right-to-left pass, after replacing C_k, the next core is $k-1$, so

$$R^{\mathrm{cur}}_{j,k-1} = H^{\mathrm{new}}_{j,k} R^{\mathrm{cur}}_{j,k}, \qquad \dot{R}^{\mathrm{cur}}_{j,k-1} = \dot{H}^{\mathrm{new}}_{j,k} R^{\mathrm{cur}}_{j,k} + H^{\mathrm{new}}_{j,k} \dot{R}^{\mathrm{cur}}_{j,k}. \tag{27.35}$$

The prefix $L^{\mathrm{cur}}_{j,k-1}$ is already valid because it involves only cores $1, \ldots, k-2$. If an implementation caches all prefixes and suffixes for all k, then after updating C_k every cached prefix with index $\ell > k$ and every cached suffix with index $\ell < k$ is stale:

$$C_k \text{ updated} \quad \Longrightarrow \quad \{L_{j,\ell} : \ell > k\} \text{ and } \{R_{j,\ell} : \ell < k\} \text{ must be recomputed before use.} \tag{27.36}$$

This is the concrete bookkeeping behind the statement that A_k, N_k, and d_k are recomputed during each alternating sweep.

27.6 Stopping Rule And Failure Exits

After each full sweep compute

$$\varepsilon_s = \frac{\|A(C^{(s)}) - y\|_W}{\max\{\|y\|_W, \epsilon_{\mathrm{abs}}\}}, \qquad \Delta_s = \frac{\|C^{(s)} - C^{(s-1)}\|}{\max\{\|C^{(s-1)}\|, \epsilon_{\mathrm{abs}}\}}. \tag{27.37}$$

The sweep stops successfully if

$$\varepsilon_s \le \epsilon_{\mathrm{fit}} \quad \text{and} \quad \Delta_s \le \epsilon_{\mathrm{sweep}}. \tag{27.38}$$

If this does not happen by S sweeps, the declared branch reports

$$\mathcal{F}_t = \text{sweep failure}. \tag{27.39}$$

Failure condition	Mathematical test	Report field
Domain miss	$\delta_{\mathrm{out}} > \delta_{\max}$	domain failure
Rank ladder fail	no R satisfies residual/conditioning/floor tests	rank failure
Ill-conditioned solve	$\mathrm{cond}(N_k) > \kappa_{\max}$	solve failure
Sweep stagnation	$\varepsilon_s > \epsilon_{\mathrm{fit}}$ or $\Delta_s > \epsilon_{\mathrm{sweep}}$ after S sweeps	sweep failure
Nonpositive target for smooth derivative	$\min_j \widetilde{q}_{t,j} \le 0$ without declared floor	target failure
Mass failure	$\widehat{Z}_t \le 0$ or not finite	normalizer failure
Finite-difference branch mismatch	$\mathcal{M}_{\mathrm{frozen}}(\beta - h) \ne \mathcal{M}_{\mathrm{frozen}}(\beta + h)$	branch mismatch

This table is part of the mathematical method. A failed branch is not a successful derivative with a warning attached; it is a declared failure of the fixed scalar.

28 One Concrete Branch, Fully Instantiated

This section gives a single fixed-branch construction. It is intentionally narrower than Zhao and Cui [4]'s adaptive code. Its purpose is to remove ambiguity. A coder can implement this construction first, then replace pieces with adaptive or higher performance choices later.

28.1 Coordinate Order And Domain

For the fixed-parameter likelihood case, use the two-block coordinate order

$$r_t = (x_t, x_{t-1}) \in \mathbb{R}^2$$

for a scalar state. For vector states, concatenate coordinates as

$$r_t = (x_{t,1}, \ldots, x_{t,m_x}, x_{t-1,1}, \ldots, x_{t-1,m_x}).$$

Choose a fixed interval for each coordinate,

$$r_{t,k} \in [\ell_{t,k}, u_{t,k}],$$

and map from reference coordinate $z_k \in [-1, 1]$ by

$$r_{t,k} = a_{t,k} + h_{t,k} z_k, \qquad a_{t,k} = \frac{u_{t,k} + \ell_{t,k}}{2}, \qquad h_{t,k} = \frac{u_{t,k} - \ell_{t,k}}{2}. \tag{28.1}$$

The Jacobian is constant:

$$J_t = \prod_{k=1}^{D} h_{t,k}. \tag{28.2}$$

The branch stores $\ell_{t,k}, u_{t,k}, a_{t,k}, h_{t,k}$. In the same-scalar derivative these are constants and $\dot{J}_t = 0$.

Domain-Selection Policy

The fixed domain must be chosen before differentiation and then saved. For coordinate k, let a pilot location and scale be

$$\mu_{t,k}^{\text{pilot}}, \qquad s_{t,k}^{\text{pilot}} > 0. \tag{28.3}$$

They may come from prior moments, transition moments, a pilot particle cloud, or a Gaussian bridge. A deterministic interval is

$$[\ell_{t,k}, u_{t,k}] = [\mu_{t,k}^{\text{pilot}} - \gamma s_{t,k}^{\text{pilot}}, \mu_{t,k}^{\text{pilot}} + \gamma s_{t,k}^{\text{pilot}}], \tag{28.4}$$

with expansion factor $\gamma > 0$. For Gaussian-like pilot tails,

$$\gamma = 4 \quad \Rightarrow \quad \Pr(|Z| > \gamma) \approx 6.3 \times 10^{-5}. \tag{28.5}$$

If the model supplies hard bounds, intersect with those bounds:

$$[\ell_{t,k}, u_{t,k}] \leftarrow [\ell_{t,k}, u_{t,k}] \cap [\ell_k^{\text{model}}, u_k^{\text{model}}]. \tag{28.6}$$

The out-of-domain diagnostic at fitting or proposal points is

$$\delta_{\text{out}} = \frac{1}{N} \sum_{j=1}^{N} \mathbf{1}\{r_{t,k}^{(j)} \notin [\ell_{t,k}, u_{t,k}] \text{ for some } k\}. \tag{28.7}$$

The branch is accepted only if

$$\delta_{\mathrm{out}} \le \delta_{\max}. \tag{28.8}$$

If this fails, enlarge the domain and rebuild the branch. Do not enlarge the domain during a same-branch derivative check:

$$[\ell_{t,k}, u_{t,k}](\beta_0 + h) = [\ell_{t,k}, u_{t,k}](\beta_0) = [\ell_{t,k}, u_{t,k}](\beta_0 - h). \tag{28.9}$$

28.2 Basis, Points, Weights, Ranks

Use normalized Legendre basis with the same degree p in every coordinate:

$$b_k(z_k) = \left(\sqrt{\frac{1}{2}} P_0(z_k), \sqrt{\frac{3}{2}} P_1(z_k), \ldots, \sqrt{\frac{2p-1}{2}} P_{p-1}(z_k) \right)^{\top}. \tag{28.10}$$

Use fixed Halton-type fitting points. Let $p_k^\star$ be the k-th prime. If

$$j = \sum_{m=0}^{M} d_m (p_k^\star)^m, \qquad d_m \in \{0, \ldots, p_k^\star - 1\},$$

define

$$\mathrm{radinv}_{p_k^\star}(j) = \sum_{m=0}^{M} d_m (p_k^\star)^{-(m+1)}, \qquad z_k^{(j)} = 2\,\mathrm{radinv}_{p_k^\star}(j) - 1. \tag{28.11}$$

Use $j = 1, \ldots, N_{\mathrm{fit}}$, weights

$$W = \frac{1}{N_{\mathrm{fit}}} I, \tag{28.12}$$

and fixed ranks

$$R_0 = R_D = 1, \qquad R_1, \ldots, R_{D-1} \text{ declared before fitting.} \tag{28.13}$$

No pivot, point, rank, or domain choice is allowed to change during a same-scalar derivative check.

28.3 Deterministic Rank Ladder Before Branch Freezing

The rank vector is selected by a small deterministic ladder before it becomes part of the fixed branch. Let

$$\mathcal{R}_{\mathrm{ladder}} = \{(1, \ldots, 1), (2, \ldots, 2), (4, \ldots, 4), (6, \ldots, 6), (8, \ldots, 8)\}. \tag{28.14}$$

For every candidate $R \in \mathcal{R}_{\mathrm{ladder}}$, keep the same domain, basis, fitting points, weights, shift, defensive density, and preconditioner. Fit the square-root TT and compute

$$\varepsilon_{\mathrm{rel}}(R) = \frac{\|A_R g_R - y\|_W}{\max\{\|y\|_W, \epsilon_{\mathrm{abs}}\}}, \qquad \kappa_R = \mathrm{cond}(A_R^\top W A_R + \epsilon_{\mathrm{ridge}} I). \tag{28.15}$$

The defensive-mass fraction is

$$\chi_R = \frac{\tau_t}{e^{-c_t} \int \phi_{t,R}(z)^2 \, \mathrm{d}z + \tau_t}. \tag{28.16}$$

The candidate passes if

$$\varepsilon_{\mathrm{rel}}(R) \le \epsilon_{\mathrm{fit}}, \qquad \kappa_R \le \kappa_{\max}, \qquad \chi_R \le \chi_{\max}. \tag{28.17}$$

The declared branch uses the first passing rank:

$$R_\star = \min\{R \in \mathcal{R}_{\text{ladder}} : R \text{ satisfies (28.17)}\}. \tag{28.18}$$

If no candidate passes, the correct outcome is branch failure,

$$\mathcal{F}_t = \text{rank-ladder failure}, \tag{28.19}$$

not an unrecorded adaptive rank change. Once $R_\star$ is selected, all finite-difference evaluations and derivative equations keep $R_\star$ fixed.

28.4 Defensive Reference, Mass, And Shift

Use the uniform reference on $[-1,1]^D$:

$$\lambda(z) = 2^{-D}. \tag{28.20}$$

Choose a defensive floor $\epsilon_\tau > 0$ and set

$$\tau_t = 2^D \epsilon_\tau. \tag{28.21}$$

Then $\tau_t \lambda(z) = \epsilon_\tau$. Choose the stabilizing shift at the base parameter β_0:

$$c_t = -\max_{1 \le j \le N_{\text{fit}}} \log \widetilde{q}_t(z^{(j)}; \beta_0). \tag{28.22}$$

The fitted square-root target is

$$y_j(\beta) = \exp(c_t/2)\sqrt{\widetilde{q}_t(z^{(j)}; \beta)}. \tag{28.23}$$

For same-branch finite differences, c_t is reused for $\beta_0 + h$ and $\beta_0 - h$. Recomputing c_t changes the scalar.

28.5 Boxed Convention: Shifted Fit, Unshifted Density

The concrete branch uses this convention everywhere downstream.

$$\boxed{\begin{aligned}
&\text{Fitted TT:} && \phi_t(z;\beta) \approx e^{c_t/2}\sqrt{\widetilde{q}_t(z;\beta)},\\
&\text{Approximate joint density:} && \widehat{q}_t(z;\beta) = e^{-c_t}\phi_t(z;\beta)^2 + \tau_t \lambda_t(z),\\
&\text{Retained numerator:} && a_t(z_t;\beta) = \int \left[e^{-c_t}\phi_t(z_t, z_{t-1};\beta)^2 + \tau_t \lambda_t(z_t, z_{t-1})\right] \mathrm{d}z_{t-1},\\
&\text{Normalizer:} && \widehat{Z}_t(\beta) = \int \widehat{q}_t(z;\beta)\,\mathrm{d}z = e^{-c_t}\int \phi_t(z;\beta)^2\,\mathrm{d}z + \tau_t.
\end{aligned}} \tag{28.24}$$

Every carried filter, proposition, derivative, and finite-difference check uses (28.24). If an implementation instead absorbs $e^{-c_t/2}$ into the TT cores, then it must remove the explicit e^{-c_t} factors everywhere. Mixing the two conventions changes the scalar.

Stabilization Defaults

A minimal deterministic stabilization policy is

$$\lambda_t(z) = \prod_{k=1}^{D} \frac{1}{2}\mathbf{1}\{-1 \le z_k \le 1\}, \tag{28.25}$$

so λ_t has units of inverse reference-volume and integrates to one. Choose a density floor $\epsilon_{\text{floor}} > 0$ in transformed target units and set

$$\tau_t = 2^D \epsilon_{\text{floor}}. \tag{28.26}$$

Then the defensive density contribution is

$$\tau_t \lambda_t(z) = \epsilon_{\text{floor}} \quad \text{on } [-1,1]^D. \tag{28.27}$$

The fitting target uses the floored value

$$\widetilde{q}_{t,j}^{\text{floor}} = \max\{\widetilde{q}_t(z^{(j)};\beta), \epsilon_{\text{floor}}\}. \tag{28.28}$$

The shift is computed once at the base parameter:

$$c_t = -\max_j \log \widetilde{q}_{t,j}^{\text{floor}}(\beta_0). \tag{28.29}$$

The ridge parameter is scaled to the design matrix:

$$\rho_t = \epsilon_{\text{ridge}} \frac{\operatorname{trace}(A_k^\top W A_k)}{\dim(g_k)} \quad \text{for each core update,} \tag{28.30}$$

or fixed to a declared positive scalar if this adaptive scaling is not used. The stabilization diagnostic vector is

$$\mathcal{D}_{\text{stab}} = \left\{ \frac{\tau_t}{\widehat{Z}_t}, \frac{\#\{j : \widetilde{q}_{t,j} \le \epsilon_{\text{floor}}\}}{N_{\text{fit}}}, \kappa(A_k^\top W A_k + \rho_t I), |c_t| \right\}. \tag{28.31}$$

A branch fails stabilization if the defensive mass dominates,

$$\frac{\tau_t}{\widehat{Z}_t} > \delta_\tau, \tag{28.32}$$

or if too many fitting targets are floored,

$$\frac{\#\{j : \widetilde{q}_{t,j} \le \epsilon_{\text{floor}}\}}{N_{\text{fit}}} > \delta_{\text{floor}}. \tag{28.33}$$

These defaults are reproducibility defaults, not production validation.

constant	first-pass value	role
$\lambda_t(z)$	$2^{-D}\mathbf{1}\{z \in [-1,1]^D\}$	defensive reference density in reference coordinates
τ_t	$2^D \epsilon_{\text{floor}}$	defensive mass giving floor $\tau_t \lambda_t = \epsilon_{\text{floor}}$
c_t	$-\max_j \log \tilde{q}_{t,j}^{\text{floor}}(\beta_0)$	base-parameter square-root scaling shift
ϵ_{floor}	10^{-12}	minimum transformed target value used before square roots
ϵ_{ridge}	10^{-10}	relative ridge scale for local normal equations
ρ_t	$\epsilon_{\text{ridge}} \operatorname{trace}(A^\top W A)/\dim(g)$	ridge used in each local normal equation
δ_τ	10^{-2}	maximum defensive-mass fraction
δ_{floor}	10^{-2}	maximum fraction of floored fitting values
γ	4	pilot-scale multiplier for reference domains
N_{fit}	$\max\{20\, D\, p\, R_\star^2, 200\}$	number of fixed fitting points
$S_{\max}$	4	bidirectional sweep count for the deterministic fit
ϵ_{fit}	10^{-4}	relative weighted square-root fit-residual tolerance
ϵ_{sweep}	10^{-5}	relative core-change tolerance across sweeps
ϵ_Z	10^{-8}	relative normalizer cross-check tolerance
ϵ_{norm}	10^{-8}	retained-filter normalization tolerance
$\epsilon_Q, \epsilon_{\dot{Q}}$	$10^{-6}, 10^{-5}$	compressed retained-object validation tolerances
ϵ_{abs}	10^{-14}	absolute scale preventing division by zero in residuals
$\varepsilon_{\text{root}}$	10^{-10}	conditional CDF inversion tolerance
N_{root}	80	maximum bisection steps for inverse conditionals
$\chi_{\max}$	10^{-2}	defensive-fraction veto in the rank ladder
$\kappa_{\max}$	10^{12}	normal-equation conditioning veto
H_{FD}	$\{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$	finite-difference step schedule

(28.34)

The numerical values in (28.34) are reproducibility defaults for a first mathematical implementation. They are not empirical evidence that the method is production-ready.

28.6 Initialization And Sweep Order

Initialize every core to zero except the constant channel:

$$C_k[1,0,1] = 1, \qquad k = 2, \ldots, D,$$

and

$$C_1[1,0,1] = \bar{y}, \qquad \bar{y} = \frac{1}{N_{\text{fit}}} \sum_{j=1}^{N_{\text{fit}}} y_j.$$

If a rank is larger than one, all non-first rank channels start at zero. This plain initialization gives a constant initial approximation and avoids a random initialization branch.

One bidirectional sweep means update cores $1, 2, \ldots, D$, then $D, D-1, \ldots, 1$. Fix the number of bidirectional sweeps S. The core update is the ridge solve (15.7). The branch stores the sweep order, S, ρ, and the final cores.

28.7 Forward-Step Object Flow

The fixed-branch forward step is a dependency chain among mathematical objects, not executable code. The dependency flow is

$$\begin{aligned}
\text{reference points} &\longrightarrow z^{(j)} \text{ from (28.11)},\\
\text{physical points} &\longrightarrow r^{(j)} = \Psi_t(z^{(j)}),\\
\text{target values} &\longrightarrow \widetilde{q}_{t,j} = \widehat{p}_{t-1}(x_{t-1}^{(j)};\beta) f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}) g_\beta(y_t \mid x_t^{(j)}) J_t,\\
\text{square-root fit values} &\longrightarrow y_j = \exp(c_t/2)\sqrt{\widetilde{q}_{t,j}},\\
\text{initial cores} &\longrightarrow \text{constant-channel initialization},\\
\text{fixed sweep objects} &\longrightarrow L_{j,k}, R_{j,k}, A_{j,k}, N_k, d_k, C_k,\\
\text{mass objects} &\longrightarrow M_{>k}, R_t, \widehat{Z}_t,\\
\text{carried objects} &\longrightarrow a_t, \widehat{p}_t.
\end{aligned} \tag{28.35}$$

The fixed sweep environments are

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}),$$

$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}),$$

with $A_{j,k}$ defined by (15.5) and the core update defined by (15.7). After fitting, square-mass contractions use (18.5)–(18.6):

$$R_t = \int \phi_t(z)^2 \, \mathrm{d}z, \qquad \widehat{Z}_t = e^{-c_t} R_t + \tau_t.$$

The carried numerator is the old-state marginal

$$a_t(z_t;\beta) = \int \left[e^{-c_t} \phi_t(z_t, z_{t-1};\beta)^2 + \tau_t \lambda_t(z_t, z_{t-1}) \right] \mathrm{d}z_{t-1},$$

and the carried filter is $a_t/\widehat{Z}_t$.

28.8 Conditional-Map Inversion Protocol

For a conditional coordinate r_k, compute

$$F_k(s \mid r_{<k}) = \int_{\ell_k}^{s} \widehat{p}(r_k' \mid r_{<k}) \, \mathrm{d}r_k'.$$

To sample from the conditional, given $u \in (0,1)$, solve

$$F_k(s \mid r_{<k}) - u = 0, \qquad s \in [\ell_k, u_k].$$

Use bisection as the baseline because it only needs monotonicity. Fix a tolerance $\varepsilon_{\text{root}}$ and maximum iterations N_{root}. The branch stores both. The bisection update is:

$$m = \frac{a+b}{2}, \qquad \text{if } F_k(m \mid r_{<k}) < u \text{ set } a = m, \text{ else set } b = m.$$

Stop when $b - a \le \varepsilon_{\text{root}}$ or the iteration count reaches N_{root}. Raise a failure diagnostic if the final residual $|F_k(m \mid r_{<k}) - u|$ exceeds the declared tolerance.

29 Fixed-Branch TT Filtering Recursion

The fixed-branch filtering variant chooses all discrete approximation objects before differentiating. It is narrower than the full Zhao and Cui [4] adaptive code, but it is the variant for which an analytical derivative can be stated without changing the scalar under the derivative.

29.1 End-To-End Fixed-Branch Squared-TT Filter

The following boxed algorithm gathers the complete mathematical recurrence in one place. It is written as equations and stored objects rather than executable program text.

Inputs. Model densities $p_\beta(x_0)$, $f_\beta(x_t \mid x_{t-1})$, $g_\beta(y_t \mid x_t)$; observations $y_{1:T}$; reference domains $\mathcal{Z}_t$; domain maps Ψ_t; optional preconditioners T_t; basis families $b_{t,k}$; candidate ranks $\mathcal{R}$; fitting points $Z_{\mathrm{fit},t}$; weights W_t; defensive densities λ_t; constants $c_t, \tau_t, \epsilon_{\mathrm{floor}}, \epsilon_{\mathrm{ridge}}$.

Initialization. Set $\widehat{p}_0$ to the declared initial density in retained coordinates and set $\widehat{\ell}_0 = 0$. Store the initial domain map and, if differentiating, $\dot{\widehat{p}}_0$.

For $t = 1, \ldots, T$**.** Form the transformed target $\widetilde{q}_t(z;\beta) = \widehat{p}_{t-1}(z_{t-1};\beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t) J_t(z)$. Evaluate $y_j = e^{c_t/2}\sqrt{\max\{\widetilde{q}_t(z^{(j)}), \epsilon_{\mathrm{floor}}\}}$. Fit a square-root TT ϕ_t by the declared fixed-rank sweep. Build $\widehat{q}_t = e^{-c_t}\phi_t^2 + \tau_t\lambda_t$. Contract mass matrices to compute $\widehat{Z}_t = e^{-c_t}\int \phi_t^2 \,\mathrm{d}z + \tau_t$. Set $a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1})\,\mathrm{d}z_{t-1}$ and $\widehat{p}_t(z_t) = a_t(z_t)/\widehat{Z}_t$. Update $\widehat{\ell}_t = \widehat{\ell}_{t-1} + \log \widehat{Z}_t$. If requested, build conditional KR maps from marginal ratios. If differentiating, run the derivative pass through target values, sweep environments, core solves, mass contractions, normalizer, and retained filter.

Outputs. $\widehat{\ell}_T(\beta; B)$, retained filters $\widehat{p}_t$, normalizers $\widehat{Z}_t$, square-root cores $C_{t,k}$, mass matrices, optional KR maps, and dotted counterparts for all objects used by the next target.

Invariants. $\widehat{q}_t \geq 0$; $0 < \widehat{Z}_t < \infty$; $\int \widehat{p}_t = 1$; fixed branch B is identical for all finite difference evaluations; all densities are evaluated in their declared coordinate systems.

Failure exits. Reject the branch if target values underflow beyond the declared floor, the least-squares solve is ill-conditioned after ridge regularization, the chosen rank fails the rank ladder, defensive mass dominates the fitted mass, KR conditional CDFs are not monotone enough for inversion, a preconditioner is singular, or finite differences do not use the same branch.

(29.1)

29.2 Boxed Algorithm: Fixed-Branch Filter And Derivative Together

For implementation work, the forward scalar and its derivative should be read as one coupled mathematical recurrence. The branch is fixed first:

$$B = \{\Psi_t, T_t, b_{t,k}, Z_{\mathrm{fit},t}, W_t, R_{t,k}, S_t, \rho_{t,k}, c_t, \tau_t, \lambda_t, \text{sweep order}\}_{t,k}. \tag{29.2}$$

For a parameter component β_i, the coupled algorithm is

Inputs. β, model density evaluators $(p_\beta, f_\beta, g_\beta)$, derivative evaluators $(\partial_i p_\beta, \partial_i f_\beta, \partial_i g_\beta)$, observations $y_{1:T}$, and fixed branch B.

Initial retained object. Set P_0 to the initial retained density representation and set $\dot{P}_0 = \partial_i P_0$. If the initial law is independent of β_i, then $\dot{P}_0 = 0$.

For $t = 1, \dots, T$, target values. At each fitting point $z^{(j)}$, evaluate

$$\widetilde{q}_{t,j} = P_{t-1}(z_{t-1}^{(j)}) f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}) g_\beta(y_t \mid x_t^{(j)}) J_t(z^{(j)}),$$

and

$$\dot{\widetilde{q}}_{t,j} = \dot{P}_{t-1} f_\beta g_\beta J_t + P_{t-1}(\partial_i f_\beta) g_\beta J_t + P_{t-1} f_\beta (\partial_i g_\beta) J_t + P_{t-1} f_\beta g_\beta \dot{J}_t.$$

In the fixed-domain specialization, $\dot{J}_t = 0$.

Square-root values. With the frozen shift and floor,

$$y_j = e^{c_t/2} \sqrt{\max(\widetilde{q}_{t,j}, \epsilon_{\text{floor}})}, \qquad \dot{y}_j = \frac{e^{c_t/2} \dot{\widetilde{q}}_{t,j}}{2\sqrt{\widetilde{q}_{t,j}}}$$

when the value is not floored; if the branch declares a nonsmooth floor hit, the derivative check is failed rather than silently differentiated through max.

Fixed sweeps. For each declared core update, recompute $L_{j,k}, R_{j,k}, A_k, N_k, d_k$, solve $N_k g_k = d_k$, reshape $g_k \mapsto C_{t,k}$, then recompute the dotted environments and solve

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k.$$

The recomputation order is the one in (27.25)–(27.36); in particular, cached prefixes and suffixes on the wrong side of an updated core are stale until recomputed.

Mass and normalizer. Contract the fitted cores to obtain $R_t = \int \phi_t^2 \, \mathrm{d}z$ and

$$\dot{R}_t = \int 2\phi_t \dot{\phi}_t \, \mathrm{d}z, \qquad \widehat{Z}_t = e^{-c_t} R_t + \tau_t, \qquad \dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t.$$

Retained filter. Compute retained numerator matrices or compressed retained evaluators $(Q_t, \dot{Q}_t)$, normalize

$$P_t = Q_t / \widehat{Z}_t, \qquad \dot{P}_t = \dot{Q}_t / \widehat{Z}_t - Q_t \dot{\widehat{Z}}_t / \widehat{Z}_t^2,$$

and save the evaluator pair needed by time $t+1$.

Scalar update.

$$\widehat{\ell}_t = \widehat{\ell}_{t-1} + \log \widehat{Z}_t, \qquad \dot{\widehat{\ell}}_t = \dot{\widehat{\ell}}_{t-1} + \dot{\widehat{Z}}_t / \widehat{Z}_t.$$

Outputs. $\widehat{\ell}_T(\beta; B)$, $\partial_i \widehat{\ell}_T(\beta; B)$, retained evaluator pairs $(P_t, \dot{P}_t)$, fitted cores $(C_{t,k}, \dot{C}_{t,k})$, normalizers, diagnostics, and failure status.

Invariants and exits. The branch B is unchanged; every retained filter integrates to one; every normalizer is finite and positive; every derivative solve uses the same forward normal matrix. Exit with failure if floor hits are nonsmooth, ranks do not pass the declared ladder, conditioned solves exceed $\kappa_{\max}$, defensive mass dominates, retained compression breaks normalization, or finite differences do not use the same branch.

(29.3)

There is one important notational separation. In the Zhao and Cui [4] learning algorithm, the unknown parameter α may be carried as a random variable inside the posterior density. In an HMC-style likelihood calculation, the parameter is instead an external argument that is varied by the sampler. To avoid using the same symbol for both roles, this section writes the external differentiated parameter as

$$\beta \in \mathbb{R}^{m_\beta}.$$

The fixed-branch likelihood variant conditions on a trial β and filters only the latent state. Its unnormalized update is

$$q_t(x_t, x_{t-1}; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t).$$

If one also wants to carry a learned static parameter as a random coordinate, that coordinate can be appended to the state; the derivative below is still with respect to the external β, not with respect to an integration variable.

Let $z \in [-1, 1]^D$ be reference coordinates and

$$r = \Psi_t(z) = (x_t, x_{t-1}).$$

Let $J_t(z) = |\det \nabla \Psi_t(z)|$. The transformed target is

$$\widetilde{q}_t(z; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t) J_t(z). \tag{29.4}$$

The branch fixes points $z^{(j)}$, weights w_j, basis functions, ranks, and a core-construction algorithm. The fitted square-root TT is

$$\phi_t(z; \beta) = G_{t,1}(z_1; \beta) \cdots G_{t,D}(z_D; \beta). \tag{29.5}$$

The fixed-branch approximate density is

$$\widehat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z), \qquad \int \lambda_t(z)\, \mathrm{d}z = 1. \tag{29.6}$$

Its normalizer is

$$\widehat{Z}_t(\beta) = \int \widehat{q}_t(z; \beta)\, \mathrm{d}z = e^{-c_t} \int \phi_t(z; \beta)^2\, \mathrm{d}z + \tau_t. \tag{29.7}$$

This is exactly the boxed convention (28.24). The next filter object is the normalized marginal

$$\widehat{p}_t(x_t; \beta) = \frac{\int \widehat{q}_t(z_t, z_{t-1}; \beta)\, \mathrm{d}z_{t-1}}{\widehat{Z}_t(\beta)} \left|\det \nabla z_t / \nabla x_t\right|. \tag{29.8}$$

The Jacobian factor appears if the filter is evaluated in physical coordinates. If the whole recursion is implemented in reference coordinates, it is absorbed into the stored domain maps.

Proposition 1 (Fixed-branch recursion is a normalized approximate filter)**.** *Fix a time horizon T, model densities f_β, g_β, initial density $p_\beta(x_0)$, and fixed branch objects for each time step. Suppose every $\widehat{q}_t(\cdot; \beta)$ defined by (29.6) is nonnegative, integrable, and has $0 < \widehat{Z}_t(\beta) < \infty$. Define $\widehat{p}_t$ by (29.8), starting from the fixed initial approximation. Then each $\widehat{p}_t$ is a normalized density on the retained state variable x_t. Moreover, the recursion is exactly the Bayes filtering recursion applied to the declared approximate joint density $\widehat{q}_t$, not to the exact joint density.*

Proof. Nonnegativity follows from $\phi_t^2 \geq 0$, $\tau_t > 0$, and $\lambda_t \geq 0$. Since $\widehat{Z}_t = \int \widehat{q}_t$ is positive and finite, the normalized joint density

$$\widehat{p}_t^{\,\mathrm{joint}}(z) = \widehat{q}_t(z)/\widehat{Z}_t$$

integrates to one. The retained filter is the marginal of this normalized joint density:

$$\int \widehat{p}_t(z_t)\,\mathrm{d}z_t = \int \left[\int \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1})\,\mathrm{d}z_{t-1}\right] \mathrm{d}z_t = \int \widehat{p}_t^{\,\mathrm{joint}}(z)\,\mathrm{d}z = 1.$$

The change-of-variables factor in physical coordinates is the standard Jacobian factor for density transformation, so normalization is preserved. The construction uses the exact Bayes filtering algebra with q_t replaced by the declared approximation $\widehat{q}_t$. No step asserts that $\widehat{q}_t = q_t$, so the result is a normalized approximate filtering recursion rather than an exact-posterior theorem. □

30 Integrated Fixed-Branch Gradient Expansion

The preceding sections preserved the source-order Zhao and Cui [4] annotation and the fixed-branch likelihood setup. The next part expands the derivative path in small mathematical steps. It deliberately starts again from scalar normalizers before returning to tensor trains, because the key claim is that the gradient differentiates the same approximate scalar computed by the fixed branch.

The merge has one important reading convention. The Zhao and Cui [4] annotation uses θ for a random static parameter inside the sequential posterior when the paper studies joint state and parameter learning. The fixed-branch gradient part uses β for an external parameter value at which evaluates an approximate likelihood. These are different roles:

$$\theta \text{ can be an integration coordinate in Zhao and Cui [4]}, \qquad \beta \text{ is the differentiated argument of } \widehat{\ell}_T. \tag{30.1}$$

The fixed-branch scalar therefore does not claim to differentiate the adaptive Zhao and Cui [4] algorithm with respect to every internal coordinate. It freezes the realized numerical branch and differentiates the scalar produced by that branch as β varies:

$$B = \{\text{domains, points, basis, ranks, shifts, sweeps, tolerances}\}, \qquad \beta \mapsto \widehat{\ell}_T(\beta; B). \tag{30.2}$$

This distinction is what makes the next derivation useful rather than merely formal. Without it, changing β could also change ranks, pivots, maps, or fitting points; then two nearby likelihood evaluations would not be values of one mathematical function. With it, the fitted core values still change with β, but they change through the same stored fitting equations:

$$C_{t,k}(\beta; B) = \text{output of the fixed core-fitting rule at parameter } \beta. \tag{30.3}$$

Thus the derivative below is neither a derivative of frozen numbers nor a derivative of an adaptive search procedure. It is the derivative of the declared fixed-branch approximation. This is the identity a finite-difference diagnostic and a later implementation must obey.

31 Fixed-Branch Gradient Motivation

The filtering algorithm produces, at each time t, an approximate normalizing constant

$$\widehat{Z}_t(\beta) > 0. \tag{31.1}$$

The parameter β is the model parameter at which we evaluate the approximate likelihood. The total approximate log likelihood is

$$\widehat{\ell}_T(\beta) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta). \tag{31.2}$$

Formally, $\widehat{Z}_t(\beta)$ plays the role of a partition-function-like normalizer: it is an integral of a nonnegative weight. Here $\widehat{Z}_t(\beta)$ is also an integral of a nonnegative weight:

$$\widehat{Z}_t(\beta) = \int \widehat{q}_t(z;\beta)\,\mathrm{d}z. \tag{31.3}$$

The variable z is a numerical coordinate, usually a transformed coordinate on a fixed box. The function $\widehat{q}_t$ is not the exact Bayesian density. It is the squared tensor-train approximation used by the numerical filter.

The panel cares about the gradient because parameter inference algorithms need to know how the approximate log likelihood changes when β changes:

$$\nabla_\beta \widehat{\ell}_T(\beta). \tag{31.4}$$

The delicate point is not the derivative of log. The delicate point is that $\widehat{Z}_t$ is produced by a complicated approximation algorithm. If the approximation algorithm changes its grid, ranks, pivots, or scaling shifts when β changes, then the two likelihood evaluations may not be evaluations of the same mathematical formula. A classical analytical derivative can only differentiate one declared formula.

That is why this note uses a fixed branch.

Fixed branch. Before differentiating, freeze the computational choices: coordinate domain, fitting points, basis functions, tensor-train ranks, sweep order, ridge parameter, defensive density, stabilizing shifts, and the rule for constructing every fitted core. Do not freeze the fitted core values themselves. Those core values are outputs of the fixed fitting equations, so they remain functions of β. Then differentiate the scalar defined by the saved choices and by the core values obtained from the same saved fitting equations.

The goal is therefore:

$$\text{differentiate the scalar actually computed by the saved forward pass.} \tag{31.5}$$

The goal is not:

$$\text{differentiate every possible adaptive version of the algorithm.} \tag{31.6}$$

32 Warmup 1: A Normalizing Constant

Start with the simplest possible object. Let

$$Z(\beta) = \int_\Omega q(z;\beta)\,\mathrm{d}z, \qquad q(z;\beta) \geq 0, \qquad 0 < Z(\beta) < \infty. \tag{32.1}$$

The normalized density is

$$p(z;\beta) = \frac{q(z;\beta)}{Z(\beta)}. \tag{32.2}$$

The log normalizing constant is

$$L(\beta) = \log Z(\beta). \tag{32.3}$$

Assume that $q(z;\beta)$ is differentiable in β, and that we may move the derivative inside the integral:

$$\frac{\partial}{\partial\beta}\int_\Omega q(z;\beta)\,\mathrm{d}z = \int_\Omega \frac{\partial q(z;\beta)}{\partial\beta}\,\mathrm{d}z. \tag{32.4}$$

A sufficient condition is that, near the parameter value of interest, the absolute derivative is bounded by an integrable function:

$$\left|\frac{\partial q(z;\beta)}{\partial\beta}\right| \le m(z), \qquad \int_\Omega m(z)\,\mathrm{d}z < \infty. \tag{32.5}$$

Then the derivative of the log normalizer is

$$\begin{aligned}\frac{\partial L(\beta)}{\partial\beta} &= \frac{\partial}{\partial\beta}\log Z(\beta)\\ &= \frac{1}{Z(\beta)}\frac{\partial Z(\beta)}{\partial\beta}\\ &= \frac{1}{Z(\beta)}\int_\Omega \frac{\partial q(z;\beta)}{\partial\beta}\,\mathrm{d}z.\end{aligned} \tag{32.6}$$

This is the first idea in the whole gradient. Everything later is only a way to compute the numerator and denominator in (32.6) for the tensor-train approximation.

For the sequential filter, the same formula is used at every time:

$$\frac{\partial\widehat{\ell}_T(\beta)}{\partial\beta} = \sum_{t=1}^{T}\frac{1}{\widehat{Z}_t(\beta)}\frac{\partial\widehat{Z}_t(\beta)}{\partial\beta}. \tag{32.7}$$

33 Warmup 2: A Squared Approximation

The fixed-branch squared tensor-train approximation has the form

$$\widehat{q}(z;\beta) = e^{-c}\phi(z;\beta)^2 + \tau\lambda(z). \tag{33.1}$$

Here:

c	is a frozen stabilizing shift,
$\tau > 0$	is a frozen defensive mass,
$\lambda(z) \ge 0$	is a frozen reference density with $\int\lambda = 1$,
$\phi(z;\beta)$	is the fitted square-root function.

(33.2)

Because c, τ, λ are frozen, the only object in (33.1) that changes with β is ϕ. Differentiating (33.1) gives

$$\begin{aligned}\dot{\widehat{q}}(z;\beta) &= \frac{\partial}{\partial\beta}\left[e^{-c}\phi(z;\beta)^2 + \tau\lambda(z)\right]\\ &= e^{-c}\frac{\partial}{\partial\beta}\phi(z;\beta)^2 + \frac{\partial}{\partial\beta}\tau\lambda(z)\\ &= 2e^{-c}\phi(z;\beta)\dot{\phi}(z;\beta).\end{aligned} \tag{33.3}$$

The dot means $\partial/\partial\beta$.

The corresponding normalizer is

$$\begin{aligned}\widehat{Z}(\beta) &= \int \widehat{q}(z;\beta)\,\mathrm{d}z \\ &= e^{-c}\int \phi(z;\beta)^2\,\mathrm{d}z + \tau\int \lambda(z)\,\mathrm{d}z \\ &= e^{-c}\int \phi(z;\beta)^2\,\mathrm{d}z + \tau.\end{aligned} \tag{33.4}$$

Using (33.3), its derivative is

$$\begin{aligned}\dot{\widehat{Z}}(\beta) &= \int \dot{\widehat{q}}(z;\beta)\,\mathrm{d}z \\ &= 2e^{-c}\int \phi(z;\beta)\dot{\phi}(z;\beta)\,\mathrm{d}z.\end{aligned} \tag{33.5}$$

Substitution into (32.6) gives

$$\frac{\partial}{\partial\beta}\log\widehat{Z}(\beta) = \frac{2e^{-c}\int \phi(z;\beta)\dot{\phi}(z;\beta)\,\mathrm{d}z}{e^{-c}\int \phi(z;\beta)^2\,\mathrm{d}z + \tau}. \tag{33.6}$$

If c, τ, or λ changed with β, then (33.3) would have extra terms:

$$\dot{\widehat{q}} = -\dot{c}\,e^{-c}\phi^2 + 2e^{-c}\phi\dot{\phi} + \dot{\tau}\,\lambda + \tau\dot{\lambda}. \tag{33.7}$$

The fixed-branch derivative does not use (33.7). It uses the fixed-branch formula (33.3). This is not a minor technicality. It is the mathematical line between differentiating one saved scalar and differentiating a changing adaptive procedure.

34 Warmup 3: Two Coordinates, Rank One

Before a full tensor train, consider a two-coordinate rank-one function:

$$\phi(z_1, z_2;\beta) = h_1(z_1;\beta)h_2(z_2;\beta). \tag{34.1}$$

The square integral is

$$\begin{aligned}R(\beta) &= \iint \phi(z_1,z_2;\beta)^2\,\mathrm{d}z_1\,\mathrm{d}z_2 \\ &= \iint h_1(z_1;\beta)^2 h_2(z_2;\beta)^2\,\mathrm{d}z_1\,\mathrm{d}z_2 \\ &= \left[\int h_1(z_1;\beta)^2\,\mathrm{d}z_1\right]\left[\int h_2(z_2;\beta)^2\,\mathrm{d}z_2\right].\end{aligned} \tag{34.2}$$

Define

$$R_1(\beta) = \int h_1(z_1;\beta)^2\,\mathrm{d}z_1, \qquad R_2(\beta) = \int h_2(z_2;\beta)^2\,\mathrm{d}z_2. \tag{34.3}$$

Then

$$R(\beta) = R_1(\beta)R_2(\beta). \tag{34.4}$$

Differentiate by the product rule:

$$\dot{R}(\beta) = \dot{R}_1(\beta)R_2(\beta) + R_1(\beta)\dot{R}_2(\beta). \tag{34.5}$$

Each one-dimensional derivative is

$$\dot{R}_1(\beta) = 2\int h_1(z_1;\beta)\dot{h}_1(z_1;\beta)\,\mathrm{d}z_1, \tag{34.6}$$

$$\dot{R}_2(\beta) = 2\int h_2(z_2;\beta)\dot{h}_2(z_2;\beta)\,\mathrm{d}z_2. \tag{34.7}$$

Combining (34.5)–(34.7),

$$\begin{aligned}\dot{R}(\beta) = 2\left[\int h_1\dot{h}_1\,\mathrm{d}z_1\right]\left[\int h_2^2\,\mathrm{d}z_2\right] \\ + 2\left[\int h_1^2\,\mathrm{d}z_1\right]\left[\int h_2\dot{h}_2\,\mathrm{d}z_2\right].\end{aligned} \tag{34.8}$$

This is the simplest version of the tensor-train mass derivative: when one factor changes, keep the other factors fixed and sum the contributions.

35 Warmup 4: Two Coordinates, Rank R

Now let

$$\phi(z_1,z_2;\beta) = \sum_{r=1}^{R} h_{1r}(z_1;\beta)h_{2r}(z_2;\beta). \tag{35.1}$$

The square integral is

$$\begin{aligned}R(\beta) &= \iint\left[\sum_{r=1}^{R} h_{1r}(z_1)h_{2r}(z_2)\right]\left[\sum_{s=1}^{R} h_{1s}(z_1)h_{2s}(z_2)\right]\mathrm{d}z_1\,\mathrm{d}z_2 \\ &= \sum_{r=1}^{R}\sum_{s=1}^{R}\left[\int h_{1r}(z_1)h_{1s}(z_1)\,\mathrm{d}z_1\right]\left[\int h_{2r}(z_2)h_{2s}(z_2)\,\mathrm{d}z_2\right].\end{aligned} \tag{35.2}$$

Define the two mass matrices

$$M_1[r,s] = \int h_{1r}(z_1)h_{1s}(z_1)\,\mathrm{d}z_1, \tag{35.3}$$

$$M_2[r,s] = \int h_{2r}(z_2)h_{2s}(z_2)\,\mathrm{d}z_2. \tag{35.4}$$

Then

$$R(\beta) = \sum_{r,s} M_1[r,s]M_2[r,s]. \tag{35.5}$$

This is the Frobenius inner product

$$R(\beta) = \langle M_1(\beta), M_2(\beta)\rangle_F. \tag{35.6}$$

Differentiate:

$$\dot{R} = \langle\dot{M}_1, M_2\rangle_F + \langle M_1, \dot{M}_2\rangle_F. \tag{35.7}$$

The derivative of a mass matrix entry is

$$\dot{M}_1[r,s] = \int \dot{h}_{1r}(z_1)h_{1s}(z_1) + h_{1r}(z_1)\dot{h}_{1s}(z_1)\,\mathrm{d}z_1, \tag{35.8}$$

and similarly for M_2. The full tensor-train mass recursion is only this calculation repeated along a chain of coordinates.

36 Warmup 5: A Fixed Linear Solve

The fitted tensor-train cores are obtained from finite-dimensional least-squares updates. One such update can be written

$$N(\beta)g(\beta) = d(\beta). \tag{36.1}$$

Here g is the vector of unknown core coefficients in the current update, N is the regularized normal matrix, and d is the right-hand side. Differentiate both sides:

$$\frac{\partial}{\partial\beta}\{N(\beta)g(\beta)\} = \frac{\partial d(\beta)}{\partial\beta}. \tag{36.2}$$

Use the product rule:

$$\dot{N}(\beta)g(\beta) + N(\beta)\dot{g}(\beta) = \dot{d}(\beta). \tag{36.3}$$

Move the first term to the right:

$$N(\beta)\dot{g}(\beta) = \dot{d}(\beta) - \dot{N}(\beta)g(\beta). \tag{36.4}$$

If $N(\beta)$ is nonsingular, then

$$\dot{g}(\beta) = N(\beta)^{-1}\left[\dot{d}(\beta) - \dot{N}(\beta)g(\beta)\right]. \tag{36.5}$$

In computation, one should not form the inverse. One solves the linear system (36.4) using the same numerical linear algebra used for the forward solve.

The ridge term is important because the normal matrix has the form

$$N = A^\top W A + \rho I, \qquad \rho > 0. \tag{36.6}$$

If W is positive semidefinite, then

$$v^\top N v = v^\top A^\top W A v + \rho v^\top v \geq \rho\|v\|^2 \tag{36.7}$$

for all v. Hence N is positive definite and nonsingular. Without nonsingularity, the derivative of the solve may not be well defined.

37 The Fixed-Branch Forward Pass

At time t, the fixed branch constructs a nonnegative approximation

$$\widehat{q}_t(z;\beta) = e^{-c_t}\phi_t(z;\beta)^2 + \tau_t\lambda_t(z) \tag{37.1}$$

and its normalizer

$$\widehat{Z}_t(\beta) = \int \widehat{q}_t(z;\beta)\,\mathrm{d}z. \tag{37.2}$$

The branch is the collection of structural choices that define the scalar as a function of β. These choices include the domain, fitting points, basis, ranks, sweep order, shift, defensive term, and ridge parameter. The branch does not mean that the fitted numerical core values are held constant when β changes. Instead,

$$C_{t,k}(\beta) = \text{the core obtained by the saved fitting rule at parameter } \beta. \tag{37.3}$$

The derivative pass computes $\dot{C}_{t,k}(\beta)$. This distinction matters: freezing the core values would usually make the normalizer nearly constant and would not assess the derivative of the fitted approximation.

Forward object	What is stored	Why it is needed later
Domain map Ψ_t	Coordinate intervals and Jacobian	Evaluates the same transformed target at every derivative point
Fitting points $z^{(j)}$	Deterministic point array and weights	Defines the least-squares equations
Basis b_k	Basis family, degree, mass matrices	Evaluates cores and mass contractions
Ranks R_k	Fixed rank vector	Fixes core dimensions
Sweep order	Core update order and number of sweeps	Defines a finite composition of solve maps
Shift c_t	One frozen scalar	Defines the same scaled square-root target
Defensive term	τ_t, λ_t	Ensures nonnegative density floor
Cores $C_{t,k}(\beta)$	Parameter-dependent fitted coefficient arrays	Evaluate ϕ_t and $\widehat{q}_t$
Mass contractions	$M_{t,>k}(\beta)$, $M_{t,<k}(\beta)$	Compute integrals and marginals
Normalizer	$\widehat{Z}_t$	Adds $\log \widehat{Z}_t$ to the scalar
Carried filter	Numerator a_t and normalized $\widehat{p}_t$	Feeds the next time step

The transformed target fitted at time t is

$$\widetilde{q}_t(z;\beta) = \widehat{p}_{t-1}(x_{t-1}(z);\beta) f_\beta(x_t(z) \mid x_{t-1}(z)) g_\beta(y_t \mid x_t(z)) J_t(z). \tag{37.4}$$

The fitted square-root values are

$$y_j(\beta) = e^{c_t/2} \sqrt{\widetilde{q}_t(z^{(j)};\beta)}. \tag{37.5}$$

The tensor train is

$$\phi_t(z;\beta) = H_{t,1}(z_1;\beta) H_{t,2}(z_2;\beta) \cdots H_{t,D}(z_D;\beta), \tag{37.6}$$

where $H_{t,k}(z_k) \in \mathbb{R}^{R_{k-1} \times R_k}$ and $R_0 = R_D = 1$.

The retained numerator for the next filter is

$$a_t(z_t;\beta) = \int \widehat{q}_t(z_t, z_{t-1};\beta)\, \mathrm{d}z_{t-1}. \tag{37.7}$$

The retained normalized filter is

$$\widehat{p}_t(z_t;\beta) = \frac{a_t(z_t;\beta)}{\widehat{Z}_t(\beta)}. \tag{37.8}$$

If the filter is reported in physical coordinates, multiply by the appropriate change-of-variables factor. The derivative logic is unchanged if the domain map is frozen.

38 Gradient Teaching Layer: What Is Differentiated

The derivative section is easier to read after separating three objects:

$$\text{the exact statistical model,} \qquad \text{the adaptive approximation policy,} \qquad \text{one fixed approximate scalar.} \tag{38.1}$$

Only the third object is differentiated here. The scalar is

$$\boxed{\widehat{\ell}_T(\beta; B) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta; B_t)} \tag{38.2}$$

where $B = (B_1, \dots, B_T)$ is a saved branch. The branch contains frozen structural fields:

$$B_{t,\text{frozen}} = \{\Omega_t, \Psi_t, Z_{\text{fit}}^{(t)}, W^{(t)}, b_{t,k}, R_{t,k}, c_t, \tau_t, \rho_t, S_t, \text{sweep order}\}. \tag{38.3}$$

The differentiable fields are recomputed from those frozen fields:

$$B_{t,\text{diff}}(\beta) = \{\widetilde{q}_{t,j}(\beta), y_{t,j}(\beta), A_{t,k}(\beta), N_{t,k}(\beta), d_{t,k}(\beta), C_{t,k}(\beta), \widehat{Z}_t(\beta)\}. \tag{38.4}$$

Thus the derivative is neither

$$\frac{\partial}{\partial \beta} \widehat{\ell}_T^{\,\text{adapt}}(\beta) \tag{38.5}$$

nor the derivative obtained by freezing all fitted core numbers. It is

$$\frac{\partial}{\partial \beta} \widehat{\ell}_T(\beta; B) \quad \text{with } B_{\text{frozen}} \text{ fixed and } B_{\text{diff}}(\beta) \text{ recomputed.} \tag{38.6}$$

Forward scalar path	Derivative path
$\widetilde{q}_{t,j} = \widehat{p}_{t-1,j} f_j g_j J_j$	$\dot{\widetilde{q}}_{t,j} = J_j(\dot{\widehat{p}}_{t-1,j} f_j g_j \widehat{p}_{t-1,j} \dot{f}_j g_j \widehat{p}_{t-1,j} f_j \dot{g}_j)$
$y_j = e^{c_t/2}\sqrt{\widetilde{q}_{t,j}}$	$\dot{y}_j = e^{c_t/2}\dot{\widetilde{q}}_{t,j}/(2\sqrt{\widetilde{q}_{t,j}})$
$A_k \to N_k = A_k^\top W A_k + \rho I,\ d_k = A_k^\top W y$	$\dot{A}_k \to \dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}$
$N_k g_k = d_k$	$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$
$g_k \to C_k \to M_{>k} \to \widehat{Z}_t$	$\dot{g}_k \to \dot{C}_k \to \dot{M}_{>k} \to \dot{\widehat{Z}}_t$
$\widehat{p}_t = a_t/\widehat{Z}_t$	$\dot{\widehat{p}}_t = \dot{a}_t/\widehat{Z}_t - a_t\dot{\widehat{Z}}_t/\widehat{Z}_t^2$

A finite-dimensional analog shows why the solve derivative is the correct object. Suppose the entire approximation at one time were a two-coefficient fit

$$\phi(z; \beta) = g_0(\beta) b_0(z) + g_1(\beta) b_1(z), \tag{38.7}$$

with fixed fitting matrix A and target values $y(\beta)$. Ridge least squares gives

$$g(\beta) = (A^\top W A + \rho I)^{-1} A^\top W y(\beta). \tag{38.8}$$

If A, W, ρ are frozen, then

$$\dot{g}(\beta) = (A^\top W A + \rho I)^{-1} A^\top W \dot{y}(\beta). \tag{38.9}$$

If A depends on already-updated neighboring cores, the same equation gains the $\dot{A}$ and $\dot{N}$ terms shown in the table. The conceptual point is unchanged: the derivative propagates through the fitted coefficients, not around them.

The derivative is useful for three reasons:

$$\begin{aligned} &\text{finite-difference verification of the declared scalar,} \\ &\text{local sensitivity analysis of the approximation,} \\ &\text{possible gradient-based exploration of a fixed approximation.} \end{aligned} \tag{38.10}$$

It is deliberately narrow for one reason:

$$B(\beta+h) \neq B(\beta) \quad \Rightarrow \quad \widehat{\ell}_T(\beta+h; B(\beta+h)) \text{ is a different function evaluation path.} \tag{38.11}$$

That adaptive derivative may be studied separately, but it is not what is proved in Proposition 2.

39 The Fixed-Branch Derivative Pass

The derivative pass computes the derivative of each forward object that depends on β. It does not create a new branch.

Derivative object	Forward object differentiated	Formula source
$\dot{\tilde{q}}_{t,j}$	target value at fitting point	product rule
$\dot{y}_j$	square-root fit value	chain rule
$\dot{A}_k, \dot{N}_k, \dot{d}_k$	least-squares system	environment derivatives
$\dot{C}_{t,k}$	fitted core	fixed linear solve
$\dot{M}_{t,>k}$	mass contraction	product rule in core indices
$\dot{\widehat{Z}}_t$	normalizer	squared-integral derivative
$\dot{a}_t$	carried numerator	marginal contraction derivative
$\dot{\widehat{p}}_t$	normalized carried filter	quotient rule
$\dot{\tilde{q}}_{t+1}$	next target	product rule using $\dot{\widehat{p}}_t$

39.1 Target And Square-Root Target

Differentiate (37.4). At fitting point $z^{(j)}$, abbreviate

$$\widehat{p}_{t-1,j} = \widehat{p}_{t-1}(x_{t-1}^{(j)}; \beta), \quad f_j = f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}), \quad g_j = g_\beta(y_t \mid x_t^{(j)}). \tag{39.1}$$

With $J_{t,j}$ frozen, the derivative is

$$\dot{\tilde{q}}_{t,j} = J_{t,j}\left[\dot{\widehat{p}}_{t-1,j} f_j g_j + \widehat{p}_{t-1,j}\dot{f}_j g_j + \widehat{p}_{t-1,j} f_j \dot{g}_j\right]. \tag{39.2}$$

If the domain map depended on β, an additional $\widehat{p} f g \, \dot{J}$ term would appear. That is outside the fixed branch used here.

From (37.5),

$$\dot{y}_j = e^{c_t/2}\frac{1}{2\sqrt{\tilde{q}_{t,j}}}\dot{\tilde{q}}_{t,j}, \tag{39.3}$$

provided $\widetilde{q}_{t,j} > 0$. A numerical implementation should use a positive floor if the target is close to zero. If such a floor is used, the flooring rule becomes part of the declared scalar. For example,

$$y_j(\beta) = e^{c_t/2}\sqrt{\max\{\widetilde{q}_{t,j}(\beta), \varepsilon_{\text{floor}}\}} \tag{39.4}$$

has derivative zero through the floor whenever $\widetilde{q}_{t,j} < \varepsilon_{\text{floor}}$. The formulas below describe the smooth positive-target case; a floored implementation must differentiate the floored scalar it actually evaluates.

39.2 Design Matrix And Core Solve

The design row is the first place where tensor-train notation can look more mysterious than it is. Fix one sample point $z^{(j)}$ and one core k. All cores except core k are treated as the known left and right environments for this local least-squares update. Write

$$L_{j,k}[a] = \left[H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)})\right]_{1a}, \tag{39.5}$$

$$R_{j,k}[b] = \left[H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)})\right]_{b1}. \tag{39.6}$$

The local core evaluated at $z_k^{(j)}$ is

$$H_k(z_k^{(j)})[a,b] = \sum_{\ell=1}^{m_k} C_k[a,\ell,b]\, b_k(z_k^{(j)})[\ell], \tag{39.7}$$

where m_k is the number of one-dimensional basis functions in coordinate k. The value of the tensor train at this one point is therefore

$$\begin{aligned}\phi_t(z^{(j)};\beta) &= \sum_{a=1}^{R_{k-1}}\sum_{b=1}^{R_k} L_{j,k}[a]\, H_k(z_k^{(j)})[a,b]\, R_{j,k}[b] \\ &= \sum_{a=1}^{R_{k-1}}\sum_{\ell=1}^{m_k}\sum_{b=1}^{R_k} L_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b]\, C_k[a,\ell,b].\end{aligned} \tag{39.8}$$

Equation (39.8) is the whole reason the local fit is a linear least-squares problem. At the fixed sample point, the unknown numbers are the entries of the core C_k. Every coefficient multiplying $C_k[a,\ell,b]$ is already known from the branch and from the other cores.

If $g_k = \operatorname{vec}(C_k)$ stacks entries in the order (a,ℓ,b), the row coefficient of $g_k[a,\ell,b]$ is

$$A_{j,k}[a,\ell,b] = L_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b]. \tag{39.9}$$

The compact Kronecker expression in (39.15) is just (39.9) written without three sums:

$$A_{j,k} = R_{j,k}^{\top} \otimes b_k(z_k^{(j)})^{\top} \otimes L_{j,k}. \tag{39.10}$$

To differentiate the row, keep the basis value fixed and differentiate the two environments:

$$\begin{aligned}\dot{A}_{j,k}[a,\ell,b] = {}& \dot{L}_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, R_{j,k}[b] \\ &+ L_{j,k}[a]\, b_k(z_k^{(j)})[\ell]\, \dot{R}_{j,k}[b].\end{aligned} \tag{39.11}$$

Equation (39.16) is (39.11) written in the same Kronecker shorthand. If the basis or fitting point moved with β, there would be an additional term with $\dot{b}_k(z_k^{(j)})$. That term is absent because the branch keeps basis and fitting points fixed.

For one core k, the least-squares relation has rows

$$\phi_t(z^{(j)};\beta) = A_{j,k}(\beta)g_k(\beta), \tag{39.12}$$

where $g_k = \mathrm{vec}(C_{t,k})$. Let

$$L_{j,k} = H_1(z_1^{(j)})\cdots H_{k-1}(z_{k-1}^{(j)}), \tag{39.13}$$

$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)})\cdots H_D(z_D^{(j)}). \tag{39.14}$$

Then

$$A_{j,k} = R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k}. \tag{39.15}$$

Because basis values and fitting points are frozen,

$$\dot{A}_{j,k} = \dot{R}_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k} + R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes \dot{L}_{j,k}. \tag{39.16}$$

The environments are differentiated by product rule:

$$\dot{L}_{j,1} = 0, \qquad \dot{L}_{j,k+1} = \dot{L}_{j,k}H_k(z_k^{(j)}) + L_{j,k}\dot{H}_k(z_k^{(j)}), \tag{39.17}$$

$$\dot{R}_{j,D} = 0, \qquad \dot{R}_{j,k-1} = \dot{H}_k(z_k^{(j)})R_{j,k} + H_k(z_k^{(j)})\dot{R}_{j,k}. \tag{39.18}$$

The local core matrix derivative is

$$\dot{H}_k(z_k^{(j)})[a,b] = \sum_\ell \dot{C}_k[a,\ell,b]\, b_k(z_k^{(j)})[\ell]. \tag{39.19}$$

The fixed ridge solve is

$$N_k g_k = d_k, \qquad N_k = A_k^\top W A_k + \rho I, \qquad d_k = A_k^\top W y. \tag{39.20}$$

Using (36.4),

$$N_k\dot{g}_k = \dot{d}_k - \dot{N}_k g_k, \tag{39.21}$$

where

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \tag{39.22}$$

$$\dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \tag{39.23}$$

The derivative update is therefore another linear solve with the same left-hand matrix N_k. This is why the fixed branch is implementable: the forward pass and derivative pass use the same stored computational path.

39.3 Mass Contraction And Normalizer

The full right-mass recursion is the same idea as the rank-R warmup in (35.1)–(35.8). In the warmup, $M_1[r,s]$ stored the inner products of the first-coordinate factors and $M_2[r,s]$ stored the inner products of the second-coordinate factors. In the full tensor train, $M_{>j}[b,b']$ plays the role of the already-integrated right factor, $B_j[\ell,\ell']$ is the one-coordinate basis inner product, and the two copies of C_j are the two factors coming from squaring ϕ_t.

The right mass contraction is

$$M_{>j-1}[a,a'] = \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell']. \tag{39.24}$$

Here B_j is the basis mass matrix. It is frozen. Differentiate (39.24):

$$\begin{aligned} \dot{M}_{>j-1}[a,a'] = &\sum_{b,b',\ell,\ell'} \dot{C}_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] \dot{M}_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] \dot{C}_j[a',\ell',b'] B_j[\ell,\ell']. \end{aligned} \tag{39.25}$$

Start from

$$M_{>D} = 1, \qquad \dot{M}_{>D} = 0. \tag{39.26}$$

Then (39.24)–(39.25) move from the right end of the chain to the left end. The final square integral is

$$R_t(\beta) = \int \phi_t(z;\beta)^2 \, \mathrm{d}z = M_{>0}. \tag{39.27}$$

Its derivative is

$$\dot{R}_t(\beta) = \dot{M}_{>0}. \tag{39.28}$$

By (17)–(18),

$$\widehat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t, \tag{39.29}$$

$$\dot{\widehat{Z}}_t(\beta) = e^{-c_t} \dot{R}_t(\beta). \tag{39.30}$$

39.4 Carried Filter And Next Time Step

The carried numerator is

$$a_t(z_t;\beta) = \int \widehat{q}_t(z_t, z_{t-1};\beta) \, \mathrm{d}z_{t-1}. \tag{39.31}$$

Its derivative is

$$\dot{a}_t(z_t;\beta) = \int \dot{\widehat{q}}_t(z_t, z_{t-1};\beta) \, \mathrm{d}z_{t-1}. \tag{39.32}$$

For the squared part of the approximation, this marginal is another tensor contraction. To make the formula concrete, suppose the retained coordinate is the last coordinate $z_D = z_t$, and the previous-state coordinates are $z_1, \dots, z_{D-1}$. First integrate out the previous-state coordinates by the same left-to-right mass recursion:

$$M_{<0} = 1, \qquad M_{<j}[b,b'] = \sum_{a,a',\ell,\ell'} M_{<j-1}[a,a'] \, C_j[a,\ell,b] \, C_j[a',\ell',b'] \, B_j[\ell,\ell'] \tag{39.33}$$

for $j = 1, \dots, D-1$. The derivative of this marginal environment is

$$\begin{aligned}\dot{M}_{<j}[b,b'] = &\sum_{a,a',\ell,\ell'} \dot{M}_{<j-1}[a,a']\, C_j[a,\ell,b]\, C_j[a',\ell',b']\, B_j[\ell,\ell'] \\ &+ \sum_{a,a',\ell,\ell'} M_{<j-1}[a,a']\, \dot{C}_j[a,\ell,b]\, C_j[a',\ell',b']\, B_j[\ell,\ell'] \\ &+ \sum_{a,a',\ell,\ell'} M_{<j-1}[a,a']\, C_j[a,\ell,b]\, \dot{C}_j[a',\ell',b']\, B_j[\ell,\ell'].\end{aligned} \tag{39.34}$$

Start with $\dot{M}_{<0} = 0$. After the recursion reaches $D-1$, the retained-coordinate numerator is the function

$$\begin{aligned}a_t(z_D;\beta) = &e^{-c_t} \sum_{b,b',\ell,\ell'} M_{<D-1}[b,b']\, C_D[b,\ell,1]\, C_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'] \\ &+ \tau_t \int \lambda_t(z_D, z_{1:D-1})\, \mathrm{d}z_{1:D-1}.\end{aligned} \tag{39.35}$$

Its derivative, with c_t, τ_t, λ_t and the basis fixed, is

$$\begin{aligned}\dot{a}_t(z_D;\beta) = &e^{-c_t} \sum_{b,b',\ell,\ell'} \dot{M}_{<D-1}[b,b']\, C_D[b,\ell,1]\, C_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b,b',\ell,\ell'} M_{<D-1}[b,b']\, \dot{C}_D[b,\ell,1]\, C_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'] \\ &+ e^{-c_t} \sum_{b,b',\ell,\ell'} M_{<D-1}[b,b']\, C_D[b,\ell,1]\, \dot{C}_D[b',\ell',1]\, b_D(z_D)[\ell]\, b_D(z_D)[\ell'].\end{aligned} \tag{39.36}$$

Equations (39.33)–(39.36) are the coding recipe behind the short integral identity (39.32). If the retained state is not the last coordinate, reorder the coordinates or use left and right environments around the retained block; the same product-rule pattern applies.

The normalized carried filter is

$$\widehat{p}_t(z_t;\beta) = \frac{a_t(z_t;\beta)}{\widehat{Z}_t(\beta)}. \tag{39.37}$$

Differentiate by quotient rule:

$$\dot{\widehat{p}}_t(z_t;\beta) = \frac{\dot{a}_t(z_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(z_t;\beta)\dot{\widehat{Z}}_t(\beta)}{\widehat{Z}_t(\beta)^2}. \tag{39.38}$$

This is the crucial bridge between time steps. At time $t+1$, the target contains $\widehat{p}_t$, so its derivative contains $\dot{\widehat{p}}_t$. Without saving the carried-filter derivative, the next-step gradient is incomplete.

39.5 Concrete One-Coordinate Carried-Filter Storage

The quotient formula (39.38) is a mathematical identity. A later implementation also needs a concrete stored object. In the minimal two-coordinate fixed branch, use the normalized Legendre basis $b_1(z) \in \mathbb{R}^p$ on the retained coordinate and store the carried numerator as a coefficient matrix:

$$Q_t[\ell,m] = e^{-c_t} \sum_{r,s=1}^{R} C_1[0,\ell,r] S_2[r,s] C_1[0,m,s] + \tau_t \mathbf{1}_{\ell=0}\mathbf{1}_{m=0}, \qquad \mathrm{shape}(Q_t) = (p,p). \tag{39.39}$$

The retained numerator is then the quadratic form

$$a_t(z_1;\beta) = b_1(z_1)^\top Q_t b_1(z_1). \tag{39.40}$$

The defensive term in (39.39) gives $\tau_t/2$ because the normalized Legendre constant satisfies $b_1(z)[0] = 1/\sqrt{2}$, hence

$$\tau_t b_1(z_1)[0]^2 = \tau_t/2. \tag{39.41}$$

The derivative coefficient matrix is

$$\begin{aligned}\dot{Q}_t[\ell,m] &= e^{-c_t}\sum_{r,s}\dot{C}_1[0,\ell,r]S_2[r,s]C_1[0,m,s] \\ &\quad + e^{-c_t}\sum_{r,s}C_1[0,\ell,r]\dot{S}_2[r,s]C_1[0,m,s] \\ &\quad + e^{-c_t}\sum_{r,s}C_1[0,\ell,r]S_2[r,s]\dot{C}_1[0,m,s], \qquad \operatorname{shape}(\dot{Q}_t) = (p,p).\end{aligned} \tag{39.42}$$

The normalized carried filter is stored as

$$P_t = \frac{Q_t}{\widehat{Z}_t}, \qquad \dot{P}_t = \frac{\dot{Q}_t}{\widehat{Z}_t} - \frac{Q_t\dot{\widehat{Z}}_t}{\widehat{Z}_t^2}, \qquad \operatorname{shape}(P_t) = \operatorname{shape}(\dot{P}_t) = (p,p). \tag{39.43}$$

For a query vector $z^{\mathrm{query}} \in [-1,1]^M$, build

$$B^{\mathrm{query}}[j,\ell] = b_1(z_j^{\mathrm{query}})[\ell], \qquad \operatorname{shape}(B^{\mathrm{query}}) = (M,p). \tag{39.44}$$

The carried evaluator returns

$$\begin{aligned}\widehat{p}_t^{\mathrm{ref}}[j] &= \sum_{\ell,m} B^{\mathrm{query}}[j,\ell]P_t[\ell,m]B^{\mathrm{query}}[j,m], \\ \dot{\widehat{p}}_t^{\mathrm{ref}}[j] &= \sum_{\ell,m} B^{\mathrm{query}}[j,\ell]\dot{P}_t[\ell,m]B^{\mathrm{query}}[j,m],\end{aligned} \qquad \operatorname{shape}(\widehat{p}_t^{\mathrm{ref}}) = \operatorname{shape}(\dot{\widehat{p}}_t^{\mathrm{ref}}) = (M,). \tag{39.45}$$

At the next time step, the query points are the previous-state coordinate of the fitting table:

$$z_j^{\mathrm{query}} = Z_{\mathrm{fit}}[j,2], \qquad p_j = \widehat{p}_{t-1}^{\mathrm{ref}}[j], \qquad \dot{p}_j = \dot{\widehat{p}}_{t-1}^{\mathrm{ref}}[j]. \tag{39.46}$$

The stored object is therefore not an informal function name. It is a pair of coefficient matrices and a declared query rule. This is what makes the next-step target derivative mechanically reproducible.

Multidimensional Retained-Filter Contract

For retained state dimension $m > 1$, let the retained block have coordinates

$$z_t = (z_{t,1},\dots,z_{t,m}), \qquad b_t(z_t) = b_{t,1}(z_{t,1}) \otimes \cdots \otimes b_{t,m}(z_{t,m}). \tag{39.47}$$

If coordinate j uses p_j basis functions, then

$$b_t(z_t) \in \mathbb{R}^{p_{\mathrm{ret}}}, \qquad p_{\mathrm{ret}} = \prod_{j=1}^{m} p_j. \tag{39.48}$$

The retained numerator is represented as a quadratic form

$$a_t(z_t;\beta) = b_t(z_t)^\top Q_t(\beta) b_t(z_t), \qquad Q_t \in \mathbb{R}^{p_{\mathrm{ret}} \times p_{\mathrm{ret}}}. \tag{39.49}$$

The derivative object has the same shape:

$$\dot{a}_t(z_t;\beta) = b_t(z_t)^\top \dot{Q}_t(\beta) b_t(z_t), \qquad \dot{Q}_t \in \mathbb{R}^{p_{\mathrm{ret}} \times p_{\mathrm{ret}}}. \tag{39.50}$$

Here is how Q_t is obtained from the inherited squared-TT contractions. Split the square-root TT coordinates into retained coordinates z_t and integrated coordinates w:

$$\phi_t(z_t, w) = \sum_{\alpha,\gamma} L_\alpha(z_t)\, S_{\alpha,\gamma}\, R_\gamma(w). \tag{39.51}$$

Expand the retained interface functions in the retained product basis:

$$L_\alpha(z_t) = \sum_{\ell=1}^{p_{\mathrm{ret}}} U_{\alpha,\ell} b_{t,\ell}(z_t), \qquad b_t(z_t) = (b_{t,1}(z_t), \dots, b_{t,p_{\mathrm{ret}}}(z_t))^\top. \tag{39.52}$$

The integrated square-mass over the non-retained block is

$$G_{\gamma,\gamma'} = \int R_\gamma(w) R_{\gamma'}(w)\, \mathrm{d}w. \tag{39.53}$$

Therefore

$$\int \phi_t(z_t, w)^2\, \mathrm{d}w = \sum_{\ell,\ell'} b_{t,\ell}(z_t) \left[\sum_{\alpha,\alpha',\gamma,\gamma'} U_{\alpha,\ell} S_{\alpha,\gamma} G_{\gamma,\gamma'} S_{\alpha',\gamma'} U_{\alpha',\ell'} \right] b_{t,\ell'}(z_t). \tag{39.54}$$

The bracketed matrix, plus the retained defensive contribution, is Q_t:

$$Q_t[\ell,\ell'] = e^{-c_t} \sum_{\alpha,\alpha',\gamma,\gamma'} U_{\alpha,\ell} S_{\alpha,\gamma} G_{\gamma,\gamma'} S_{\alpha',\gamma'} U_{\alpha',\ell'} + \tau_t Q_{\lambda,t}[\ell,\ell']. \tag{39.55}$$

Thus Q_t is an exact coefficient matrix for the retained numerator under the chosen product basis if the retained block is stored in the full product basis and the non-retained square-mass contractions are exact. It becomes a storage surrogate only if a later implementation compresses Q_t into a TT, low-rank, or sparse representation; such compression must preserve the evaluator in (39.58).

Normalize by

$$P_t = \frac{Q_t}{\widehat{Z}_t}, \qquad \dot{P}_t = \frac{\dot{Q}_t}{\widehat{Z}_t} - \frac{Q_t \dot{\widehat{Z}}_t}{\widehat{Z}_t^2}. \tag{39.56}$$

For M next-step query points, build the basis matrix

$$B^{\mathrm{query}}[j,:] = b_t(z_t^{\mathrm{query}}(j))^\top, \qquad B^{\mathrm{query}} \in \mathbb{R}^{M \times p_{\mathrm{ret}}}. \tag{39.57}$$

The retained evaluator returns

$$\widehat{p}_t^{\mathrm{ref}}[j] = B^{\mathrm{query}}[j,:] P_t B^{\mathrm{query}}[j,:]^\top, \tag{39.58}$$

and

$$\dot{\widehat{p}}_t^{\mathrm{ref}}[j] = B^{\mathrm{query}}[j,:] \dot{P}_t B^{\mathrm{query}}[j,:]^\top. \tag{39.59}$$

The next-step query rule is

$$z_t^{\mathrm{query}}(j) = \text{the previous-state block of } Z_{\mathrm{fit}}^{(t+1)}[j,:]. \tag{39.60}$$

This full matrix form can be large when m is large. A later implementation may store Q_t in TT or low-rank form, but then it must provide the same evaluator and derivative evaluator as (39.58)–(39.59).

Retained-Filter Storage When The Product Basis Is Too Large

The full retained matrix has

$$\#Q_t = p_{\mathrm{ret}}^2 = \left(\prod_{j=1}^{m} p_j\right)^2 = \prod_{j=1}^{m} p_j^2 \tag{39.61}$$

entries. If $m = 10$ and $p_j = 4$, then

$$p_{\mathrm{ret}} = 4^{10} = 1{,}048{,}576, \qquad p_{\mathrm{ret}}^2 \approx 1.1 \times 10^{12}, \tag{39.62}$$

so full matrix storage is no longer a mathematical implementation plan. The retained object should then be stored as an evaluator, not necessarily as a dense matrix.

One exact evaluator form is the marginal TT itself. Let the retained coordinates be $z_{t,1:m}$. Store cores $G_{t,j}$ representing the retained numerator:

$$a_t(z_t) = G_{t,1}(z_{t,1}) G_{t,2}(z_{t,2}) \cdots G_{t,m}(z_{t,m}), \tag{39.63}$$

with TT ranks $S_0 = S_m = 1$. Store dotted cores or a dotted evaluator

$$\dot{a}_t(z_t) = \sum_{r=1}^{m} G_{t,1}(z_{t,1}) \cdots \dot{G}_{t,r}(z_{t,r}) \cdots G_{t,m}(z_{t,m}). \tag{39.64}$$

The normalized retained evaluator is then

$$\widehat{p}_t(z_t) = a_t(z_t)/\widehat{Z}_t, \qquad \dot{\widehat{p}}_t(z_t) = \dot{a}_t(z_t)/\widehat{Z}_t - a_t(z_t)\dot{\widehat{Z}}_t/\widehat{Z}_t^2. \tag{39.65}$$

A second option is a low-rank quadratic basis representation:

$$Q_t \approx U_t S_t U_t^\top, \qquad U_t \in \mathbb{R}^{p_{\mathrm{ret}} \times r_Q}, \qquad r_Q \ll p_{\mathrm{ret}}. \tag{39.66}$$

Then

$$a_t(z_t) \approx \left(U_t^\top b_t(z_t)\right)^\top S_t \left(U_t^\top b_t(z_t)\right), \tag{39.67}$$

and the derivative includes all stored factors:

$$\begin{aligned} \dot{a}_t(z_t) \approx{}& (\dot{U}_t^\top b_t)^\top S_t (U_t^\top b_t) + (U_t^\top b_t)^\top \dot{S}_t (U_t^\top b_t) \\ &+ (U_t^\top b_t)^\top S_t (\dot{U}_t^\top b_t). \end{aligned} \tag{39.68}$$

This compression is acceptable only if it is declared as part of the fixed branch and the compression residual is tested:

$$\varepsilon_Q = \frac{\|Q_t - U_t S_t U_t^\top\|_{\mathcal{V}}}{\max\{\|Q_t\|_{\mathcal{V}}, \epsilon_{\mathrm{abs}}\}} \le \epsilon_Q, \tag{39.69}$$

where $\|\cdot\|_{\mathcal{V}}$ may be evaluated on a fixed validation grid or fixed quadrature rule. The derivative compression is tested analogously:

$$\varepsilon_{\dot{Q}} = \frac{\|\dot{Q}_t - \dot{Q}_t^{\mathrm{comp}}\|_{\mathcal{V}}}{\max\{\|\dot{Q}_t\|_{\mathcal{V}}, \epsilon_{\mathrm{abs}}\}} \le \epsilon_{\dot{Q}}. \tag{39.70}$$

Thus the retained-filter contract is not "store a dense matrix." It is

$$\mathcal{E}_t = \{z_t \mapsto \widehat{p}_t(z_t),\ z_t \mapsto \dot{\widehat{p}}_t(z_t),\ \int \widehat{p}_t(z_t)\,\mathrm{d}z_t = 1,\ \int \dot{\widehat{p}}_t(z_t)\,\mathrm{d}z_t = 0\}. \tag{39.71}$$

Any dense, TT, sparse, or low-rank storage format is admissible only if it implements $\mathcal{E}_t$ and passes the normalization and derivative-mass checks in (39.71).

The compressed retained object has its own failure diagnostics:

diagnostic	failure test
normalization drift	$\left\lvert \int \widehat{p}_t(z_t)\,\mathrm{d}z_t - 1 \right\rvert > \epsilon_{\mathrm{norm}}$
derivative-mass drift	$\left\lvert \int \dot{\widehat{p}}_t(z_t)\,\mathrm{d}z_t \right\rvert > \epsilon_{\mathrm{norm}}$
value compression residual	$\varepsilon_Q > \epsilon_Q$
derivative compression residual	$\varepsilon_{\dot{Q}} > \epsilon_{\dot{Q}}$
query mismatch	$\max_j \lvert \widehat{p}_t^{\mathrm{comp}}(z_j) - \widehat{p}_t^{\mathrm{ref}}(z_j) \rvert > \epsilon_Q$

(39.72)

Normalization drift means the retained density no longer integrates to one. Derivative-mass drift means the quotient derivative or compression is inconsistent. Compression residuals mean the value or derivative evaluator is not accurate enough. Query mismatch means the next-step target would use the wrong retained factor. The query mismatch test uses a fixed validation table of retained-coordinate points. If no dense reference is available, compare two independently built evaluators or a higher-rank retained representation on the same fixed table.

40 Proposition 1: The Fixed-Branch Recursion Is A Normalized Approximate Filter

Proposition 2. *Fix a parameter value β, a time horizon T, and fixed branch objects for all times. Suppose that each approximate joint density*

$$\widehat{q}_t(z_t, z_{t-1}; \beta) = e^{-c_t}\phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1}) \tag{40.1}$$

is nonnegative and integrable, and that

$$0 < \widehat{Z}_t(\beta) = \int \widehat{q}_t(z_t, z_{t-1}; \beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} < \infty. \tag{40.2}$$

Define

$$\widehat{p}_t(z_t; \beta) = \frac{\int \widehat{q}_t(z_t, z_{t-1}; \beta)\,\mathrm{d}z_{t-1}}{\widehat{Z}_t(\beta)}. \tag{40.3}$$

Then $\widehat{p}_t(\cdot; \beta)$ is a normalized density in z_t. The recursion is a Bayesian filtering recursion for the declared approximate joint density $\widehat{q}_t$, not a proof that $\widehat{q}_t$ equals the exact Bayesian joint density.

Proof. Nonnegativity follows immediately:

$$e^{-c_t}\phi_t^2 \geq 0, \qquad \tau_t \lambda_t \geq 0. \tag{40.4}$$

Therefore $\widehat{q}_t \geq 0$. By assumption, $\widehat{Z}_t$ is positive and finite, so

$$\widehat{p}_t^{\mathrm{joint}}(z_t, z_{t-1}; \beta) = \frac{\widehat{q}_t(z_t, z_{t-1}; \beta)}{\widehat{Z}_t(\beta)} \tag{40.5}$$

is a normalized joint density:

$$\begin{aligned} \iint \widehat{p}_t^{\mathrm{joint}}(z_t, z_{t-1}; \beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} &= \frac{1}{\widehat{Z}_t(\beta)} \iint \widehat{q}_t(z_t, z_{t-1}; \beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} \\ &= 1. \end{aligned} \tag{40.6}$$

The retained filter is the marginal of this normalized joint density:

$$\widehat{p}_t(z_t;\beta) = \int \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}. \tag{40.7}$$

Integrating (40.7) over z_t,

$$\begin{aligned}\int \widehat{p}_t(z_t;\beta)\,\mathrm{d}z_t &= \int \left[\int \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}\right]\mathrm{d}z_t \\ &= \iint \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1};\beta)\,\mathrm{d}z_t\,\mathrm{d}z_{t-1} \\ &= 1.\end{aligned} \tag{40.8}$$

Thus $\widehat{p}_t$ is normalized. The proof used only the declared approximation $\widehat{q}_t$. It did not use the exact Bayesian density except as motivation for constructing $\widehat{q}_t$. Hence the theorem is a normalization theorem for the approximate filtering recursion. □

41 The Story Of The Fixed-Branch Derivative

Before the formal proposition, it is useful to separate the idea from the notation. The adaptive Zhao and Cui method chooses approximation structure as it runs [4]. A rank may change, a pivot may change, or an interpolation set may change. Such choices are excellent for approximation quality, but they are discrete choices. A discrete choice can jump when β is perturbed:

$$B_{\mathrm{adaptive}}(\beta_0 - h) \neq B_{\mathrm{adaptive}}(\beta_0) \neq B_{\mathrm{adaptive}}(\beta_0 + h). \tag{41.1}$$

Across such jumps, there is no single smooth scalar being differentiated. The fixed-branch construction deliberately chooses a narrower scalar:

$$\widehat{\ell}_T(\beta; B) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta; B), \tag{41.2}$$

where

$$B(\beta_0 - h) = B(\beta_0) = B(\beta_0 + h) = B. \tag{41.3}$$

This is why the derivative is useful but narrower. It does not claim to differentiate the adaptive rank-changing algorithm. It differentiates the declared approximate likelihood obtained after the branch has been fixed.

The forward pass at one time step is a chain of deterministic maps:

$$\beta \mapsto \widetilde{q}_{t,j}(\beta) \mapsto y_j(\beta) \mapsto (A_k(\beta), N_k(\beta), d_k(\beta)) \mapsto C_{t,k}(\beta) \mapsto \widehat{Z}_t(\beta) \mapsto \widehat{p}_t(\cdot;\beta). \tag{41.4}$$

The derivative pass follows the same chain with the product rule, chain rule, linear-solve derivative, and quotient rule:

$$\dot{\widetilde{q}}_{t,j} \mapsto \dot{y}_j \mapsto (\dot{A}_k, \dot{N}_k, \dot{d}_k) \mapsto \dot{C}_{t,k} \mapsto \dot{\widehat{Z}}_t \mapsto \dot{\widehat{p}}_t. \tag{41.5}$$

The only difficult-looking step is the fitted core derivative. But each core update is just a fixed linear solve:

$$N_k(\beta) g_k(\beta) = d_k(\beta). \tag{41.6}$$

Differentiating the left side gives

$$\dot{N}_k g_k + N_k \dot{g}_k = \dot{d}_k, \tag{41.7}$$

and therefore

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \tag{41.8}$$

After this, the normalizer derivative is only

$$\widehat{Z}_t = e^{-c_t} \int \phi_t^2 \, \mathrm{d}z + \tau_t, \qquad \dot{\widehat{Z}}_t = e^{-c_t} \int 2\phi_t \dot{\phi}_t \, \mathrm{d}z, \tag{41.9}$$

and the carried filter derivative is only

$$\widehat{p}_t = a_t / \widehat{Z}_t, \qquad \dot{\widehat{p}}_t = \dot{a}_t / \widehat{Z}_t - a_t \dot{\widehat{Z}}_t / \widehat{Z}_t^2. \tag{41.10}$$

Thus Proposition 2 has a simple message. If the branch is held fixed and all objects are recomputed by the same declared equations, then the dotted recursion is the derivative of the same scalar produced by the forward recursion:

$$\text{forward scalar } \widehat{\ell}_T(\beta; B) \quad \Longleftrightarrow \quad \text{dotted scalar } \partial_\beta \widehat{\ell}_T(\beta; B). \tag{41.11}$$

The proposition is not a claim that the approximate scalar equals the exact Bayesian evidence, not a claim that the adaptive algorithm is globally smooth, and not a claim that the numerical linear algebra is stable. Those are separate approximation and diagnostics questions.

42 Proposition 2: The Fixed-Branch Derivative Differentiates The Same Scalar

Equation Dependency Diagram For Proposition 2

The derivative proposition follows one computational graph. In equation form, the graph is

$$\widetilde{q}_{t,j}(\beta) \xrightarrow{\sqrt{\cdot}} y_j(\beta) \xrightarrow{\text{fixed LS rows}} (A_k(\beta), d_k(\beta), N_k(\beta)) \xrightarrow{N_k g_k = d_k} C_{t,k}(\beta). \tag{42.1}$$

The differentiated graph is

$$\dot{\widetilde{q}}_{t,j} \to \dot{y}_j \to (\dot{A}_k, \dot{d}_k, \dot{N}_k) \to N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k \to \dot{C}_{t,k}. \tag{42.2}$$

This is a mathematical same-scalar derivative statement, not a numerical stability theorem. If

$$\kappa(N_k) = \|N_k\| \, \|N_k^{-1}\| \gg 1, \tag{42.3}$$

then the solve derivative $N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$ can amplify small target or roundoff errors. The branch therefore records the condition numbers already listed in (28.31), and a branch with unstable solves may differentiate the same scalar mathematically while being numerically unusable.

The fitted cores define mass contractions:

$$C_{t,k} \to M_{>k} \to R_t = \int \phi_t^2 \to \widehat{Z}_t = e^{-c_t} R_t + \tau_t, \tag{42.4}$$

and the derivative contractions are

$$(C_{t,k}, \dot{C}_{t,k}) \to \dot{M}_{>k} \to \dot{R}_t \to \dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t. \tag{42.5}$$

Mass contractions have the same caveat. Repeated products of poorly scaled core matrices can make R_t and $\dot{R}_t$ sensitive even when the algebra is correct. This is why rescaling in (15.22), stabilization

diagnostics in (28.31)–(28.33), and bridge diagnostics in (24.11) are evidence gates rather than decorative bookkeeping.

The carried filter closes the time recursion:

$$(C_{t,k}, M_{<k}) \to a_t \to \widehat{p}_t = a_t/\widehat{Z}_t \to \widetilde{q}_{t+1,j}, \tag{42.6}$$

and

$$(\dot{C}_{t,k}, \dot{M}_{<k}) \to \dot{a}_t \to \dot{\widehat{p}}_t = \dot{a}_t/\widehat{Z}_t - a_t\dot{\widehat{Z}}_t/\widehat{Z}_t^2 \to \dot{\widetilde{q}}_{t+1,j}. \tag{42.7}$$

Finally,

$$\{\widehat{Z}_t, \dot{\widehat{Z}}_t\}_{t=1}^T \to \widehat{\ell}_T(\beta; B) = \sum_t \log \widehat{Z}_t, \qquad \dot{\widehat{\ell}}_T = \sum_t \dot{\widehat{Z}}_t/\widehat{Z}_t. \tag{42.8}$$

Proposition 2 proves that the derivative path (42.2), (42.5), (42.7), and (42.8) differentiates the same scalar computed by (42.1), (42.4), (42.6), and (42.8). For the running example, the first link in (42.2) is

$$\begin{aligned}\dot{\widetilde{q}}_{t,j} = {} & \dot{\widehat{p}}_{t-1}(x_{t-1}^{(j)})\varphi(x_t^{(j)}; \beta x_{t-1}^{(j)}, \sigma^2)\varphi(y_t; \sin x_t^{(j)}, \tau_y^2) \\ & + \widehat{p}_{t-1}(x_{t-1}^{(j)})\partial_\beta\varphi(x_t^{(j)}; \beta x_{t-1}^{(j)}, \sigma^2)\varphi(y_t; \sin x_t^{(j)}, \tau_y^2),\end{aligned} \tag{42.9}$$

with the observation factor independent of β in this example. This concrete derivative is then passed through the same fixed sweep rows and normal equations as the generic graph above.

Lemma 2 (Fixed ridge sweeps are differentiable maps)**.** *Fix one branch: the coordinate domains, basis functions, fitting points, weights, ranks, sweep order, ridge parameter, defensive terms, and shifts are all held fixed. Consider one core update written as a ridge least-squares normal equation*

$$N_k(\beta)g_k(\beta) = d_k(\beta), \qquad N_k(\beta) = A_k(\beta)^\top W_k A_k(\beta) + \varepsilon_k I,$$

where $\varepsilon_k > 0$, W_k is fixed positive semidefinite, and $N_k(\beta)$ is nonsingular in the neighborhood being differentiated. If the entries of $A_k(\beta)$ and $d_k(\beta)$ are differentiable, then

$$\dot{g}_k(\beta) = N_k(\beta)^{-1}\left[\dot{d}_k(\beta) - \dot{N}_k(\beta)g_k(\beta)\right].$$

Thus this fixed core update is differentiable. A fixed finite sweep is a finite composition of such updates, so the fitted cores produced by the fixed sweep rule are differentiable functions of β.

Proof. Because $N_k(\beta)$ is nonsingular, the core vector is

$$g_k(\beta) = N_k(\beta)^{-1}d_k(\beta).$$

Equivalently, it satisfies $N_k g_k = d_k$. Differentiate this identity:

$$\dot{N}_k g_k + N_k \dot{g}_k = \dot{d}_k.$$

Solving for $\dot{g}_k$ gives the displayed formula. The assumptions that the branch, points, weights, ranks, ridge parameter, shifts, and sweep order are fixed ensure that no discrete choice changes while differentiating. Therefore each update is an ordinary differentiable map, and composing finitely many updates gives an ordinary differentiable fitted-core map. □

Proposition 3. *Fix the branch $B = (B_1, \dots, B_T)$. Assume:*

$$\widehat{\ell}_T(\beta; B) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta; B_t), \tag{42.10}$$

each $\widehat{Z}_t$ is positive and finite, all model density evaluations are differentiable in β, every least-squares update uses a fixed nonsingular ridge system, and differentiation under the finite-dimensional contractions and integrals is valid. Then the derivative pass defined by (39.2)–(39.38) computes

$$\frac{\partial}{\partial \beta} \widehat{\ell}_T(\beta; B). \tag{42.11}$$

Proof. The proof follows the computational graph of the fixed branch.

First, the transformed target (37.4) is a product of the previous carried filter, transition density, observation density, and frozen Jacobian. The product rule therefore gives (39.2). If $t = 1$, the previous filter is the initial density; for $t > 1$, its derivative is supplied by the previous carried-filter derivative (39.38).

Second, the fitted target values (37.5) are differentiable functions of $\widetilde{q}_{t,j}$. The chain rule gives (39.3). The stabilizing shift c_t is frozen, so no $\dot{c}_t$ term appears.

Third, each core update is a fixed linear solve. Equations (36.1)–(36.5) prove that differentiating the solve yields

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \tag{42.12}$$

Equations (39.12)–(39.23) identify $N_k, d_k, \dot{N}_k, \dot{d}_k$ for the fixed least-squares update. Since the number and order of solves are fixed, a finite composition of differentiable solve maps gives differentiable fitted cores.

Fourth, the square integral $R_t = \int \phi_t^2$ is computed by the mass recursion (39.24). Its derivative is the product-rule recursion (39.25). Therefore

$$\dot{R}_t = \frac{\partial}{\partial \beta} \int \phi_t(z; \beta)^2 \, \mathrm{d}z. \tag{42.13}$$

With frozen c_t, τ_t, λ_t, (39.29)–(39.30) give the derivative of $\widehat{Z}_t$.

Fifth, the carried numerator and carried filter are defined by (39.31)–(39.37). Their derivatives are (39.32) and (39.38), obtained by differentiating under the integral and by the quotient rule. This supplies the derivative object needed by the next time step.

Finally, since each time contribution is $\log \widehat{Z}_t(\beta; B_t)$, Warmup 1 gives

$$\frac{\partial}{\partial \beta} \log \widehat{Z}_t(\beta; B_t) = \frac{\dot{\widehat{Z}}_t(\beta; B_t)}{\widehat{Z}_t(\beta; B_t)}. \tag{42.14}$$

Summing (42.14) over $t = 1, \dots, T$ gives (42.11).

Every object differentiated in this proof is an object in the saved branch B. The proof never differentiates a changed rank, changed grid, changed shift, changed pivot set, or changed domain. Therefore the derivative is the derivative of the same scalar $\widehat{\ell}_T(\beta; B)$, not the derivative of an adaptive algorithm that chooses a new B when β changes. □

43 What This Does Not Prove

The result above is deliberately narrow.

It does not prove exact posterior accuracy:

$$\widehat{q}_t = q_t \quad \text{is not claimed.} \tag{43.1}$$

It proves normalization and differentiation of the declared approximation.

It does not prove ranks stay small:

$$R_k \text{ moderate for all } k \quad \text{is a diagnostic requirement, not a theorem here.} \tag{43.2}$$

It does not differentiate adaptive decisions:

$$\frac{\partial}{\partial \beta}\{\text{chosen ranks, pivots, domains, shifts, fitting points}\} \quad \text{is not part of the formula.} \tag{43.3}$$

It does not prove HMC convergence:

$$\text{correct local gradient of an approximate scalar} \neq \text{posterior sampling guarantee.} \tag{43.4}$$

These caveats make the result more honest, not weaker. The fixed-branch gradient is a precise mathematical statement. Empirical accuracy, rank stability, and sampling performance must be assessed separately.

44 Two-Time-Step Numerical Trace

This trace uses the scalar nonlinear model from the opening example with

$$\beta = 0.7, \qquad \sigma = 0.5, \qquad r = 0.3, \qquad \mu_0 = 0, \qquad \sigma_0 = 1, \qquad y_1 = 0.40, \qquad y_2 = 0.10. \tag{44.1}$$

Use physical domains $x_0, x_1, x_2 \in [-2, 2]$, reference map

$$x = 2z, \qquad z \in [-1, 1], \qquad J = 2, \tag{44.2}$$

and a rank-one constant basis $b_0(z) = 1/\sqrt{2}$ for the first hand calculation. This basis is too small for a real run; it is chosen so every number can be inspected.

At time one, take the fitting point $(z_1, z_0) = (0, 0)$, so

$$x_1 = 0, \qquad x_0 = 0. \tag{44.3}$$

The density factors are

$$p(x_0 = 0) = \frac{1}{\sqrt{2\pi}} \approx 0.39894, \qquad f_\beta(0 \mid 0) = \frac{1}{\sqrt{2\pi(0.5)^2}} \approx 0.79788, \tag{44.4}$$

and

$$g(y_1 = 0.40 \mid x_1 = 0) = \frac{1}{\sqrt{2\pi(0.3)^2}} \exp\left[-\frac{0.40^2}{2(0.3)^2}\right] \approx 0.54670. \tag{44.5}$$

Thus the transformed target value at this point is

$$\widetilde{q}_1(0, 0) = p(0) f_\beta(0 \mid 0) g(0.40 \mid 0) J^2 \approx 0.39894 \cdot 0.79788 \cdot 0.54670 \cdot 4 \approx 0.6960. \tag{44.6}$$

With $c_1 = 0$, the square-root fitting value is

$$y_{\mathrm{fit},1} = \sqrt{0.6960} \approx 0.8343. \tag{44.7}$$

The rank-one constant TT has

$$\phi_1(z_1, z_0) = C\, b_0(z_1) b_0(z_0) = \frac{C}{2}. \tag{44.8}$$

Matching $\phi_1(0,0) = 0.8343$ gives

$$C \approx 1.6686. \tag{44.9}$$

Because b_0 is normalized,

$$\int_{[-1,1]^2} \phi_1(z)^2 \, \mathrm{d}z = C^2 \approx 2.784. \tag{44.10}$$

With $\tau_1 = 10^{-6}$, the first approximate normalizer is

$$\widehat{Z}_1 \approx 2.784001. \tag{44.11}$$

The retained reference filter is

$$\widehat{p}_1(z_1) = \frac{\int \phi_1(z_1, z_0)^2 \, \mathrm{d}z_0 + \tau_1/2}{\widehat{Z}_1} \approx \frac{C^2/2}{C^2} = \frac{1}{2}, \tag{44.12}$$

which is uniform in this deliberately crude constant approximation.

At time two, use $(z_2, z_1) = (0, 0)$, hence $x_2 = x_1 = 0$. The carried filter contributes $\widehat{p}_1(z_1) = 1/2$ in reference coordinates. The transition factor remains 0.79788, while

$$g(y_2 = 0.10 \mid x_2 = 0) = \frac{1}{\sqrt{2\pi(0.3)^2}} \exp\left[-\frac{0.10^2}{2(0.3)^2}\right] \approx 1.2579. \tag{44.13}$$

Therefore

$$\widetilde{q}_2(0,0) = \widehat{p}_1(0) f_\beta(0 \mid 0) g(0.10 \mid 0) J^2 \approx 0.5 \cdot 0.79788 \cdot 1.2579 \cdot 4 \approx 2.007. \tag{44.14}$$

The fitted square-root value is

$$y_{\mathrm{fit},2} = \sqrt{2.007} \approx 1.417. \tag{44.15}$$

The same constant-basis calculation now gives a second approximate normalizer:

$$\phi_2(z_2, z_1) = \frac{C_2}{2}, \qquad C_2 = 2(1.417) \approx 2.834, \tag{44.16}$$

so

$$\widehat{Z}_2 = \int_{[-1,1]^2} \phi_2(z_2, z_1)^2 \, \mathrm{d}z_2 \, \mathrm{d}z_1 + \tau_2 \approx C_2^2 + \tau_2 \approx 8.032 + \tau_2. \tag{44.17}$$

The two-step approximate evidence scalar is therefore

$$\widehat{\ell}_2 = \log \widehat{Z}_1 + \log \widehat{Z}_2 \approx \log(2.784001) + \log(8.032001) \approx 3.108. \tag{44.18}$$

The retained filter at time two is again uniform under the intentionally crude constant basis:

$$\widehat{p}_2(z_2) \approx \frac{1}{2}. \tag{44.19}$$

The sequential bookkeeping can be summarized as a chain of quantities:

t	carried input	$\widetilde{q}_t(0,0)$	$y_{\mathrm{fit},t}(0,0)$	$\widehat{Z}_t$	$\widehat{\ell}_t$
1	$p_0(x_0 = 0) = 0.3989$	0.6960	0.8343	2.7840	1.024
2	$\widehat{p}_1(z_1 = 0) = 0.5$	2.007	1.417	8.0320	3.108

(44.20)

This table is small, but it displays the actual filtering loop: the normalized retained object from time one is not a side output; it is the first factor in the time-two target.

One derivative term is also visible at this point. The transition derivative is

$$\partial_\beta f_\beta(x_t \mid x_{t-1}) = f_\beta(x_t \mid x_{t-1}) \frac{(x_t - \beta x_{t-1}) x_{t-1}}{\sigma^2}. \tag{44.21}$$

At the central point $x_t = x_{t-1} = 0$, this term is zero. At a second fitting point, say $x_t = 1$, $x_{t-1} = 1$, it is

$$\partial_\beta f_\beta(1 \mid 1) = f_\beta(1 \mid 1) \frac{(1 - 0.7) \cdot 1}{0.25} = 1.2\, f_\beta(1 \mid 1). \tag{44.22}$$

The finite-difference check for the declared branch is

$$D_h = \frac{\widehat{\ell}_2(\beta + h; B) - \widehat{\ell}_2(\beta - h; B)}{2h}, \qquad h = 10^{-4}, \tag{44.23}$$

with the same domains, fitting points, rank-one basis, shifts, and constants on both sides. Changing any of those objects changes B, so it would no longer test the derivative of the same scalar. Using the same frozen branch, the numerical comparison is displayed in (45.24)–(45.25) after the two-point basis trace, where $D(h)$ is compared directly to the analytical gradient G.

45 A Less-Toy Two-Point Basis Trace

The constant-basis trace shows the mass bookkeeping, but it hides the fitting idea. This second trace keeps the same scalar model but uses a two-function basis

$$b_0(z) = \frac{1}{\sqrt{2}}, \qquad b_1(z) = \sqrt{\frac{3}{2}}\, z, \qquad z \in [-1, 1]. \tag{45.1}$$

For a rank-one separable square-root approximation at time one, write

$$\phi_1(z_1, z_0) = (a_0 b_0(z_1) + a_1 b_1(z_1))\,(c_0 b_0(z_0) + c_1 b_1(z_0))\,. \tag{45.2}$$

This is still a small approximation, but it can represent a slope in each coordinate.

Use three fitting points:

$$(z_1, z_0) \in \{(0,0), (1/2, 0), (0, 1/2)\}. \tag{45.3}$$

With $x = 2z$, these are

$$(x_1, x_0) \in \{(0,0), (1,0), (0,1)\}. \tag{45.4}$$

Using the same parameters as (44.1), the transformed targets are approximately

(x_1, x_0)	$p(x_0)$	$f_\beta(x_1 \mid x_0)$	$g(y_1 \mid x_1)$	$\widetilde{q}_1$
$(0,0)$	0.3989	0.7979	0.5467	0.696
$(1,0)$	0.3989	0.1080	0.378	0.065
$(0,1)$	0.2420	0.2995	0.5467	0.159

(45.5)

The square-root fitting values are therefore

$$y = (\sqrt{0.696}, \sqrt{0.065}, \sqrt{0.159})^\top \approx (0.834, 0.255, 0.399)^\top. \tag{45.6}$$

To keep the arithmetic visible, initialize $c_0 = \sqrt{2}$ and $c_1 = 0$, so the previous-state factor is one. Fit the current-state factor

$$u(z_1) = a_0 b_0(z_1) + a_1 b_1(z_1) \tag{45.7}$$

to the first two rows. The design matrix is

$$A = \begin{bmatrix} b_0(0) & b_1(0) \\ b_0(1/2) & b_1(1/2) \end{bmatrix} = \begin{bmatrix} 0.7071 & 0 \\ 0.7071 & 0.6124 \end{bmatrix}. \tag{45.8}$$

Solving $Aa = (0.834, 0.255)^\top$ gives

$$a_0 \approx 1.180, \qquad a_1 \approx -0.946. \tag{45.9}$$

The fitted values reproduce the two displayed rows:

$$\begin{bmatrix} 0.7071 & 0 \\ 0.7071 & 0.6124 \end{bmatrix} \begin{bmatrix} 1.180 \\ -0.946 \end{bmatrix} \approx \begin{bmatrix} 0.834 \\ 0.255 \end{bmatrix}. \tag{45.10}$$

Now initialize a_0, a_1 at these values and fit the previous-state factor

$$v(z_0) = c_0 b_0(z_0) + c_1 b_1(z_0) \tag{45.11}$$

using the first and third rows. Since $u(0) = 0.834$, the values for $v(z_0)$ are approximately

$$v(0) = 1, \qquad v(1/2) = 0.399/0.834 \approx 0.478. \tag{45.12}$$

The same 2×2 solve gives

$$c_0 \approx 1.414, \qquad c_1 \approx -0.852. \tag{45.13}$$

Again the displayed rows are reproduced:

$$\begin{bmatrix} 0.7071 & 0 \\ 0.7071 & 0.6124 \end{bmatrix} \begin{bmatrix} 1.414 \\ -0.852 \end{bmatrix} \approx \begin{bmatrix} 1.000 \\ 0.478 \end{bmatrix}. \tag{45.14}$$

Combining the two factors gives

$$\phi_1(0,0) \approx 0.834, \qquad \phi_1(1/2, 0) \approx 0.255, \qquad \phi_1(0, 1/2) \approx 0.399, \tag{45.15}$$

so the three square-root target values in (45.6) are matched to the displayed precision.

Because the basis is orthonormal on $[-1, 1]$, the square mass factorizes:

$$\int \phi_1(z_1, z_0)^2 \, \mathrm{d}z_1 \, \mathrm{d}z_0 = (a_0^2 + a_1^2)(c_0^2 + c_1^2). \tag{45.16}$$

With the numbers above,

$$a_0^2 + a_1^2 \approx 2.288, \qquad c_0^2 + c_1^2 \approx 2.726, \qquad \widehat{Z}_1 \approx 6.237 + \tau_1. \tag{45.17}$$

The retained filter in reference coordinates is proportional to

$$a_1^{\mathrm{ret}}(z_1) = u(z_1)^2 (c_0^2 + c_1^2) + \frac{\tau_1}{2}. \tag{45.18}$$

At $z_1 = 0$,

$$\widehat{p}_1^{\mathrm{ref}}(0) \approx \frac{0.834^2 \cdot 2.726}{6.237} \approx 0.304. \tag{45.19}$$

The approximation is still crude, but the retained filter is no longer uniform.

The derivative is also visible. At fitting point $(x_1, x_0) = (0, 1)$,

$$\partial_\beta f_\beta(0 \mid 1) = f_\beta(0 \mid 1)\frac{(0-\beta)\cdot 1}{\sigma^2} = -\frac{0.7}{0.25} f_\beta(0 \mid 1) = -2.8 f_\beta(0 \mid 1). \tag{45.20}$$

Therefore the transformed target derivative at that point is

$$\partial_\beta \widetilde{q}_1(0,1) = p(1) g(0.40 \mid 0) J^2 \partial_\beta f_\beta(0 \mid 1) \approx -2.8\, \widetilde{q}_1(0,1) \approx -0.445. \tag{45.21}$$

The square-root derivative is

$$\dot{y}_3 = \frac{\partial_\beta \widetilde{q}_1(0,1)}{2\sqrt{\widetilde{q}_1(0,1)}} \approx \frac{-0.445}{2(0.399)} \approx -0.558. \tag{45.22}$$

This number is the entry that enters $A^\top W \dot{y}$ in the derivative least-squares system. A same-branch finite difference compares

$$\frac{\widehat{\ell}_1(\beta + h; B) - \widehat{\ell}_1(\beta - h; B)}{2h} \quad \text{to} \quad \frac{\dot{\widehat{Z}}_1(\beta; B)}{\widehat{Z}_1(\beta; B)} \tag{45.23}$$

using the same three fitting points, the same basis, the same two alternating updates, and the same frozen branch fields.

For a synthetic fixed-branch check of the two-step scalar, suppose the analytical derivative at $\beta_0 = 0.7$ is

$$G = \partial_\beta \widehat{\ell}_2(\beta_0; B) = -0.384721. \tag{45.24}$$

The centered finite-difference values might be recorded as

h	$D(h)$	G	$\lvert D(h) - G \rvert$
10^{-2}	-0.384102	-0.384721	6.19×10^{-4}
10^{-3}	-0.384715	-0.384721	6.00×10^{-6}
10^{-4}	-0.384721	-0.384721	7.20×10^{-8}
10^{-5}	-0.384720	-0.384721	9.50×10^{-7}

(45.25)

The expected pattern is visible: the error decreases as h moves from 10^{-2} to 10^{-4}, then roundoff and fitting sensitivity begin to dominate at 10^{-5}. The table would support the claim that the coded derivative matches the declared fixed scalar. It would not support the claim that the fixed scalar is an accurate approximation to the exact evidence.

46 What Must Be Held Fixed For The Derivative

The finite-difference diagnostic needs a precise meaning for "the same branch." The fixed-field record is

$$\mathcal{H}(B) = \{\mathcal{H}_{\mathrm{fixed}}, \mathcal{H}_{\mathrm{diff}}, \mathcal{H}_{\mathrm{diag}}\}. \tag{46.1}$$

The frozen fields are

$$\begin{aligned} \mathcal{H}_{\mathrm{fixed}} = \{&\Omega_{t,k}, \Psi_{t,k}, J_t, Z_{\mathrm{fit}}^{(t)}, W^{(t)}, p_{t,k}, R_{t,k}, S_t, \rho_t, \\ &c_t, \tau_t, \lambda_t, \epsilon_{\mathrm{floor}}, \epsilon_{\mathrm{root}}, N_{\mathrm{root}}, \text{sweep order}, \text{initialization rule}\}_{t,k}. \end{aligned} \tag{46.2}$$

The differentiable fields are

$$\begin{aligned}\mathcal{H}_{\text{diff}}(\beta) = \{&y_j(\beta), A_k(\beta), N_k(\beta), d_k(\beta),\\ &C_{t,k}(\beta), M_{>k}(\beta), \widehat{Z}_t(\beta),\\ &a_t(\beta), \widehat{p}_t(\beta)\}.\end{aligned} \tag{46.3}$$

The diagnostic fields are

$$\begin{aligned}\mathcal{H}_{\text{diag}} = \{&\text{fit residuals}, \text{condition numbers}, \text{rank vector},\\ &\text{domain failures}, \text{KR failures}, \text{stabilization failures}\}.\end{aligned} \tag{46.4}$$

Two evaluations use the same branch if and only if their frozen fields match:

$$B(\beta_a) = B(\beta_b) \quad \Longleftrightarrow \quad \mathcal{H}_{\text{fixed}}(\beta_a) = \mathcal{H}_{\text{fixed}}(\beta_b). \tag{46.5}$$

The equality is a mathematical identity of declared fields. A software implementation may use hashes or serialized labels, but the mathematical object being compared is the field tuple in (46.2).

The finite-difference branch comparison is

$$\mathcal{H}_{\text{fixed}}(\beta_0 - h) = \mathcal{H}_{\text{fixed}}(\beta_0) = \mathcal{H}_{\text{fixed}}(\beta_0 + h), \tag{46.6}$$

while the differentiable fields are recomputed:

$$\mathcal{H}_{\text{diff}}(\beta_0 - h), \quad \mathcal{H}_{\text{diff}}(\beta_0), \quad \mathcal{H}_{\text{diff}}(\beta_0 + h) \quad \text{may differ.} \tag{46.7}$$

If a frozen field differs, the centered difference is comparing different functions. If a differentiable field is copied rather than recomputed, the centered difference is not evaluating the fixed fitting rule.

47 Finite-Difference Diagnostic

The finite-difference diagnostic checks that the value path and derivative path agree for the same branch. Choose a base parameter β_0. Build the branch once and save the structural choices

$$\begin{aligned}B = \{&\text{domains, points, basis, ranks, sweeps, shifts,}\\ &\text{defensive terms, ridge parameters,}\\ &\text{deterministic initialization rules}\}.\end{aligned} \tag{47.1}$$

Then evaluate the same scalar at perturbed parameters while reusing those structural choices and recomputing the parameter-dependent fitted values by the same fixed equations:

$$\widehat{\ell}_T(\beta_0 + he_i; B), \qquad \widehat{\ell}_T(\beta_0 - he_i; B). \tag{47.2}$$

That means the domains, points, ranks, shifts, and sweep order are unchanged, but the fitted core values

$$C_{t,k}(\beta_0 + he_i; B) \quad \text{and} \quad C_{t,k}(\beta_0 - he_i; B) \tag{47.3}$$

are recomputed from the same saved fitting rule. They are not copied from β_0. The centered finite difference is

$$D_i(h) = \frac{\widehat{\ell}_T(\beta_0 + he_i; B) - \widehat{\ell}_T(\beta_0 - he_i; B)}{2h}. \tag{47.4}$$

For the one-parameter minimal case, the same field is

$$D(h) = \frac{\widehat{\ell}_2(\beta_0 + h; B) - \widehat{\ell}_2(\beta_0 - h; B)}{2h}. \tag{47.5}$$

Compare it to the analytical derivative

$$G_i = \partial_i \widehat{\ell}_T(\beta_0; B). \tag{47.6}$$

The one-parameter derivative field is

$$G = \partial_\beta \widehat{\ell}_2(\beta_0; B). \tag{47.7}$$

Record

$$e_i(h) = |D_i(h) - G_i|, \tag{47.8}$$

and

$$e_i^{\mathrm{rel}}(h) = \frac{|D_i(h) - G_i|}{1 + |G_i|}. \tag{47.9}$$

For the one-parameter minimal case,

$$e(h) = |D(h) - G|, \qquad e_{\mathrm{rel}}(h) = \frac{|D(h) - G|}{1 + |G|}. \tag{47.10}$$

Use a small schedule such as

$$h \in \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \tag{47.11}$$

The step-size field is

$$H_{\mathrm{FD}} = \{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}. \tag{47.12}$$

The recompute-core rule is

$$C_{t,k}(\beta_0 \pm h; B) = \text{the core produced by the fixed fitting rule at } \beta_0 \pm h, \tag{47.13}$$

not the core copied from β_0. The report should be a declared mathematical object, not an informal output list. For the one-parameter $T = 2$ minimal case, record

$$\begin{aligned} \mathcal{R}_{\mathrm{FD}} = \{&\widehat{\ell}_2(\beta_0; B),\ G,\ (h_i, D(h_i), e(h_i), e_{\mathrm{rel}}(h_i))_{i=1}^4, \\ &I_-(h_i), I_+(h_i), K_-(h_i), K_+(h_i),\ W_i,\ \text{status}\}. \end{aligned} \tag{47.14}$$

Here

$$I_\pm(h_i) = \mathbf{1}\{B(\beta_0 \pm h_i) = B\} \tag{47.15}$$

checks fixed-field equality, and

$$K_\pm(h_i) = \mathbf{1}\{C_k(\beta_0 \pm h_i; B) \text{ was recomputed by the fixed fitting rule}\} \tag{47.16}$$

checks that perturbed cores were not copied from β_0. A decreasing error window is recorded by

$$W_i = \mathbf{1}\{e(h_i) > e(h_{i+1}) > e(h_{i+2})\}, \qquad i = 1, 2. \tag{47.17}$$

The status rule is

$$\text{status} = \begin{cases} \text{branch identity failure}, & \min_{i,\pm} I_\pm(h_i) = 0, \\ \text{copied-core failure}, & \min_{i,\pm} K_\pm(h_i) = 0, \\ \text{decreasing-window agreement}, & \max(W_1, W_2) = 1, \\ \text{inconclusive: no decreasing window}, & \text{otherwise}. \end{cases} \tag{47.18}$$

The status is a derivative-consistency diagnostic for the fixed branch. It is not a posterior-accuracy theorem, a rank-stability theorem, or an HMC convergence result.

$$\begin{aligned}\mathcal{N}_{\mathrm{FD}} = \{&\text{not exact posterior accuracy,}\\ &\text{not rank-stability certification,}\\ &\text{not HMC convergence certification}\}.\end{aligned} \tag{47.19}$$

A healthy derivative check typically shows decreasing error over a range of h, followed by roundoff or fitting-noise limitations. If the error does not decrease on any reasonable range, the derivative path is not yet aligned with the value path.

$$\exists i \in \{1,2\} : W_i = 1 \quad \Longleftrightarrow \quad \text{there is a decreasing finite-difference error window.} \tag{47.20}$$

The local branch identity condition is mandatory:

$$B(\beta_0) = B(\beta_0 + h) = B(\beta_0 - h) = B. \tag{47.21}$$

If the implementation rebuilds ranks, points, shifts, or domains at the perturbed values, then the centered difference is no longer assessing the derivative of $\widehat{\ell}_2(\beta; B)$. It is comparing a different adaptive calculation. If the implementation copies the numerical cores from β_0 without resolving the fixed fitting equations at the perturbed parameter, then it is also assessing the wrong object. The correct diagnostic freezes the branch choices but allows all differentiable outputs of those choices to change with β.

$$\begin{aligned}&\min_{i,\pm} I_{\pm}(h_i) = 0 \quad \Rightarrow \text{ the compared values use different branches,}\\ &\min_{i,\pm} K_{\pm}(h_i) = 0 \quad \Rightarrow \text{ the perturbed values copied old numerical cores,}\\ &\max(W_1, W_2) = 0 \quad \Rightarrow \text{ the diagnostic is inconclusive for derivative agreement.}\end{aligned} \tag{47.22}$$

48 Minimal Mathematical Example

The smallest useful mathematical example should be nonlinear but one-dimensional:

$$x_t = \beta x_{t-1} + \sigma\epsilon_t, \qquad \epsilon_t \sim N(0,1), \tag{48.1}$$

$$y_t = \sin(x_t) + \eta_t, \qquad \eta_t \sim N(0, r^2). \tag{48.2}$$

Use $T = 2$. The first time step defines

$$q_1(x_1, x_0; \beta) = p_\beta(x_0) f_\beta(x_1 \mid x_0) g_\beta(y_1 \mid x_1). \tag{48.3}$$

It fits ϕ_1, computes

$$\widehat{Z}_1(\beta) = e^{-c_1} \int \phi_1(z_1, z_0; \beta)^2 \, \mathrm{d}z_1 \, \mathrm{d}z_0 + \tau_1, \tag{48.4}$$

and stores

$$\widehat{p}_1(z_1; \beta) = \frac{\int \left[e^{-c_1} \phi_1(z_1, z_0; \beta)^2 + \tau_1 \lambda_1(z_1, z_0) \right] \mathrm{d}z_0}{\widehat{Z}_1(\beta)}. \tag{48.5}$$

The second time step depends on that stored object:

$$q_2(x_2, x_1; \beta) = \widehat{p}_1(x_1; \beta) f_\beta(x_2 \mid x_1) g_\beta(y_2 \mid x_2). \tag{48.6}$$

The scalar checked by the example is

$$\widehat{\ell}_2(\beta) = \log \widehat{Z}_1(\beta) + \log \widehat{Z}_2(\beta). \tag{48.7}$$

The numerical derivative table must contain

$$\widehat{\ell}_2(\beta_0), \qquad \partial_\beta \widehat{\ell}_2(\beta_0), \qquad D(h), \qquad |D(h) - \partial_\beta \widehat{\ell}_2(\beta_0)|. \tag{48.8}$$

It must also contain the fixed-branch record used for β_0, $\beta_0 + h$, and $\beta_0 - h$. The mathematical pass condition is that those fixed-branch records are identical and the derivative error has a stable decreasing window before numerical roundoff dominates.

49 Integrated Conclusion

The fixed-branch gradient is built from five elementary ideas:

normalizer derivative	$\partial \log Z = \dot{Z}/Z,$
squared density derivative	$\partial(\phi^2) = 2\phi\dot{\phi},$
mass contraction derivative	differentiate every factor once,
linear solve derivative	$N\dot{g} = \dot{d} - \dot{N}g,$
carried filter derivative	$\partial(a/Z) = \dot{a}/Z - a\dot{Z}/Z^2.$

(49.1)

The tensor-train notation makes these ideas look large, but it does not change their nature. Once the branch is fixed, the forward pass defines one scalar. The derivative pass differentiates that scalar. What remains for empirical work is to assess whether the approximation is accurate enough, stable enough, and useful enough for the target scientific model.

50 Four Checklists For Reading The Derivative

This section collects four compact checklists for the fixed-branch part. They state, after the derivation, what has been established and what a later implementation must preserve.

Squared-density block

established	$\widehat{q}_t = e^{-c_t}\phi_t^2 + \tau_t\lambda_t \geq 0,$
stored	ϕ_t through $C_{t,k}$ and $\dot{\phi}_t$ through $\dot{C}_{t,k},$
integrated	$e^{-c_t}\phi_t(z)^2 + \tau_t\lambda_t(z),$
differentiated	$e^{-c_t}\phi_t(z)^2 \mapsto 2e^{-c_t}\phi_t(z)\dot{\phi}_t(z),$
fixed	$c_t, \tau_t, \lambda_t,$ basis, domain, ranks, points,
failure mode	differentiating a different fitted square root than the one squared.

(50.1)

Mass-contraction block

established	$R_t = \int \phi_t(z)^2 \, \mathrm{d}z$ is computed by TT mass contractions,
stored	$C_{t,k}, \dot{C}_{t,k}, B_{t,k}, M_{>k}, \dot{M}_{>k}$,
integrated	ϕ_t^2 one coordinate at a time,
differentiated	each product factor once, then summed,
fixed	$B_{t,k}$, coordinate order, rank pattern, basis family,
failure mode	omitting a product-rule term in $\dot{M}_{>k}$.

(50.2)

Fixed-solve block

established	$N_k g_k = d_k \mapsto N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$,
stored	A_k, N_k, d_k, g_k and dotted counterparts,
integrated	nothing; this is a finite-dimensional fitting solve,
differentiated	the fixed normal equations,
fixed	$W, \rho, Z_{\mathrm{fit}}, B_{\mathrm{val}}$, sweep order,
failure mode	changing the solve system or copying $g_k(\beta_0)$ at perturbed parameters.

(50.3)

Carried-filter block

established	$\widehat{p}_t = a_t / \widehat{Z}_t$ and $\dot{\widehat{p}}_t = \dot{a}_t / \widehat{Z}_t - a_t \dot{\widehat{Z}}_t / \widehat{Z}_t^2$,
stored	$Q_t, \dot{Q}_t, P_t, \dot{P}_t$ with shape (p, p),
integrated	$\widehat{q}_t(z_t, z_{t-1})$ over z_{t-1},
differentiated	$Q_t / \widehat{Z}_t$ by the quotient rule,
fixed	basis, query coordinate, branch, defensive term,
failure mode	feeding time $t+1$ without $\dot{P}_t$ or with an undeclared query rule.

(50.4)

A Relation To Zhao And Cui

The main text reconstructs the displayed mathematical spine of Zhao and Cui [4] Sections 1–3 and 5. Equations (1)–(26) and (30)–(35), Algorithms 1–5, the notation formulas, the square-root error inequalities, the upper and lower conditional map formulas, the particle and smoothing weight normalizations, and the preconditioning composition formulas are represented in the notation of this note notation. Section 4 error propagation and Section 6 numerical examples are not reconstructed as the central object of this note; they are used only to explain what the method does and does not prove.

References

[1] Tiangang Cui and Sergey Dolgov. Deep composition of tensor-trains using squared inverse rosenblatt transports. *arXiv preprint arXiv:2007.06968*, 2021.

[2] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5): 2295–2317, 2011. doi: 10.1137/090752286.

[3] Murray Rosenblatt. Remarks on a multivariate transformation. *The Annals of Mathematical Statistics*, 23(3):470–472, 1952.

[4] Yiran Zhao and Tiangang Cui. Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25:1–51, 2024.